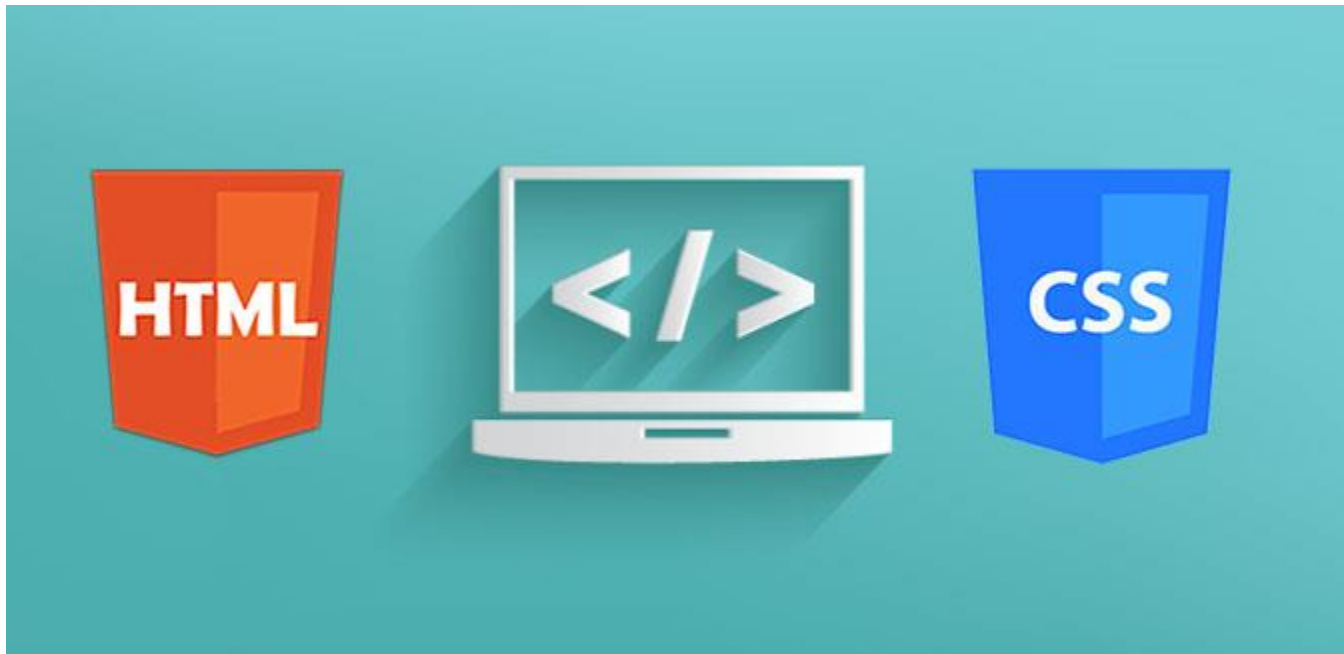


MODULE : DÉVELOPPEMENT WEB CÔTÉ CLIENT

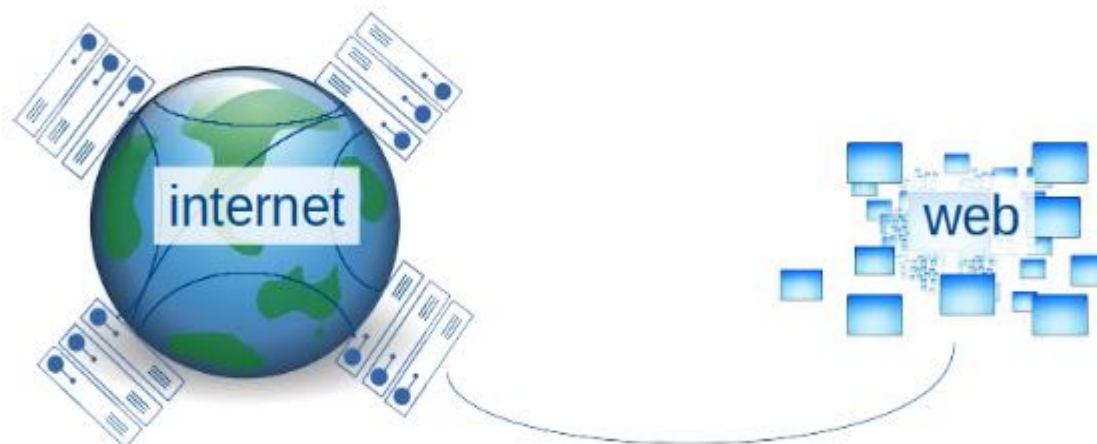
HTML et CSS



Prof. KICH Ismail

KICH ISMAIL

WEB vs Internet?



internet

- recherches ← 1960s
- système de réseaux d'ordinateurs interconnectés

web (World Wide Web)

- ← 1989
- système hypertexte fonctionnant sur internet
- = une des applications d'internet, comme courrier électronique, messagerie instantanée...

Principe

Client



URL (navigateur)

<http://perso.limsi.fr/individu/annlor/enseignement.html>



Moi

Communication



Serveur



Serveur du LIMSI

URL = Uniform Resource Locator

identifie sur le web un document ou un fragment

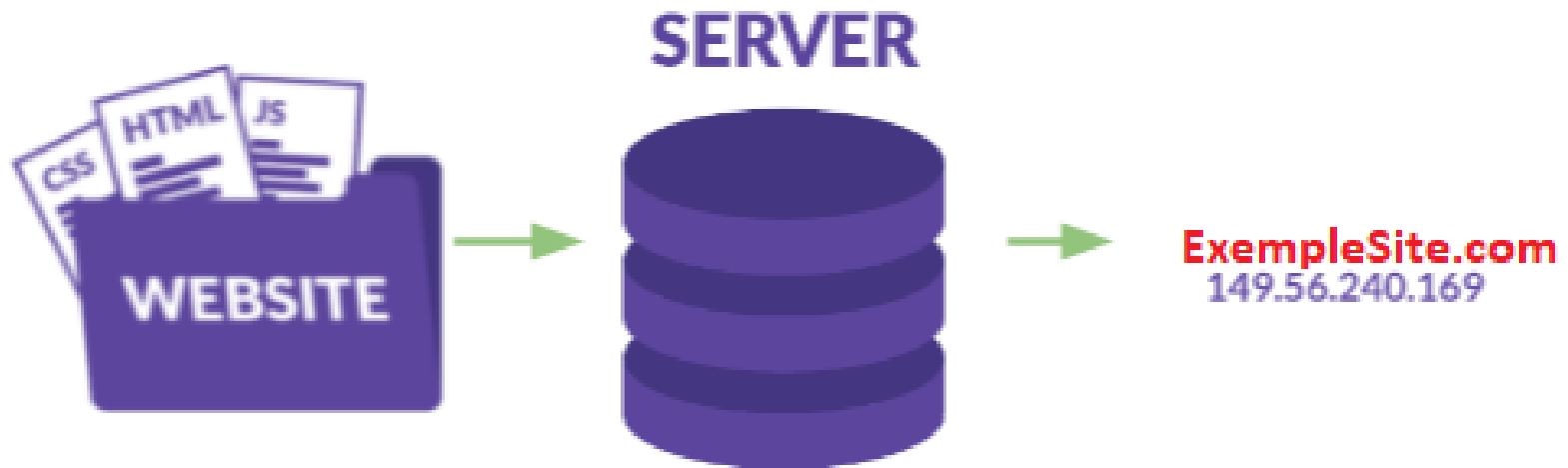
<http://www.limsi.fr/Individu/annlor/enseignement.html#pw>
 protocole machine répertoire fichier fragment

Navigateurs

- ☐ Navigateurs graphiques (IE, Firefox, Chrome...)
 - Ordinateurs,
 - Tablettes,
 - Téléphones.
- ☐ Navigateurs textuels (links, linx...),
- ☐ Navigateurs avec synthèse vocale,
- ☐ Robots.



Comment fonctionne le **web**?



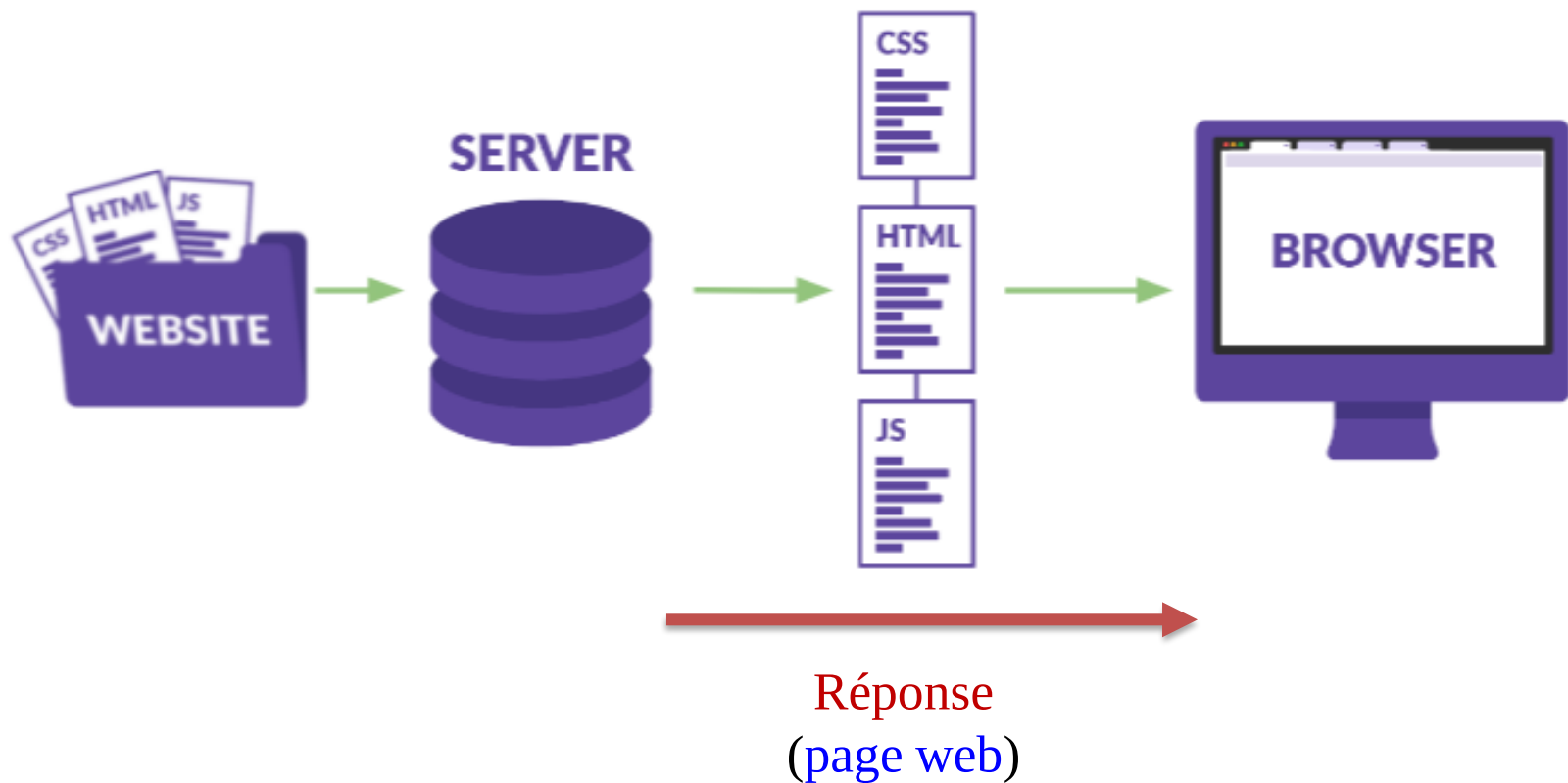
Serveur = ordinateur qui stocke
et fournit le site web

Comment fonctionne le **web**?



Client = ordinateur utilise,
généralement via le navigateur
(Firefox, Chrome, Safari,
Internet Explorer...)

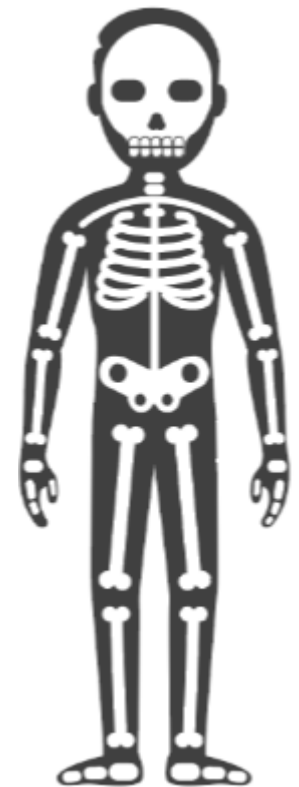
Comment fonctionne le **web**?



Comment fonctionne le **web**?

Html : Le squelette

- Html fournit des « **balises** » (**tag**) pour décrire différents types de contenu (éléments) sur vos sites Web, par exemple : Cela permet à votre navigateur de comprendre ce qu'il lit et comment le restituer.
<header>, **<link>**, **<div>**,...
- Alors que votre navigateur peut rendre le **HTML** par lui-même, vous pouvez le rendre dynamique et beau en utilisant **JavaScript** et **CSS**. Alors, pensez au **HTML** comme à un squelette qui prend vie avec **JavaScript** et **CSS**.



Comment fonctionne le **web**?

CSS : La peau

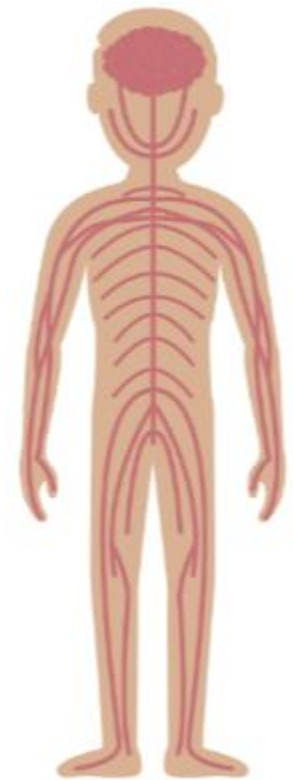
- CSS rendra notre site présentable et améliorera l'expérience utilisateur pour les visiteurs de notre site.
- CSS vous permet d'ajouter de la couleur, de modifier les tailles, d'intégrer des images d'arrière-plan et de rendre votre conception réactive pour le bureau et la visualisation.
- Soit vous aspirez à devenir un expert en **CSS** ou **JavaScript** Ninja, une compréhension approfondie du **HTML** est indispensable.



Comment fonctionne le **web**?

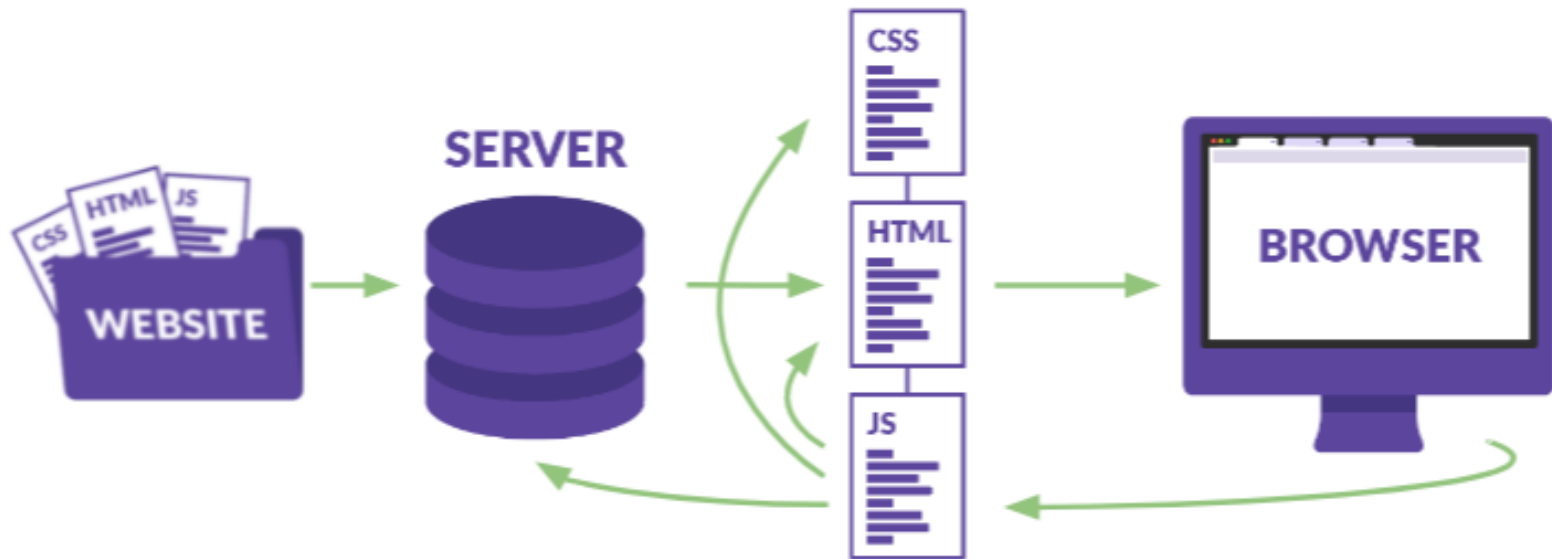
JavaScript : le cerveau

- JavaScript fournira le cerveau et les muscles au site Web, lui permettant d'interagir et de restituer le contenu HTML en fonction des entrées de l'utilisateur ou des données d'un serveur.
- Cela rend votre **HTML dynamique** plutôt que statique. Les jeux et les applications interactives peuvent être construits sur HTML grâce à la puissance de **JavaScript**



Comment fonctionne le **web**?

JAVASCRIPT
MAKES SITE DYNAMIC



Qu'est ce que HTML?

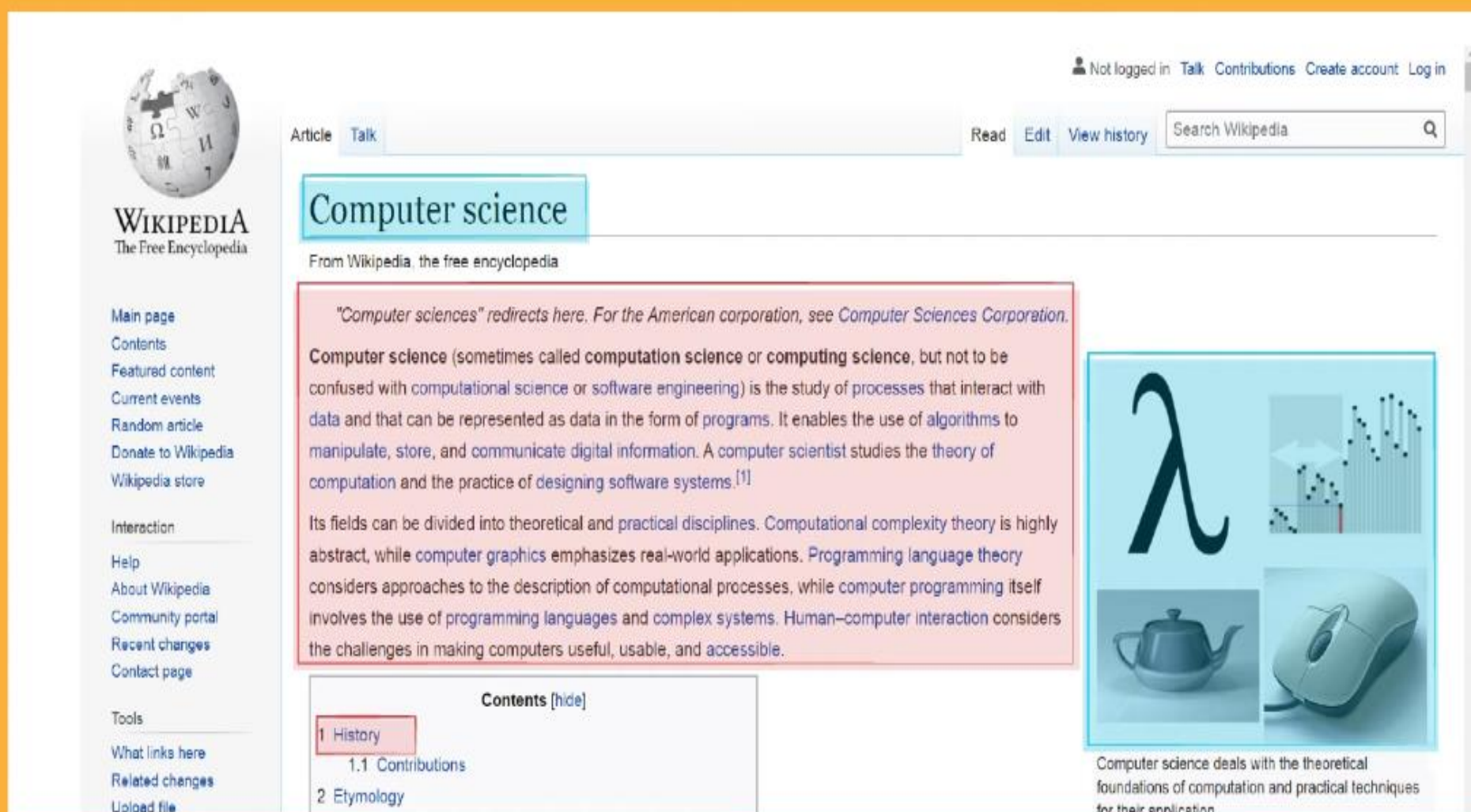
HTML est l'abréviation de **HyperText Markup Language**, soit en français « langage hypertexte de balisage ». Ce langage a été créé en 1991 et a pour fonction de structurer et de donner du sens à du contenu.

- Langage de description de pages Web.
- Présenter les documents hypertextes destinés à être affichés sur le navigateur.
- Un langage coté client (tout comme **CSS** et **Javascript**).
- Supporté et développé par W3C (World Wide Web Consortium).

HTML est un langage de **Balisages**

- ❑ **Balise** : unité syntaxique délimitant une séquence de caractères ou marquant une position précise à l'intérieur d'un flux de caractères,
- ❑ L'inclusion de **balises** permet de transférer à la fois la **structure** du document et son contenu. Cette structure est compréhensible par un **programme informatique (Navigateurs)**, ce qui permet un traitement automatisé du contenu.

Balises HTML



The screenshot shows the Wikipedia article for "Computer science". The page layout includes a sidebar on the left with navigation links, a top navigation bar with user status and search, and a main content area. The article title "Computer science" is highlighted in a light blue box. Below the title, a red box contains a redirect notice: "Computer sciences" redirects here. For the American corporation, see Computer Sciences Corporation. The main text defines computer science as the study of processes that interact with data and can be represented as data in the form of programs. It mentions the use of algorithms to manipulate, store, and communicate digital information. A computer scientist studies the theory of computation and the practice of designing software systems.^[1] The text is followed by a paragraph discussing the fields of computer science, divided into theoretical and practical disciplines. It mentions computational complexity theory, computer graphics, programming language theory, computer programming, and human-computer interaction. A "Contents" section is visible at the bottom, with "1 History" highlighted in a red box. To the right of the main text, there is a blue box containing images of a lambda symbol, a DNA helix, a teapot, and a computer mouse, with a caption below them.

Not logged in | Talk | Contributions | Create account | Log in

Article | Talk | Read | Edit | View history | Search Wikipedia

Computer science

From Wikipedia, the free encyclopedia

"Computer sciences" redirects here. For the American corporation, see Computer Sciences Corporation.

Computer science (sometimes called **computation science** or **computing science**, but not to be confused with computational science or software engineering) is the study of processes that interact with data and that can be represented as data in the form of programs. It enables the use of algorithms to manipulate, store, and communicate digital information. A computer scientist studies the theory of computation and the practice of designing software systems.^[1]

Its fields can be divided into theoretical and practical disciplines. Computational complexity theory is highly abstract, while computer graphics emphasizes real-world applications. Programming language theory considers approaches to the description of computational processes, while computer programming itself involves the use of programming languages and complex systems. Human-computer interaction considers the challenges in making computers useful, usable, and accessible.

Contents [hide]

- 1 History
 - 1.1 Contributions
- 2 Etymology

Computer science deals with the theoretical foundations of computation and practical techniques for their application.

Balises (Tags)

Balises HTML

```

1 <!DOCTYPE html>
2 <html class="client-nojs" lang="fr" dir="ltr">
3 <head>
4 <meta charset="UTF-8"/>
5 <title>Langage de balisage – Wikipédia</title>
6 <script type="text/javascript" src="https://ff.kis.v2.scr.kaspersky-labs.com/FD126C42-EBFA-4E12-B309-BB3FDD723AC1/main.js?attr=AoLrRad7Jzc306v-94OrZnOyXo8lHkMmf
7 "wgIsProbablyEditable":!0,"wgRelevantPageIsProbablyEditable":!0,"wgRestrictionEdit":[],"wgRestrictionMove":[],"wgMediaViewerOnClick":!0,"wgMediaViewerEnabledByD
8 "mediawiki.ui.button":"ready","skins.vector.styles":"ready","skins.vector.icons":"ready","mediawiki.ui.icon":"ready","ext.visualEditor.desktopArticleTarget.noscr
9 <script>(RLQ=window.RLQ||[]).push(function(){mw.loader.implement("user.options@1hzgi",function($,jQuery,require,module){/*@nomin*/mw.user.tokens.set({"patrolTok
10 });});</script>
11 <link rel="stylesheet" href="/w/load.php?lang=fr&modules=ext.cite.styles%7Cext.pygments%2CwikimediaBadges%7Cext.uls.interlanguage%7Cext.visualEditor.desktop
12 <script async="" src="/w/load.php?lang=fr&modules=startup&only=scripts&raw=1&skin=vector"></script>
13 <meta name="ResourceLoaderDynamicStyles" content="">
14 <link rel="stylesheet" href="/w/load.php?lang=fr&modules=site.styles&only=styles&skin=vector"/>
15 <meta name="generator" content="MediaWiki 1.38.0-wmf.6"/>
16 <meta name="referrer" content="origin"/>
17 <meta name="referrer" content="origin-when-crossorigin"/>
18 <meta name="referrer" content="origin-when-cross-origin"/>
19 <meta name="format-detection" content="telephone=no"/>
20 <meta property="og:title" content="Langage de balisage – Wikipédia"/>
21 <meta property="og:type" content="website"/>
22 <link rel="preconnect" href="//upload.wikimedia.org"/>
23 <link rel="alternate" media="only screen and (max-width: 720px)" href="//fr.m.wikipedia.org/wiki/Langage_de_balisage"/>
24 <link rel="alternate" type="application/x-wiki" title="Modifier" href="/w/index.php?title=Langage_de_balisage&action=edit"/>
25 <link rel="apple-touch-icon" href="/static/apple-touch/wikipedia.png"/>
26 <link rel="shortcut icon" href="/static/favicon/wikipedia.ico"/>
27 <link rel="search" type="application/opensearchdescription+xml" href="/w/opensearch_desc.php" title="Wikipédia (fr)"/>
28 <link rel="EditURI" type="application/rsd+xml" href="//fr.wikipedia.org/w/api.php?action=rsd"/>
29 <link rel="license" href="https://creativecommons.org/licenses/by-sa/3.0/">
30 <link rel="canonical" href="https://fr.wikipedia.org/wiki/Langage_de_balisage"/>
31 <link rel="dns-prefetch" href="//meta.wikimedia.org" />
32 <link rel="dns-prefetch" href="//login.wikimedia.org"/>
33 </head>

```

Balises (Tags)

Balises HTML

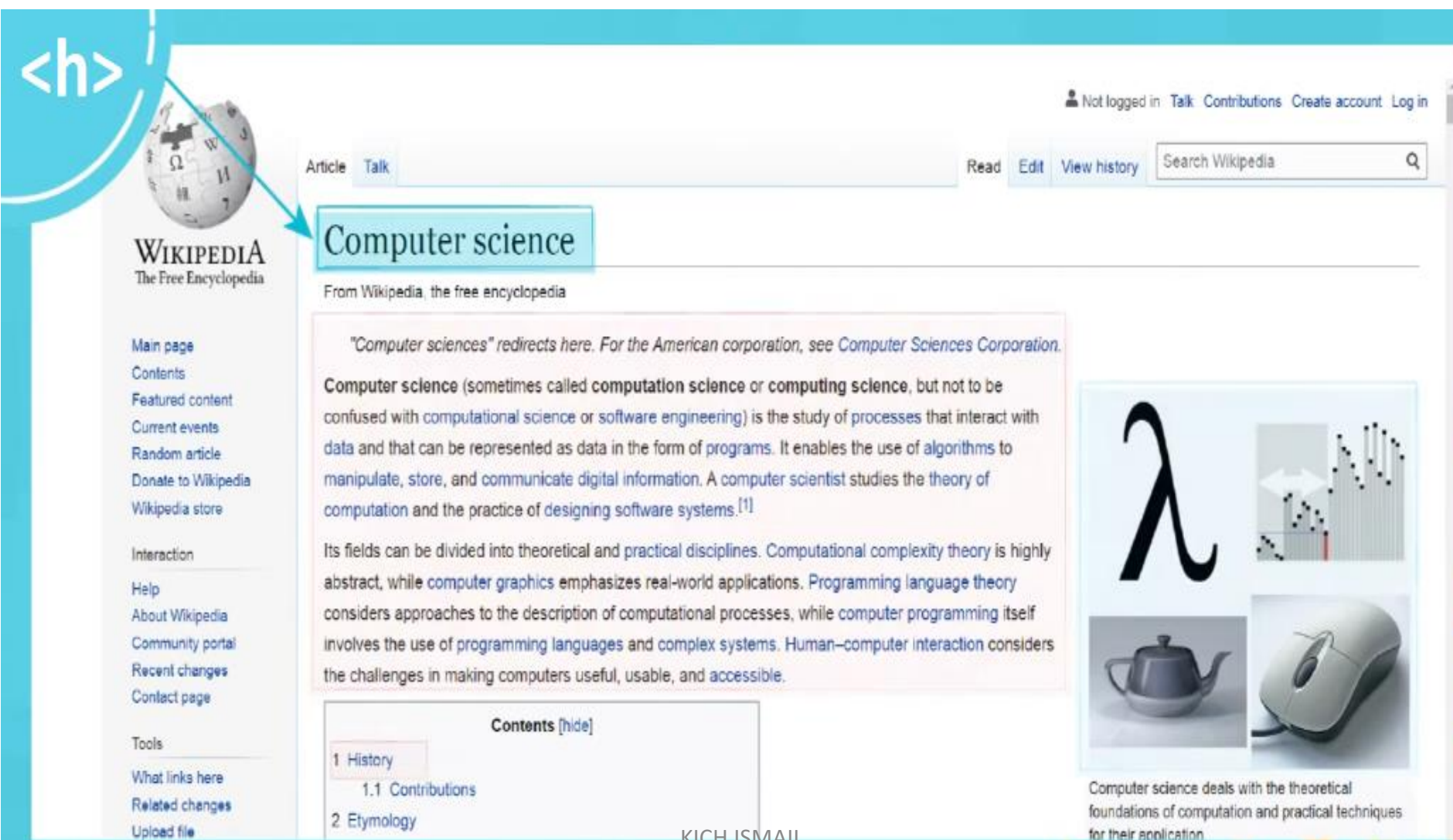
```

34 <body class="mediawiki ltr sitedir-ltr mw-hide-empty-elt ns-0 ns-subject mw-editable page-Langage_de_balisage rootpage-Langage_de_balisage skin-
35 <a class="mw-jump-link" href="#content">Aller au contenu</a>
36 <div class="mw-page-container-inner">
37
38 <input
39   type="checkbox"
40   id="mw-sidebar-checkbox"
41   class="mw-checkbox-hack-checkbox"
42   checked>
43
44 <header class="mw-header">
45   <label
46     id="mw-sidebar-button"
47     class="mw-checkbox-hack-button mw-ui-icon mw-ui-button mw-ui-quiet mw-ui-icon-element"
48     for="mw-sidebar-checkbox"
49     role="button"
50     aria-controls="mw-panel"
51     data-event-name="ui.sidebar"
52     tabindex="0"
53     title="Menu principal">
54     Afficher / masquer la barre latérale
55   </label>
56   <a href="/wiki/Wikip%C3%A9dia:Accueil_principal" class="mw-logo">
57   
59   <span class="mw-logo-container">
60     
62     
65   </span>
66 </a>

```


Balises HTML

<h>



The screenshot shows the Wikipedia page for "Computer science". The title "Computer science" is highlighted with a blue box. The first paragraph is enclosed in a red box. A blue arrow points from the "<h>" label to the title. The page includes a sidebar with navigation links, a top navigation bar with user options and a search bar, and a "Contents" section at the bottom.

WIKIPEDIA
The Free Encyclopedia

Article Talk

Read Edit View history Search Wikipedia

Computer science

From Wikipedia, the free encyclopedia

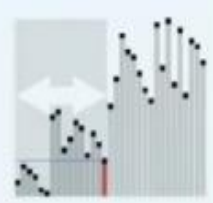
"Computer sciences" redirects here. For the American corporation, see Computer Sciences Corporation.

Computer science (sometimes called **computation science** or **computing science**, but not to be confused with **computational science** or **software engineering**) is the study of processes that interact with data and that can be represented as data in the form of *programs*. It enables the use of *algorithms* to manipulate, store, and communicate digital information. A computer scientist studies the theory of computation and the practice of designing software systems.^[1]

Its fields can be divided into theoretical and practical disciplines. Computational complexity theory is highly abstract, while **computer graphics** emphasizes real-world applications. **Programming language theory** considers approaches to the description of computational processes, while **computer programming** itself involves the use of programming languages and complex systems. **Human-computer interaction** considers the challenges in making computers useful, usable, and accessible.

Contents [hide]

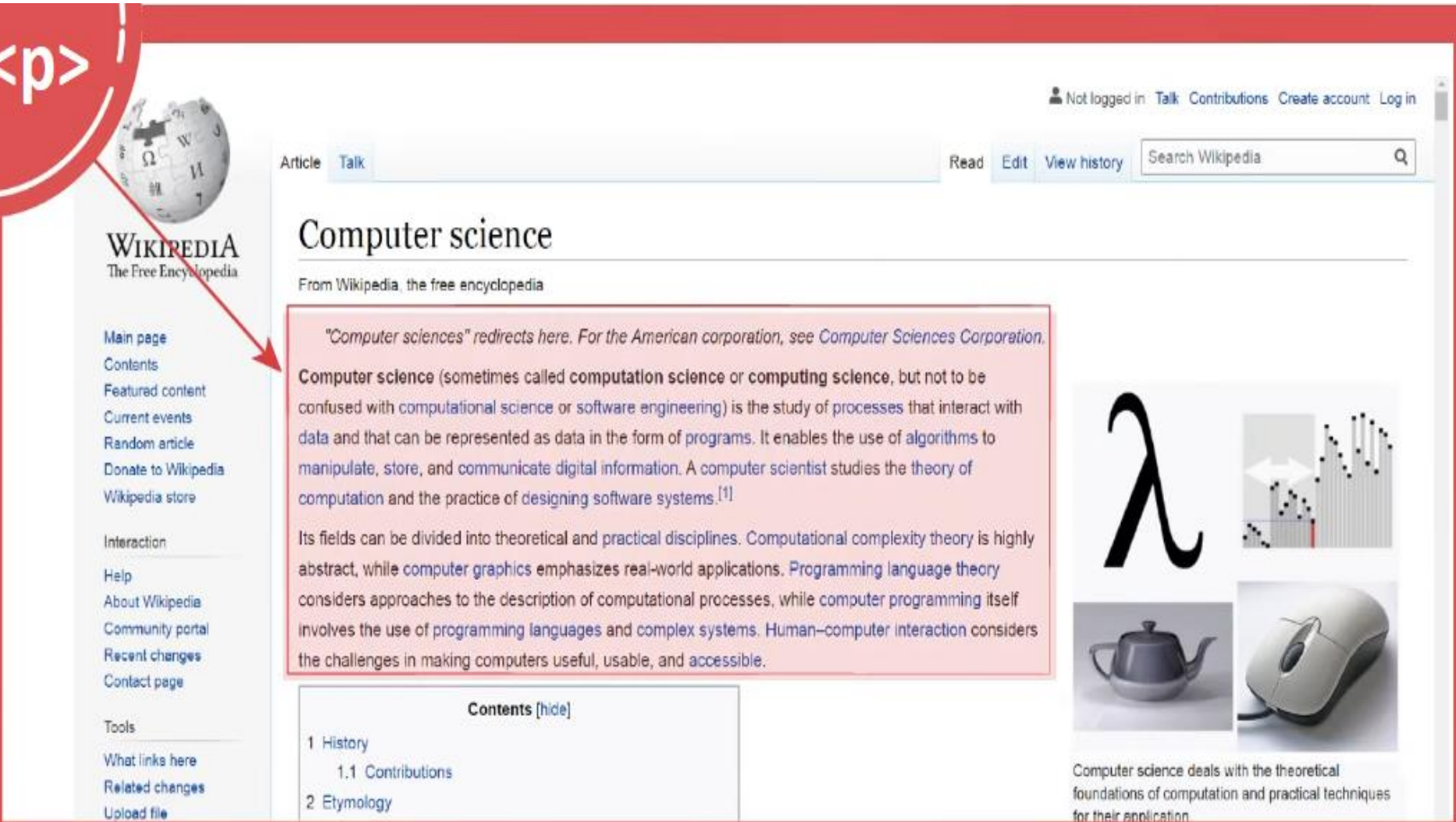
- 1 History
 - 1.1 Contributions
- 2 Etymology



Computer science deals with the theoretical foundations of computation and practical techniques for their application.

Balises HTML

<p>



The screenshot shows the Wikipedia article for "Computer science". A red circle on the left contains the HTML tag **<p>**, with a red arrow pointing to the first paragraph of the article, which is enclosed in a red rectangular box. The article text is as follows:

"Computer sciences" redirects here. For the American corporation, see Computer Sciences Corporation.

Computer science (sometimes called **computation science** or **computing science**, but not to be confused with **computational science** or **software engineering**) is the study of processes that interact with data and that can be represented as data in the form of programs. It enables the use of algorithms to manipulate, store, and communicate digital information. A computer scientist studies the theory of computation and the practice of designing software systems.^[1]

Its fields can be divided into theoretical and practical disciplines. Computational complexity theory is highly abstract, while **computer graphics** emphasizes real-world applications. **Programming language theory** considers approaches to the description of computational processes, while computer programming itself involves the use of programming languages and complex systems. Human-computer interaction considers the challenges in making computers useful, usable, and accessible.

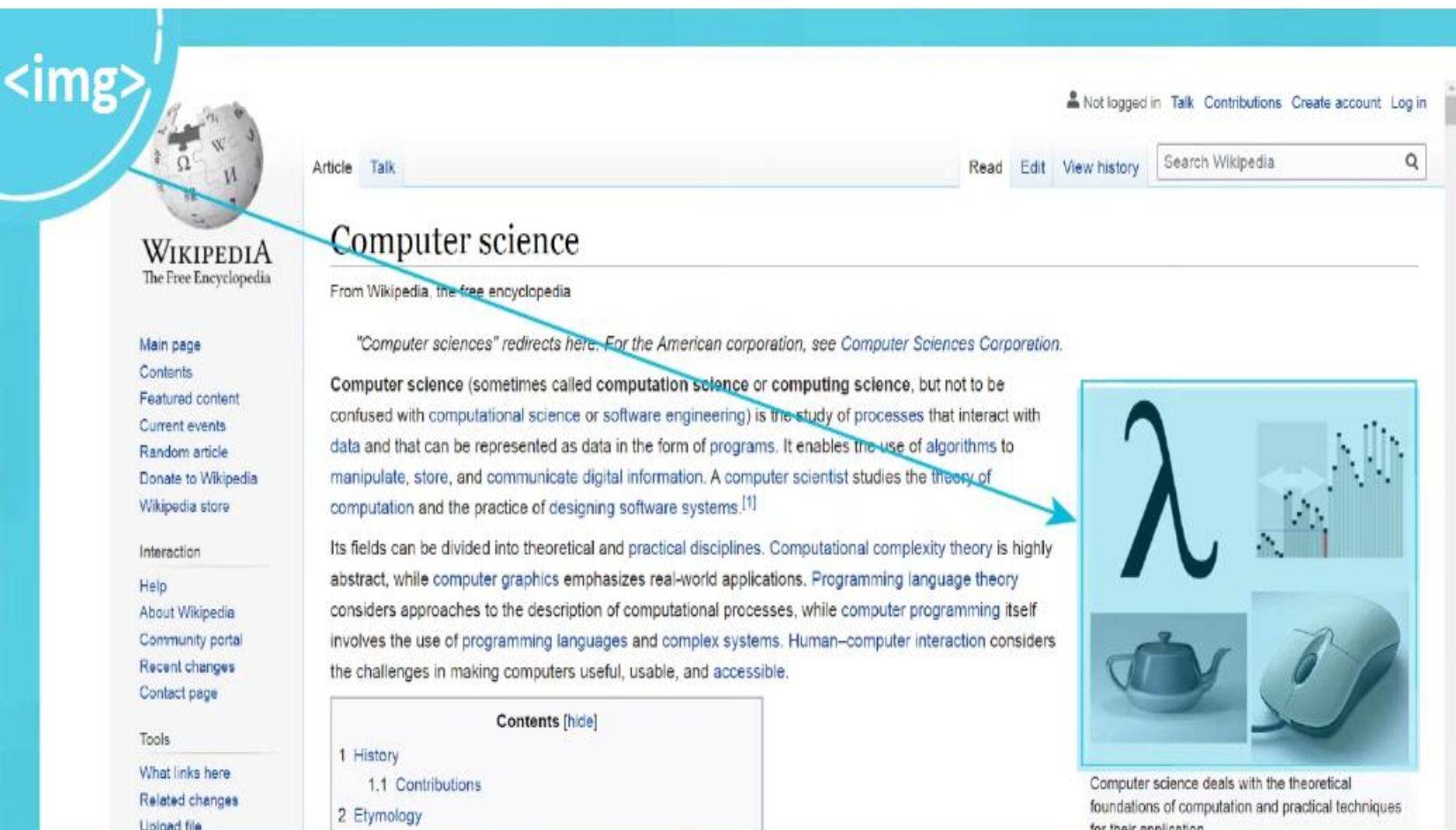
Contents [hide]

- 1 History
 - 1.1 Contributions
- 2 Etymology

On the right side of the article, there are four images: the Greek letter lambda (λ), a line graph, a teapot, and a computer mouse. Below these images is the text: "Computer science deals with the theoretical foundations of computation and practical techniques for their application."

Balises HTML

`<img>`



The screenshot shows the Wikipedia article for "Computer science". A blue arrow originates from the `<img>` tag in the top left corner and points to a gallery of four images on the right side of the article. The gallery contains: a large black lambda symbol (λ), a diagram of a signal waveform, a blue teapot, and a computer mouse. Below the gallery, a caption reads: "Computer science deals with the theoretical foundations of computation and practical techniques for their application".

WIKIPEDIA
The Free Encyclopedia

Main page
Contents
Featured content
Current events
Random article
Donate to Wikipedia
Wikipedia store

Interaction
Help
About Wikipedia
Community portal
Recent changes
Contact page

Tools
What links here
Related changes
Upload file

Article Talk

Read Edit View history

Search Wikipedia

Computer science

From Wikipedia, the free encyclopedia

"Computer sciences" redirects here. For the American corporation, see Computer Sciences Corporation.

Computer science (sometimes called **computation science** or **computing science**, but not to be confused with computational science or software engineering) is the study of processes that interact with data and that can be represented as data in the form of programs. It enables the use of algorithms to manipulate, store, and communicate digital information. A computer scientist studies the theory of computation and the practice of designing software systems.^[1]

Its fields can be divided into theoretical and practical disciplines. Computational complexity theory is highly abstract, while computer graphics emphasizes real-world applications. Programming language theory considers approaches to the description of computational processes, while computer programming itself involves the use of programming languages and complex systems. Human–computer interaction considers the challenges in making computers useful, usable, and accessible.

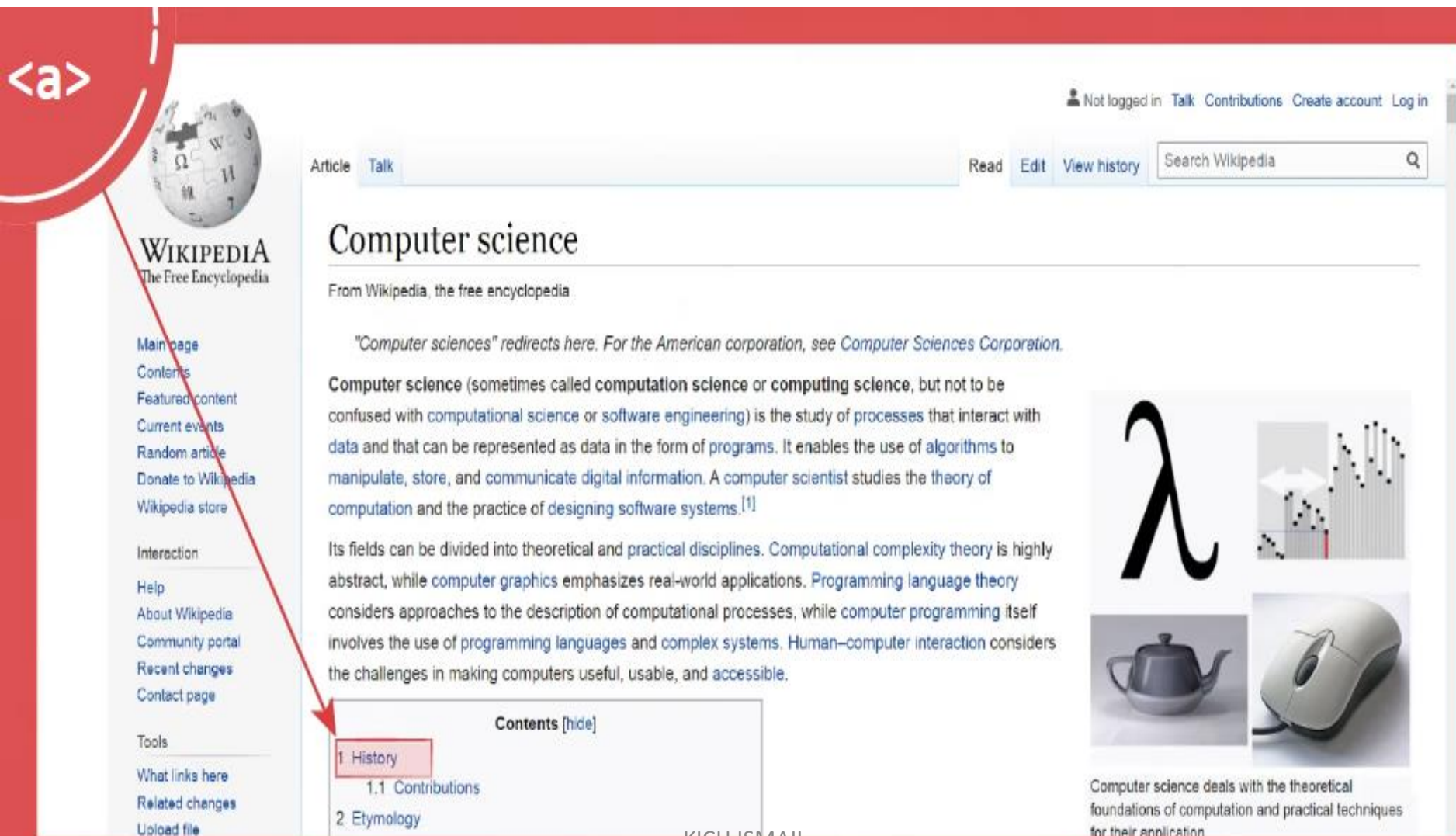
Contents [hide]

- History
 - Contributions
- Etymology

Computer science deals with the theoretical foundations of computation and practical techniques for their application

Balises HTML

<a>



The screenshot shows the Wikipedia article for "Computer science". On the left, a red circle highlights the HTML tag `<a>` with an arrow pointing to the "History" link in the "Contents" table of contents. The Wikipedia logo and navigation links are visible on the left sidebar. The article title "Computer science" is at the top, followed by the subtitle "From Wikipedia, the free encyclopedia". The main text begins with a redirect notice and a definition of computer science. The "Contents" table of contents is shown below the text, with "1 History" highlighted. On the right, there are four images: the Greek letter lambda (λ), a line graph, a teapot, and a computer mouse. Below these images is a caption: "Computer science deals with the theoretical foundations of computation and practical techniques for their application."

WIKIPEDIA
The Free Encyclopedia

Main page
Contents
Featured content
Current events
Random article
Donate to Wikipedia
Wikipedia store

Interaction
Help
About Wikipedia
Community portal
Recent changes
Contact page

Tools
What links here
Related changes
Upload file

Article Talk

Read Edit View history

Search Wikipedia

Computer science

From Wikipedia, the free encyclopedia

"Computer sciences" redirects here. For the American corporation, see Computer Sciences Corporation.

Computer science (sometimes called **computation science** or **computing science**, but not to be confused with **computational science** or **software engineering**) is the study of processes that interact with data and that can be represented as data in the form of **programs**. It enables the use of **algorithms** to manipulate, store, and communicate **digital information**. A computer scientist studies the theory of computation and the practice of **designing software systems**.^[1]

Its fields can be divided into theoretical and practical disciplines. Computational complexity theory is highly abstract, while **computer graphics** emphasizes **real-world applications**. **Programming language theory** considers approaches to the description of computational processes, while computer programming itself involves the use of **programming languages** and complex systems. **Human–computer interaction** considers the challenges in making computers useful, usable, and accessible.

Contents [hide]

- History
 - Contributions
- Etymology

λ

Computer science deals with the theoretical foundations of computation and practical techniques for their application.

Généralités

HTML = **H**yper**T**ext **M**arkup **L**anguage

- Langage de balisage Hyper-Texte : texte qui contient des liens vers d'autres textes
- Langage de présentation de documents sur le Web,
- Un document **HTML** est un fichier **ASCII** (texte) composé de:
 - Contenu utilisateur,
 - Balises.
- Un navigateur (browser) permet d'interpréter les balises et d'afficher le document HTML : **Page Web**.

Balises HTML

- Balise = Texte entouré par < et > (chevrons)
- Une balise marque une action pour le navigateur : Ce qu'il doit faire ?
- Balises insensibles à la casse :
 - <BALISE>
 - <balise>
 - <Balise>
 - <bAlIsE>
- Ouvrante : <balise>
- Fermante : </balise>
- Non visibles dans le rendu du navigateur
- Attributs/options : <balise attribut="valeur">

Historique des versions

HTML	1991
HTML 2.0	1995
HTML 3.2	1997
HTML 4.01	1999
XHTML 1.0	2000
<u>HTML 5</u>	2014

Utilisation des balises

☐ Utilisation CORRECTE

<i> Texte **** Texte Texte **** **</i>**

☐ Utilisation INCORRECTE

<i> Texte **** Texte Texte **</i>** ****

☐ Jamais de chevauchement

Structure d'une page HTML

```
<html>
```

```
<head>
```

```
<title>Page title</title>
```

```
</head>
```

```
<body>
```

```
<h1>This is a heading</h1>
```

```
<p>This is a paragraph.</p>
```

```
<p>This is another paragraph.</p>
```

```
</body>
```

```
</html>
```

Structure d'une page HTML

Structure d'un **document HTML** :

1. Information sur la version de HTML utilisée
2. En-tête déclaratif
3. Corps, contenu du document

Balises de structure

- ❑ Racine du document :

<html> ... </html>

- ❑ En-tête du document :

<head> ... </head>

- Titre descriptif du document :

<title> ... </title>

- ❑ Corps du document :

<body> ... </body>

Environnement de travail HTML

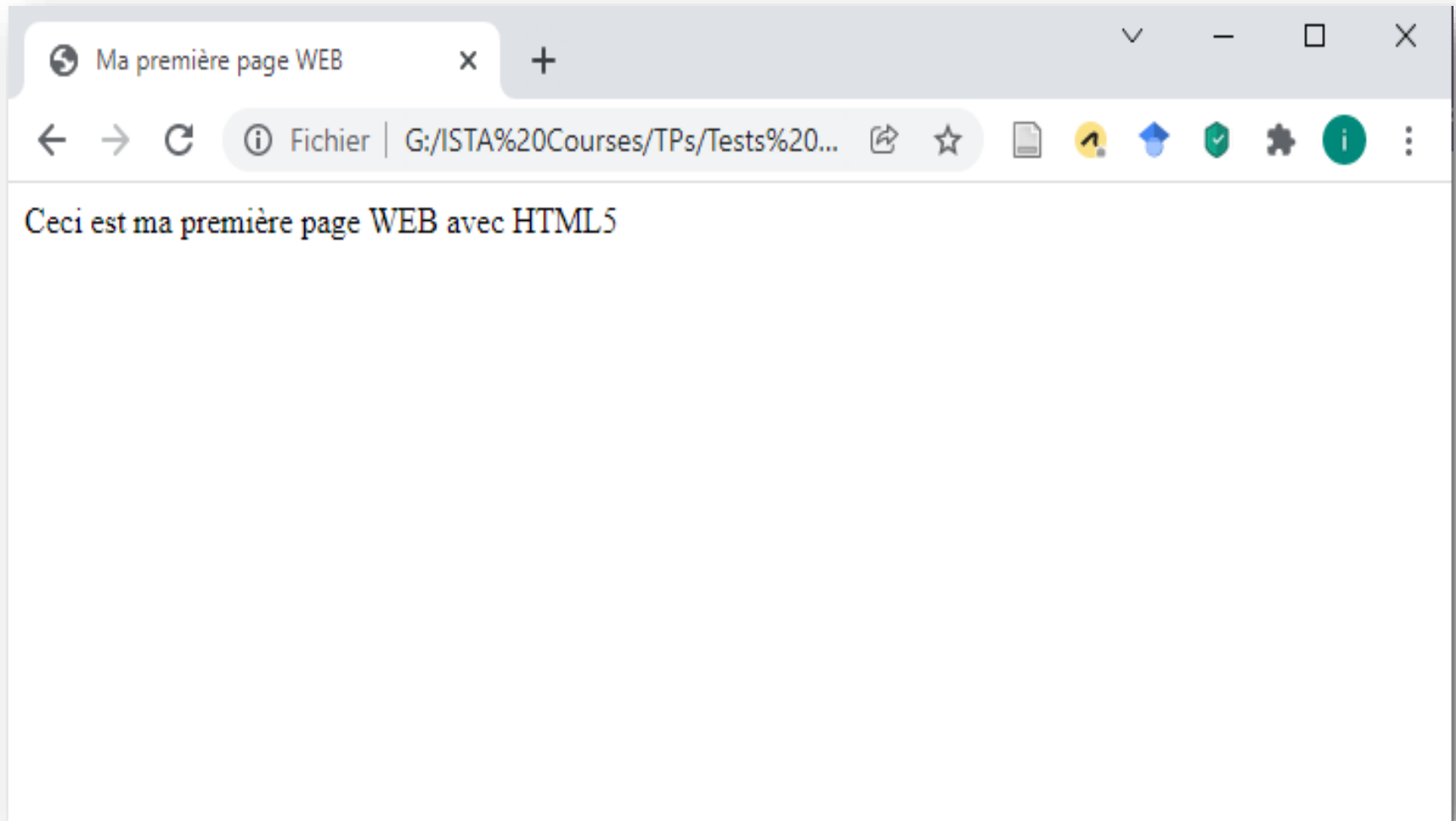
- ❑ Navigateur : Firefox, Google Chrome, Internet Explorer, ...
- ❑ Editeur de texte : Notepad, Notepad++, SublimeText, Visual Studio Code, ...

Activité 1 :

Créer une page Web contenant les éléments suivants :

- Le titre de la page « Ma première page WEB »,
- Le texte « ceci est ma première page WEB avec le langage HTML version 5.

1^{er} page WEB



Solution 1:

```
<html>
  <head>
    <title>Ma première page WEB</title>
  </head>
  <body>
    Ceci est ma première page WEB avec HTML5
  </body>
</html>
```

Balises (Tags)

Text

Headings

Lists

Links

Images

Objects

Tables

Forms



Balises (Tags)

Balises de formatage et mise en page

- `<b> text </b>` **Gras**
 - `<i> text </i>` *italique*
 - `<u> text </u>` Texte souligné
 - `<S> text </S>` ~~Texte barré~~
 - `<SUB> text </SUB>` Texte _{en indice}
 - `<SUP> text </SUP>` Texte ^{en exposant}
 - `<Center> text </Center>` Texte en centre
 - `<br>` Saut de ligne,
 - `<hr>` ligne horizontale,
- `<hr align="alignement" size=épaisseur width = longueur>`

Balises (Tags)

Balises de formatage et mise en page

- `<p> .... </p>` Paragraphe
 `<p align= "alignement">`
 Paragraphe....
 `</p>`
- `<div>.....</div>`
 `<div align= "alignement">.....</div>`
- `<pre> .....</pre>` Texte préformaté (brut)
- `<span>....</span>` formatage spécial à un ensemble de mots
- `<font> text </font>`
 `<font size="5" color="red " face="arial"> text </font>`

HEADING

h1 heading

h2 heading

h3 heading

h4 heading

h5 heading

h6 heading

Balises (Tags)

Titres (6 niveaux de hiérarchie) :

`<h1> Titre n° 1</h1>`

`<h2> Titre n° 2</h2>`

`⋮`

`<h6> Titre n° 6</h6>`

`<Hn ALIGN = "LEFT/RIGHT/ CENTER/JUSTIFY"> texte </Hn>`

Où **n=1,2,3,4,5 ou 6.**

<https://www.w3schools.com/html/>

HTML Lists

- `<ul>` : unordered list (Liste non numérotée)
- `<ol>` : ordered list (Liste numérotée)
- `<dl>` : definition list (Liste de définitions)

Balises (Tags)

HTML Lists

- `<ul>` : unordered list

```
<ul>
```

```
<li> text1</li>
```

```
<li> text2</li>
```

```
<li> text3</li>
```

```
</ul>
```

Balises (Tags)

HTML Lists

- `<ol>` : ordered list

```
<ol>  
  <li> text1</li>  
  <li> text2</li>  
  <li> text3</li>  
</ol>
```

Balises (Tags)

HTML Lists

- `<dl>` : definition list

```
<dl>
```

```
    <dt> text1</dt>
```

```
        <dd> text1 : .....</dd>
```

```
    <dt> text2</dt>
```

```
        <dd> text2 : .....</dd>
```

```
</dl>
```

Entités HTML et Caractères spéciaux

- ❑ Certains caractères sont réservés en HTML.
- ❑ Si vous utilisez les signes inférieur à (<) ou supérieur à (>) dans votre texte, le navigateur peut les mélanger avec des balises.
- ❑ Les entités caractères sont utilisées pour afficher des caractères réservés en HTML.
- ❑ Une entité de caractère ressemble à ceci :

&entity_name;

OR

&#entity_number

Balises (Tags)

Entités HTML et Caractères spéciaux

Exemple:

<	less than	<	<
>	greater than	>	>
&	ampersand	&	&
"	double quotation mark	"	"
€	euro	€	€
£	pound	£	£
©	copyright	©	©

Entités HTML et Caractères spéciaux

Exemple:

←	←	←	LEFTWARDS ARROW
↑	↑	↑	UPWARDS ARROW
→	→	→	RIGHTWARDS ARROW
↓	↓	↓	DOWNWARDS ARROW

Emojis dans HTML

Exemple:

	<code>&#128512;</code>
	<code>&#128513;</code>
	<code>&#128514;</code>
	<code>&#128515;</code>
	<code>&#128516;</code>
	<code>&#128517;</code>

Balises (Tags)

`<body>`

`<h1> Euro </h1>`

`<p> je vais afficher €</p>`

`<p> je vais afficher €</p>`

`<h1> Pound </h1>`

`<p> je vais afficher £</p>`

`<p> je vais afficher £</p>`

`</body>`

Balises (Tags)

<body>

<h1> Les trois premières lettres de alphabet </h1>

<p> A B C</p>

<p> A B C</p>

<h1> Mon premier émoji</h1>

<p>😀</p>

<h1> Plusieurs émojis</h1>

<p>😀 😄 😍 💗</p>

</body>

Tableaux

Tableau : **<table> ... </table>**

Ligne : **<tr> ... [</tr>]**

Cellule : **<td> ... [</td>]**

Cellule d'en-tête : **<th> ... [</th>]**

Ligne **<tr>** = Ensemble de colonnes (cellules **<td>**)

Balises (Tags)

Structure d'un Tableaux

```

<TABLE>
  <TR>
    <TH> En-tête 1 </TH>
    ...
    <TH> En-tête n </TH>
  </TR>
  <TR>
    <TD> Élément de colonne 1 </TD>
    ...
    <TD> Élément de colonne n </TD>
  </TR>
  ...
  <TR>
    <TD> Élément de colonne 1 </TD>
    ...
    <TD> Élément de colonne n </TD>
  </TR>
</TABLE>

```

En-tête 1
 Ligne 1
 En-tête n

 Cellule 1
 Ligne 2
 Cellule n

 Cellule 1
 Ligne m
 Cellule n

Structure d'un Tableaux

Exercice : Donner le code HTML du Tableau suivant

Matière	Trimestre	Volume Horaire
HTML	1	20
Java Script	2	30
ASP.NET	2	40

Balises (Tags)

Correction:

```

<TABLE>
  <TR>
    <TH> Matière </TH>
    <TH> Trimestre </TH>
    <TH> Volume Horaire </TH>
  </TR>
  <TR>
    <TD> HTML </TD>
    <TD> 1 </TD>
    <TD> 20 </TD>
  </TR>
  <TR>
    <TD> JavaScript </TD>
    <TD> 2 </TD>
    <TD> 30 </TD>
  </TR>
  <TR>
    <TD> ASP </TD>
    <TD> 2 </TD>
    <TD> 40 </TD>
  </TR>
</TABLE>|

```


Structure d'un Tableaux

Légende d'un tableau :

```
<TABLE>  
  <CAPTION> texte de légende </CAPTION>  
  ...  
</TABLE>
```

Exemple :

```
<TABLE WIDTH= 100% BORDER= 2>  
  <CAPTION >  
    <FONT SIZE=4 COLOR="BLUE"> Nouvelles Matières Introduites</FONT>  
  </CAPTION>  
  ....  
</TABLE>
```

Structure d'un Tableaux

Nouvelles Matières Introduites

Matière	Trimestre	Volume Horaire
HTML	1	20
Java Script	2	30
ASP.NET	2	40

Balises (Tags)

Structure d'un Tableaux

Fusion Horizontale : Extension de cellule sur C colonnes :

`<td colspan="C">`

Exemple :

1	2	
3	4	5

`<table>`

`<tr>`

`<td>1</td>`

`<td colspan=2>2</td>`

`</tr>`

`<tr>`

`<td>3</td>`

`<td>4</td>`

`<td>5</td>`

`</tr>`

`</table>`

Balises (Tags)

Structure d'un Tableaux

Fusion Verticale : Extension de cellule sur L lignes :

`<td rowspan="L">`

Exemple :

1	2	7	9
3		4	5

```
<table width="30%" border="2">
  <tr>
    <td>1</td> <td rowspan=2>2</td>
    <td>7</td> <td>9</td>
  </tr>
  <tr>
    <td>3</td> <td>4</td>
    <td>5</td>
  </table>
```

Balise d'hyperliens

Liens :

Base de la navigation hypertexte,

Lien : zone active cliquable

- ❑ `<a href="URL ou adresse"> support du lien </a>`
- ❑ **support du lien** : texte, image, contenus entre `<a>` et `</a>`
- ❑ **URL** : *Universal Resource Locator (adresse externe de site web),*
- ❑ **Adresse** : *adresse locale de la destination.*

Balise d'hyperliens

3 types de liens :

- ❑ **Lien externe** : destination dans un autre ordinateur (serveur) accessible à travers une adresse URL.

Exemples d'URL :

<i>http://serveur/chemin.../fichier</i>	Pour accéder à des pages Web
<i>ftp://serveur/chemin.../fichier</i>	Pour faire un transfert de fichiers
<i>mailto:utilisateur@hôte</i>	Pour envoyer un mail

Balises (Tags)

Balise d'hyperliens

- ❑ **Lien locale** : la destination se trouve dans le même dossier avec le document HTML.

Exemple d'appel de lien :

`<a href="info.html" > Pour des Informations cliquer ici </a>`

- ❑ **Lien mixte** : destination se trouve dans un autre dossier de votre ordinateur.

Exemples d'adresses :

lecteur://chemin.../fichier

Adressage absolu

../..../fichier

Adressage relatif

Balise d'hyperliens

Ancre :

Ancre (nommée) : point cible

- ✓ `<a name="nom">` point d'ancrage `</a>`
- ✓ **name**= nom unique (à la charge du concepteur)

Cibler l'ancre dans un lien : URL/adresse se terminant par **#nom** de l'ancre.

Exemple :

`<a href="#nom">`*Cibler ancre*`</a>`

`<a href="Adresse#nom">`*Cibler ancre*`</a>`

`<a href="URL#nom">`*Cibler ancre*`</a>`

Balise d'hyperliens

Attributs:

```
<a href = "url"  
    target = "_blank"  
    tilte = "info_bulle"> support du lien </a>
```

Les images

**** : Ajouter une image au sein du document

Syntaxe :

<img **src** = *"URL du fichier image"*

width = *largeur* **height** = *hauteur*

alt = *"texte alternatif"*

border = *épaisseur du cadre*

align = *"TOP/MIDDLE/BOTTOM/LEFT/ RIGHT"*

hspace = *espacement horizontal*

vspace = *espacement vertical*

>

Balises (Tags)

Les images

Attribut	Fonction
SRC	Spécifie l'URL (chemin) du fichier qui contient l'image.
WIDTH	Spécifie la largeur de l'image en pixels.
HEIGHT	Spécifie la hauteur de l'image en pixels.
BORDER	Définit l'épaisseur du cadre de l'image
ALT	Spécifie le texte de remplacement de l'image destiné pour les navigateurs Non graphiques ou dans lesquels l'utilisateur a désactivé le chargement des images.
ALIGN	Gère l'alignement du texte adjacent à l'image.
HSPACE	Définit l'espace qui sera réservé à droite et à gauche de l'image.
VSPACE	Définit l'espace qui sera réservé en haut et en bas de l'image.

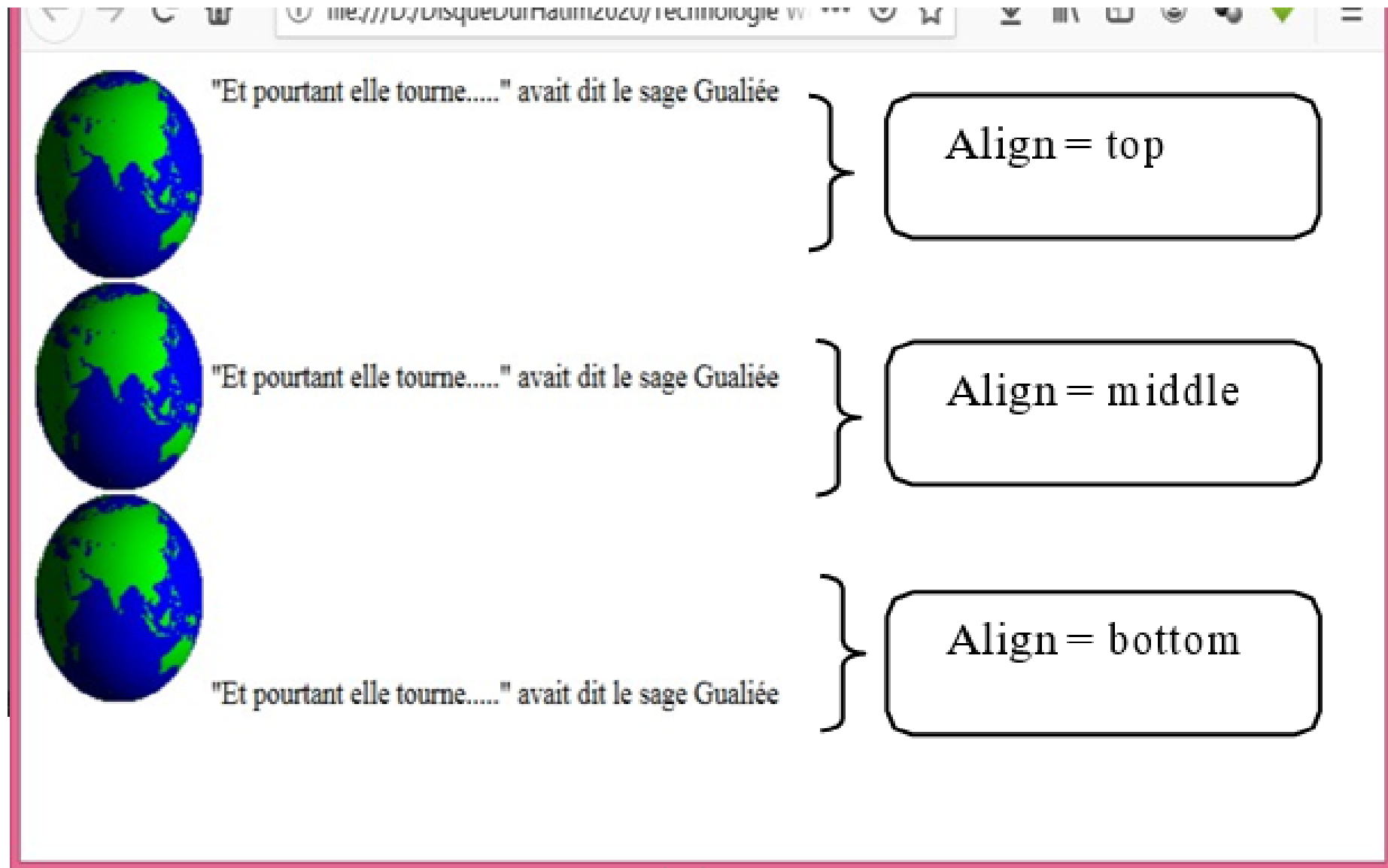
Balises (Tags)

Les images

Attribut **ALIGN** : Gère l'habillage de l'image par le texte et prend les valeurs indiquées dans le tableau ci-dessous :

Valeur	Fonction
TOP	Aligne le haut du texte adjacent sur le sommet de l'image.
MIDDLE	Aligne la ligne de base du texte adjacent sur le milieu de l'image.
BOTTOM	Aligne la ligne de base du texte adjacent sur la base de l'image.(valeur par défaut)
LEFT	Aligne l'image sur la marge gauche et permet son habillage par le texte sur sa droite.
RIGHT	Aligne l'image sur la marge droite et permet son habillage par le texte sur sa gauche.

Balises (Tags)



The screenshot shows a web browser window with a URL bar at the top. Below the browser window, there are three rows of content, each consisting of a globe image on the left and a text block on the right. The text in all three rows is "Et pourtant elle tourne....." avait dit le sage Gualée. To the right of each text block is a rounded rectangle containing an alignment property. A curly brace groups each image and text block with its corresponding alignment property.

	"Et pourtant elle tourne....." avait dit le sage Gualée	Align = top
	"Et pourtant elle tourne....." avait dit le sage Gualée	Align = middle
	"Et pourtant elle tourne....." avait dit le sage Gualée	Align = bottom

Balises (Tags)



"Et pourtant elle tourne....." avait dit le sage Gualiée



Align = left

Balises (Tags)

"Et pourtant elle tourne....." avait dit le sage Gualée

Align = right



Balises (Tags)

Donner le code HTML permettant de créer le tableau décrit dans la page WEB au-dessous avec les effets suivants :

- En cliquant sur la première image « **Tel1.gif** » du tableau, on charge sur le navigateur le fichier « **Plus_info_tel1.html** » qui se trouve dans le même répertoire.
- En cliquant sur la deuxième image « **Tel2.gif** » du tableau, on charge sur le navigateur le fichier « **Plus_info_tel2.html** » qui se trouve dans le même répertoire.

Informations Téléphones Mobiles	
 AD982704 www.fotossearch.fr	Première Série Octobre 2001
	Deuxième Série Septembre 2004
 K7080 www.hispanik.fr	Première Série Novembre 2007
	Deuxième Série Septembre 2010

KICH ISMAIL

Balises (Tags)

Audio

Balise :

```
<audio src="musique.mp3"></audio>
```

Attribut :

- ☐ Controls
- ☐ Loop
- ☐ Autoplay
- ☐ Preload
 - Auto
 - Metadata
 - none



Bonne pratique :

```
<audio controls>
```

```
<source src="musique.mp3">
```

```
<source src="musique.ogg">
```

Votre navigateur ne supporte pas l'audio

```
</audio>
```

KICH ISMAIL

Balises (Tags)

Vidéo

Balise :

```
<video src="film.webm"></video>
```

Attribut :

- ☐ Controls
- ☐ Poster
- ☐ Width, height
- ☐ Loop
- ☐ Autoplay
- ☐ Preload
 - Auto
 - Metadata
 - none



Bonne pratique :

```
<video controls poster="film.jpg" width="600">
  <source src="film.mp4">
  <source src="film.webm">
</video>
```

KICH ISMAIL

Formulaires

Les **formulaires** permettent à l'utilisateur de **saisir** des données :

- ☐ Création d'un compte,
- ☐ Petites annonces,
- ☐ Message dans un forum,
- ☐ Questionnaire,
- ☐ etc...



Les données saisies vont être **traitées** par le **serveur**.

Formulaire

La structure basique d'un formulaire HTML, contenant un bouton de validation, est :

```
<form>
```

```
...
```

```
<input type = "submit" value = "Validez le formulaire">
```

```
</form>
```

La balise **<form>** se paramètre avec les attributs **method** et **action** :

- **Method** = "get / post"
- **Action** = "url" : adresse du script qui va recevoir et traiter les données du formulaire.
- **Action** = "mailto:example@free.fr" : adresse de courrier électronique qui va recevoir les données du formulaire.

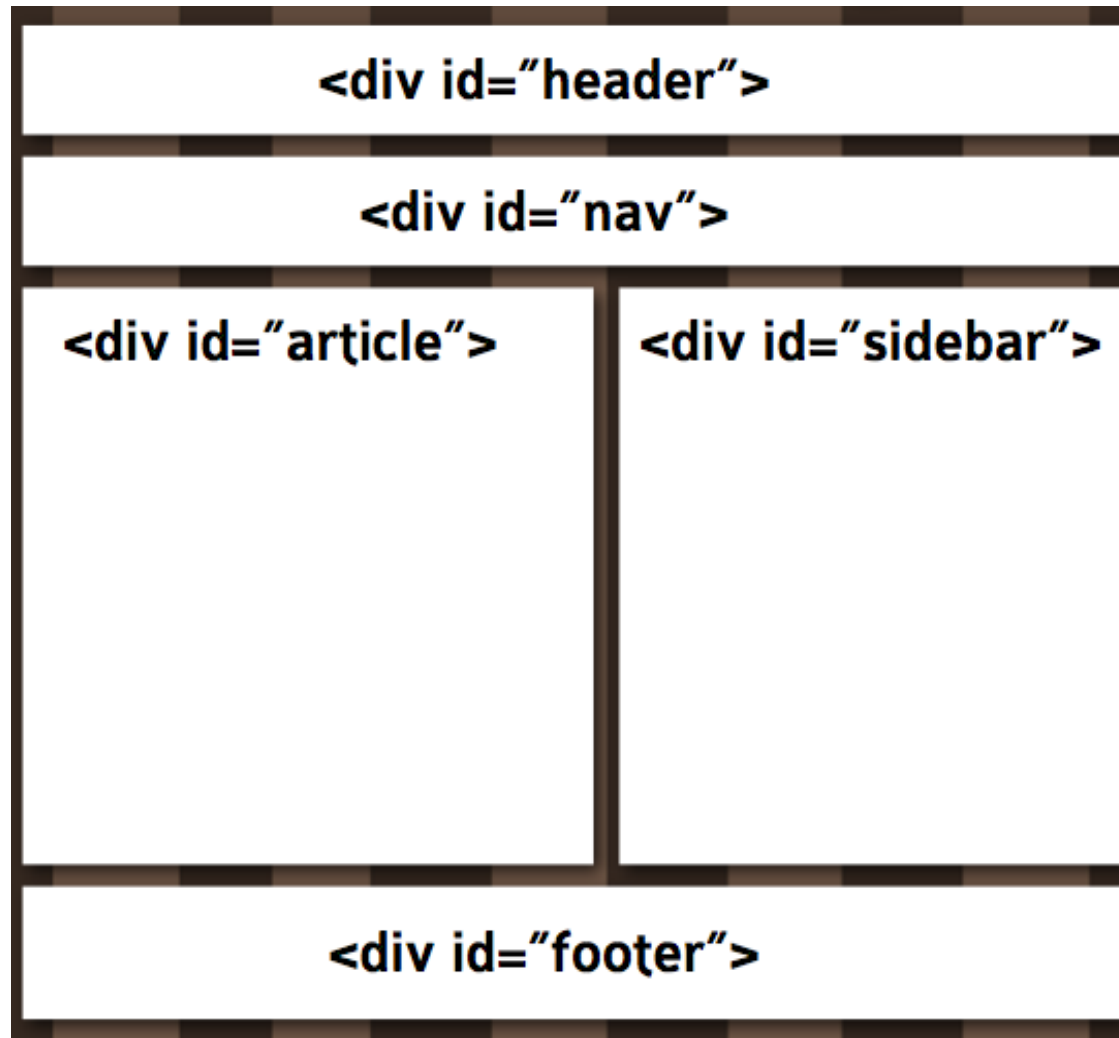
Les balises non Sémantiques

Balises génériques (non sémantique) : `<div>` ou `<span>`

- Ils servent seulement de conteneurs communiquant au navigateur comment le contenu qu'ils englobent doit être affiché.
- Ils ne donnent aucune information sur le rôle de ce contenu dans la page.
- Avec HTML4, les développeurs ont utilisé leur nom propre comme attribut favori à des éléments de page de style : **header, top, bottom, footer, menu, navigation, main, container, content, article, sidebar, topnav, ...**

Inconvénient : impossible pour les moteurs de recherche pour identifier le contenu de la page Web correcte.

Structure d'une page web (HTML4)



Structure d'une page web (HTML4)



Les balises Sémantiques

Balises sémantiques : Une balise sémantique est une balise qui donne une indication sur le contenu qu'elle entoure.

- Balises sémantiques de **structure** : délimitations de zones dans une page web.
- Balises sémantiques de **contenu** : valoriser le contenu textuel dans une page web.

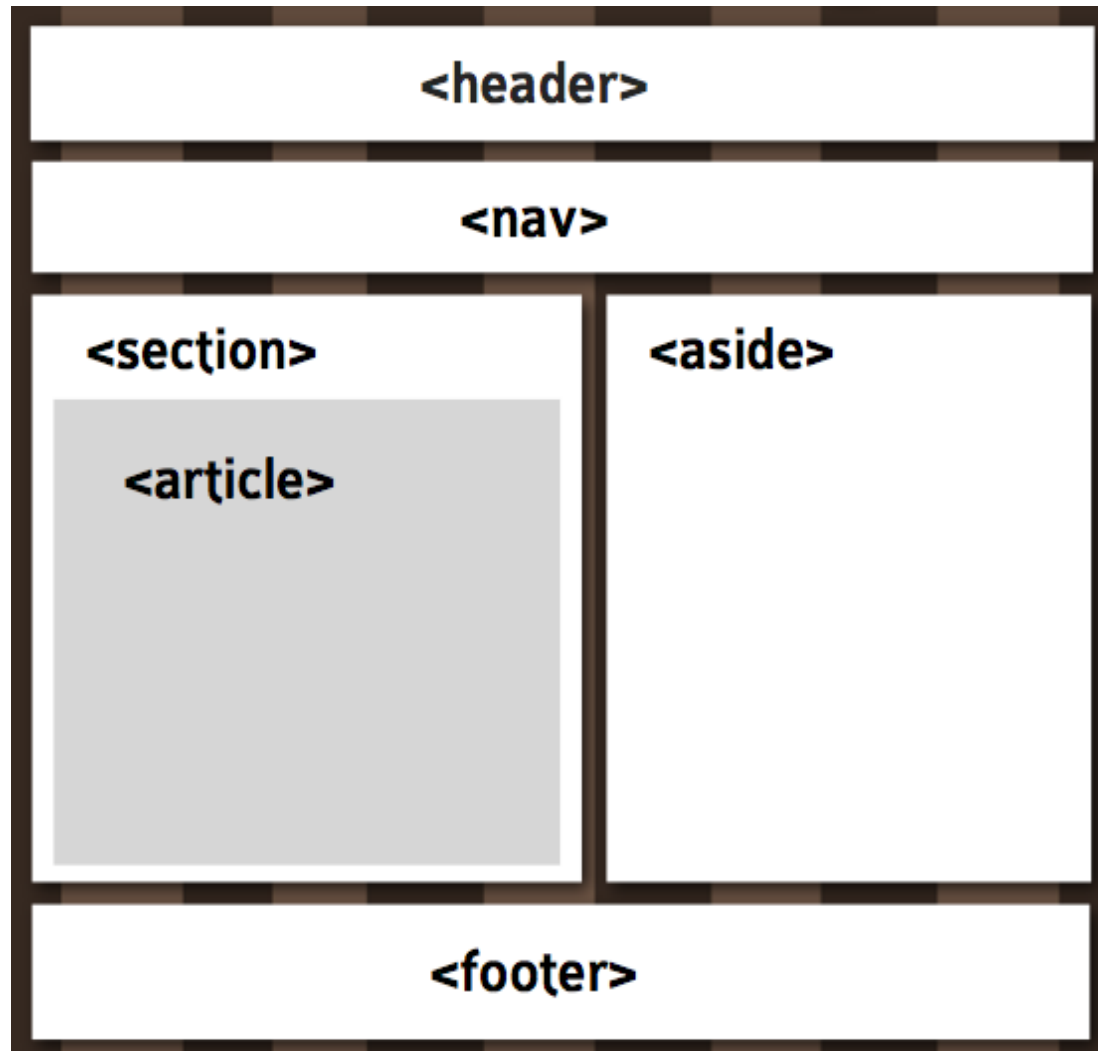
Avantage : Un bon balisage sémantique améliore la compréhension de votre contenu par les robots d'indexations des moteurs de recherche,

Les balises Sémantiques

Balises sémantiques de structure : `<header>` `<nav>` `<main>`
`<article>` `<section>` `<aside>` `<footer>` ...

Nom de l'élément	Description
header	Représente l'en-tête ou le haut d'une page
nav	Représente un menu de navigation dans une page
main	Représente le contenu principal de l'élément body
section	Représente une section dans une page, c'est-à-dire un groupement thématique de contenu
article	Représente une partie de page qui se suffit à elle-même, comme un post sur un blog par exemple
aside	Représente une partie de contenu non lié au reste
footer	Représente le pied d'une page

Structure d'une page web (HTML5)



<header>

LOGO

PRODUITS

BOUTIQUES

<nav>

BLOG

★★★★★

555 32 16 12

<main>

A PROPOS DE NOS PRODUITSChapô

UTILISATION 1 POUR NOS PRODUITS

Texte courant

UTILISATION 2 POUR NOS PRODUITS

Texte courant

UTILISATION 3 POUR NOS PRODUITS

Texte courant

Avis clients sur les produits

Infos sur notre industrie

Evaluation indépendante de nos produits par un journaliste indépendant (LIEN)

<footer>

À PROPOS

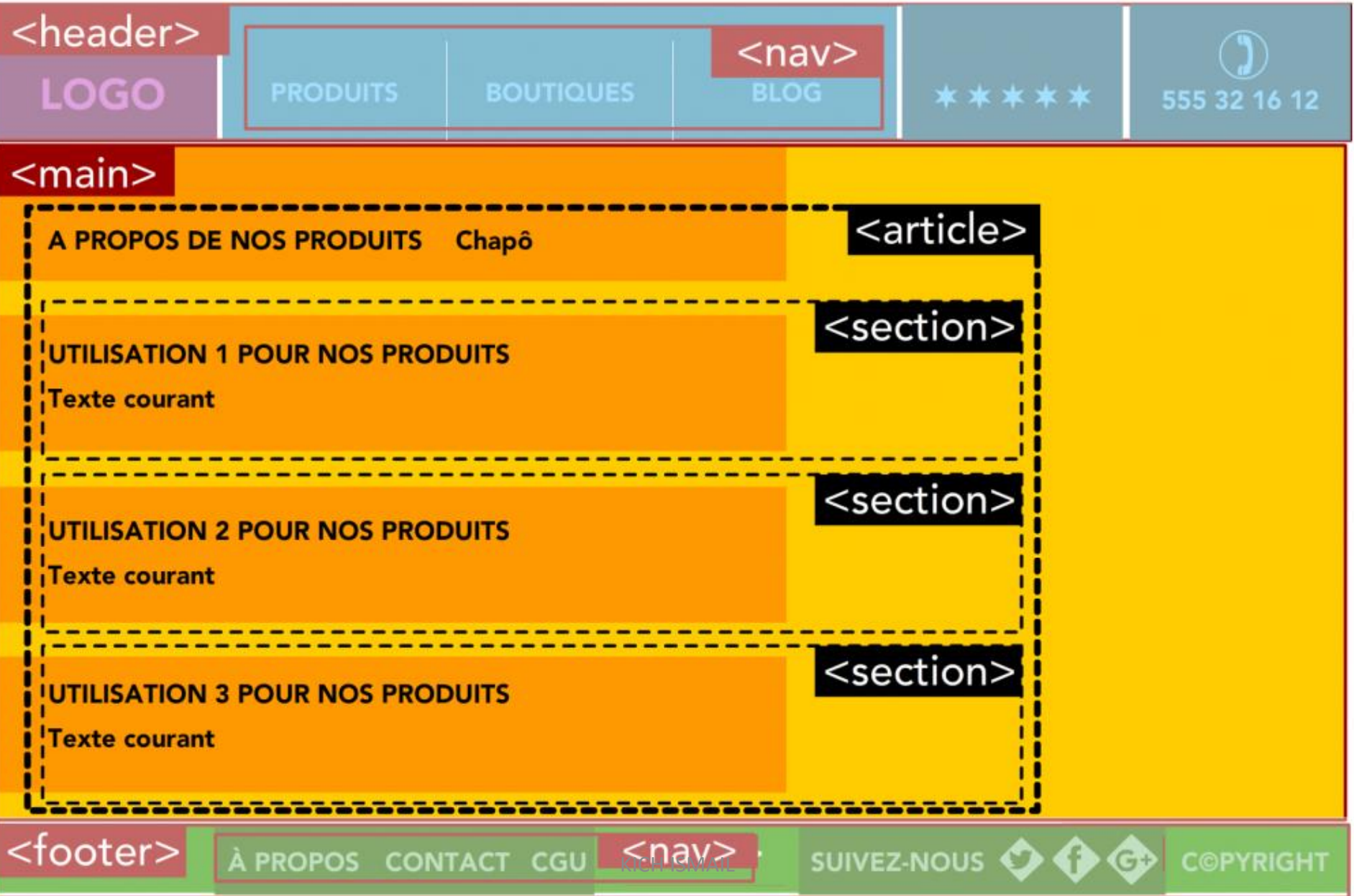
CONTACT

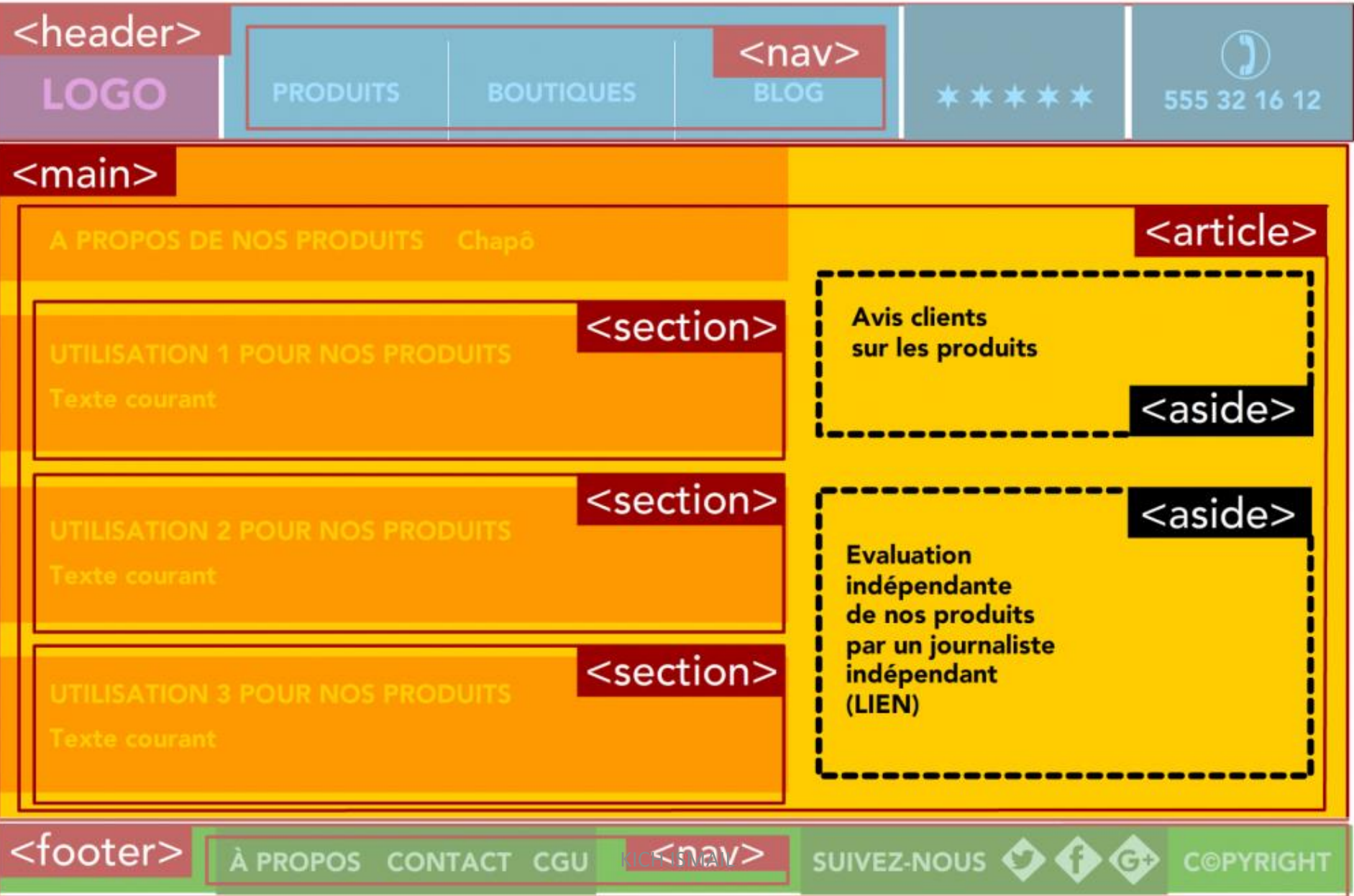
CGU

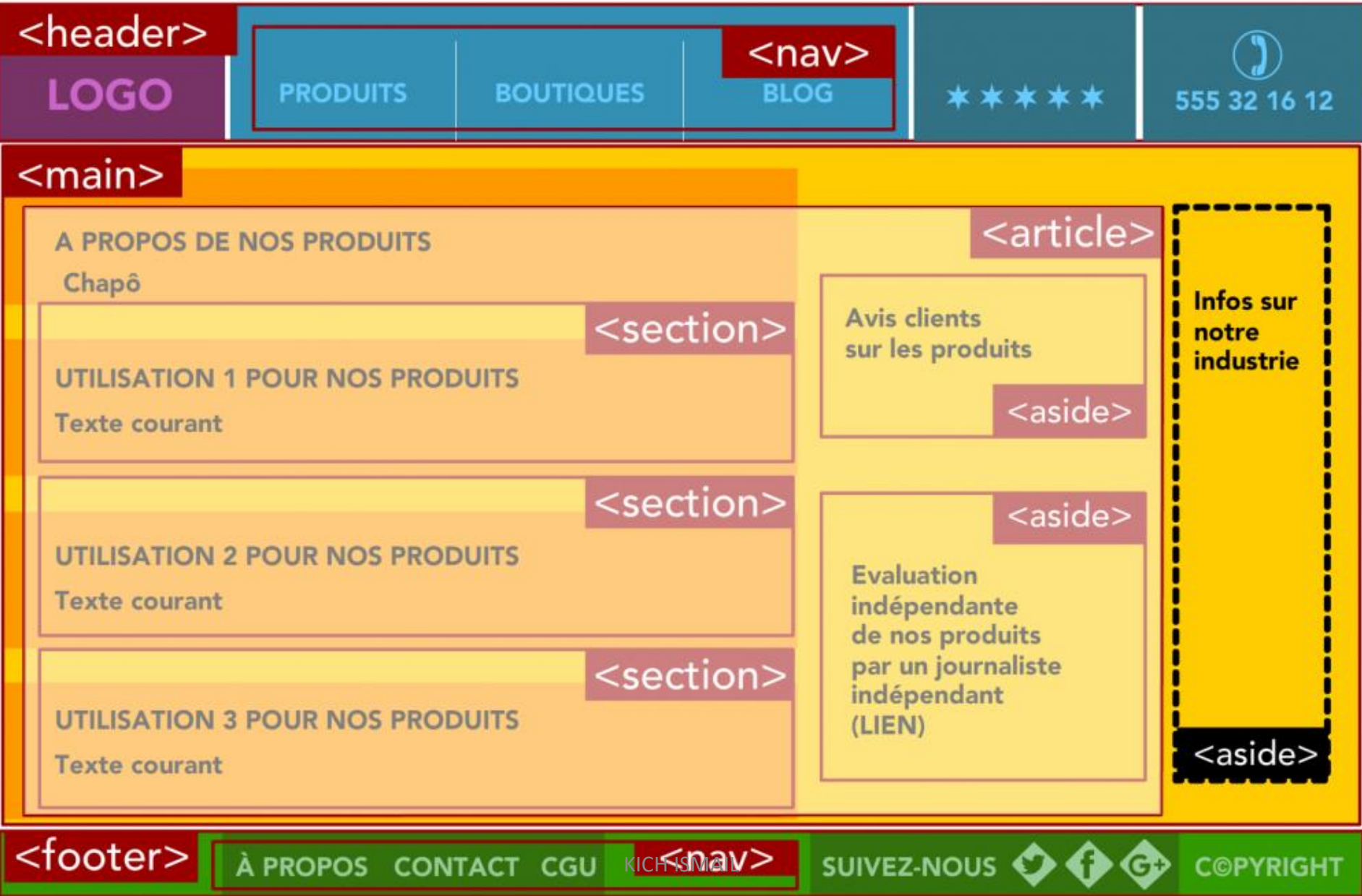
KICH ISMAIL

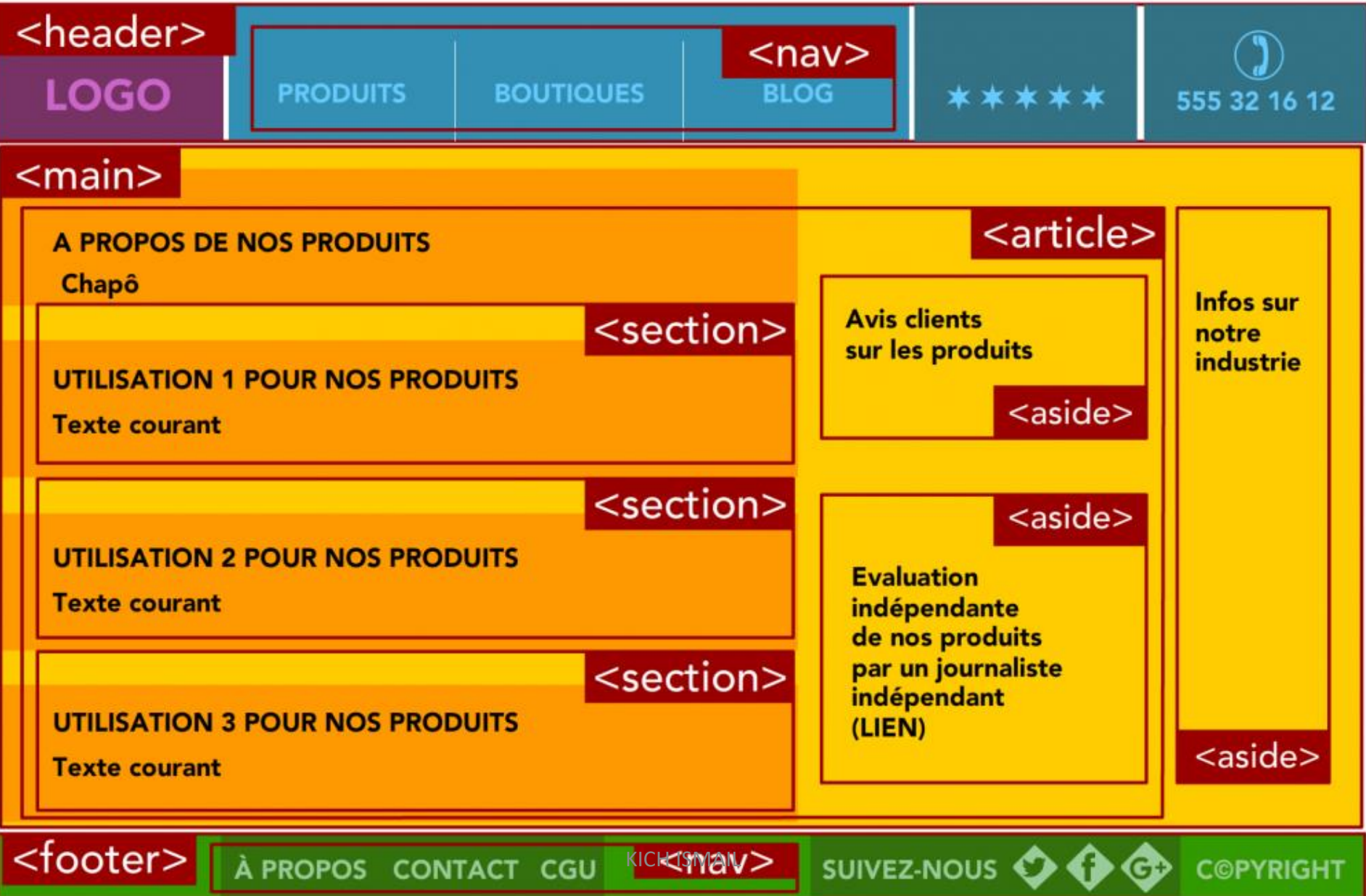
SUIVEZ-NOUS

©COPYRIGHT

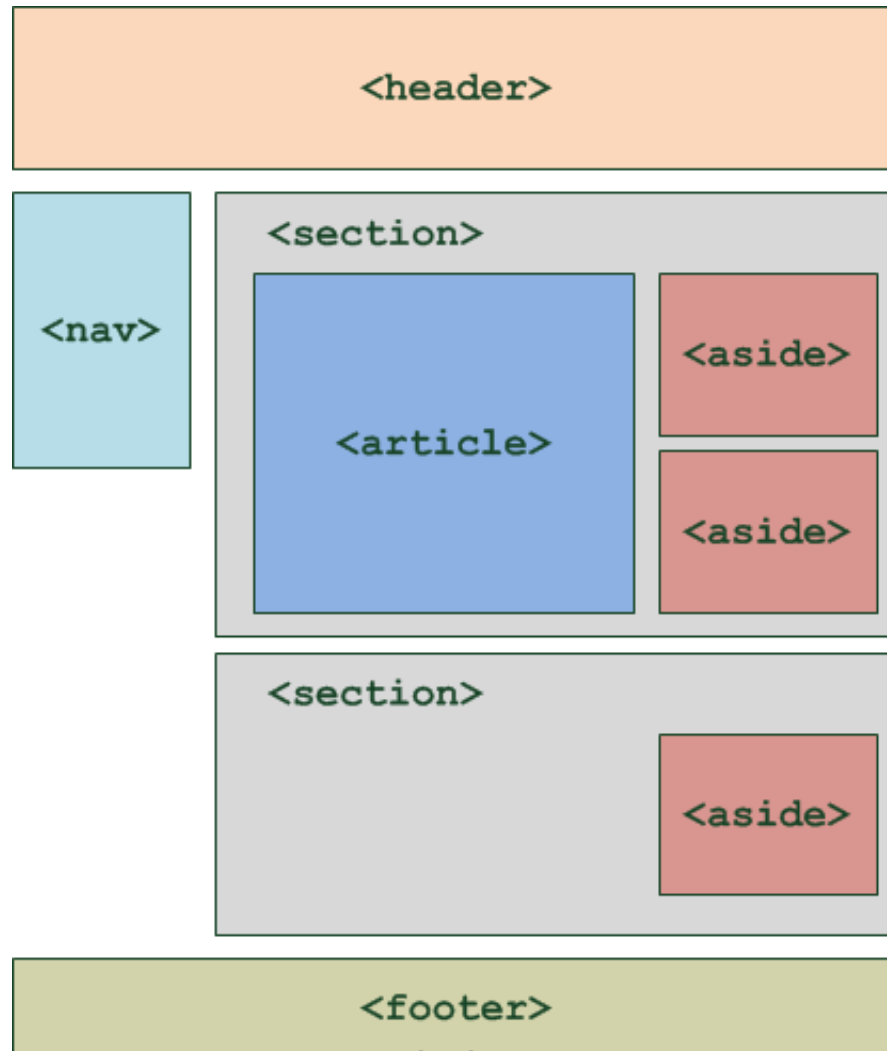




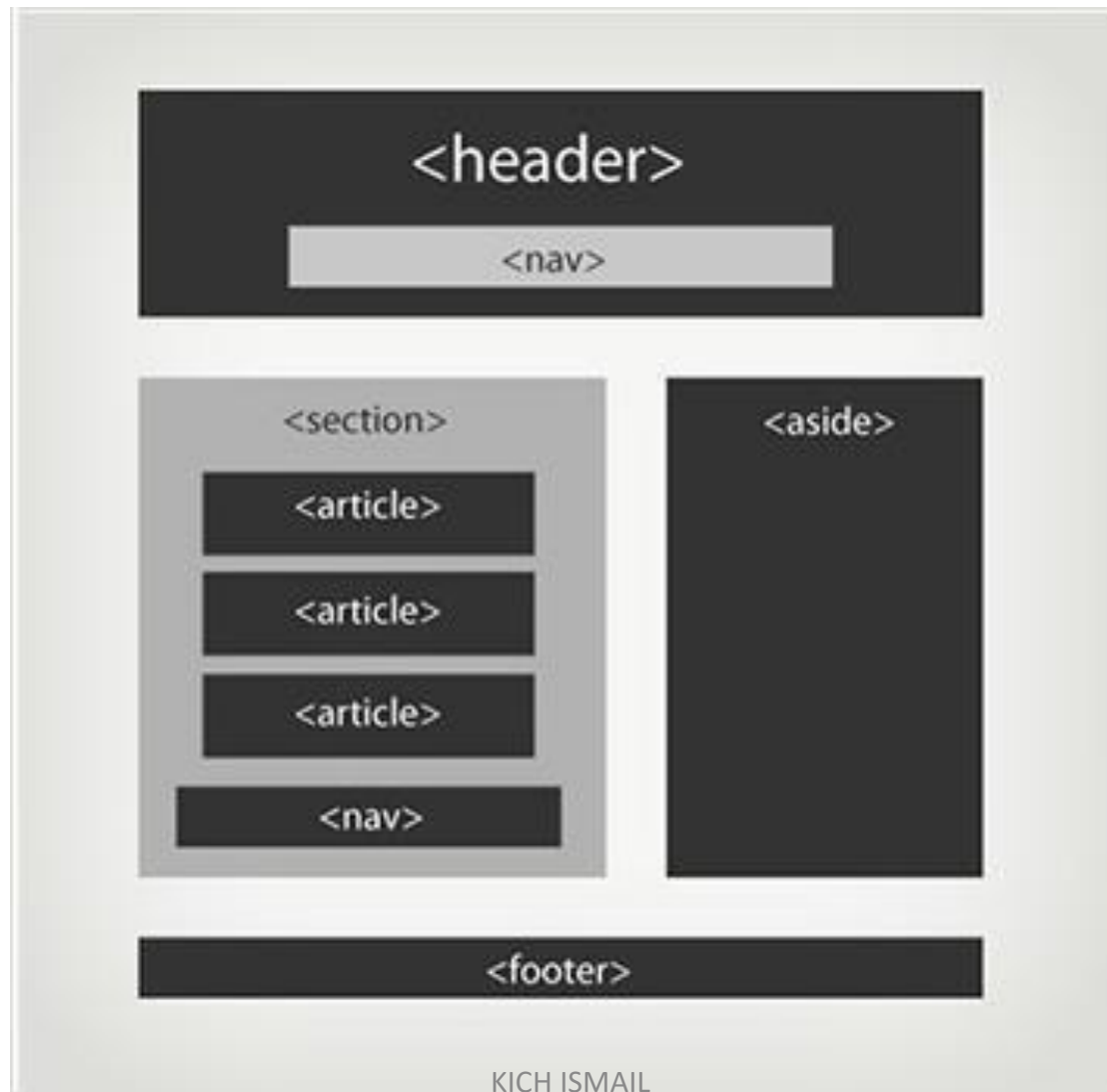




Structure d'une page web (HTML5)



Structure d'une page web (HTML5)



Les balises Sémantiques

Exemple :

`exemple_site.html`

Les balises Sémantiques

Balises sémantiques de contenu :

- divulgation des détails **<details>** & **<summary>**

<details>

<summary>Details**</summary>**

Something small enough to escape casual notice.

</details>

- Figure avec légende **<figure>** & **<figcaption>**

<figure>

<figcaption>titre de figure**</figcaption>**

</figure>

Les balises Sémantiques

Balises sémantiques de contenu :

- Les citations avec `<blockquote>` et `<cite>`

- Baliser une adresse avec `<address>`

`<address>`16 rue du centre-ville 44000 NANTES, France`</address>`

- Afficher une temporalité avec `<time>`

`<time datetime="15-03-2020 16:30:00">`Lundi 15 Mars à 16h30`</time>`

- Afficher une donnée générique avec `<data>`

`<data value="10000">`Déjà 10000 inscrits à la newsletter`</data>`

- Formater du code avec `<code>`

- haute importance `<strong>` , `<em>`, `<mark>`

- ...

Cascading Style Sheets

Feuilles de styles en cascade

Qu'est ce que CSS?

C'est un langage qui vient compléter le HTML, utilisé pour définir la présentation des pages Web.

- HTML permet de définir la structure de vos contenus (pages de votre site web),
- CSS permet de créer vos **styles** et le **design général** de votre site web,
 - ✓ la couleur de votre texte,
 - ✓ la police utilisée sur votre site,
 - ✓ la taille du texte, les bordures, le fond,
 - ✓ ...

HTML
(pas de CSS)



HTML + CSS



KICH ISMAIL

Versions CSS

Tout comme le HTML, le CSS a évolué. Il y avait quatre versions importantes de CSS :

- ❑ CSS 1
- ❑ CSS 2.0
- ❑ CSS 2.1
- ❑ CSS 3

Les styles HTML

`<p>Your formatted text goes here</p>`

`<p><font color="red" face="Verdana" size="1">Your formatted text goes here</font></p>`

Inconvénients :

- ✓ Répétitions,
- ✓ Mise à jour lourde,
- ✓ Augmentation de la taille des pages,
- ✓ Problème de portabilité des styles.

Les styles CSS

Il y a trois façon de définir des styles CSS :

- ☐ dans le code **au sein des balises html**,
- ☐ dans une **feuille de style interne**,
- ☐ dans une **feuille de style externe**. (Fichier.css)

Les styles CSS

❑ Styles CSS dans le code html ("**inline**")

on peut déclarer le style à l'intérieur d'une balise html à l'aide de l'attribut **style**.

Exemple :

```
<p style = "text-align : center; color : red">
```

lorem ipsum

```
</p>
```

Les styles CSS

❑ Feuille de style **interne**

Pour déclarer des styles qui ne s'appliqueront qu'à la page considérée, les styles sont à déclarer entre les balises suivantes :

Exemple :

```
<head>
```

```
<style>
```

```
p{color : red ;}
```

```
....
```

```
</style>
```

```
</head>
```

Les styles CSS

❑ Feuille de style **externe**

Enregistrer le code CSS dans un fichier s'appelant (par exemple) "[style.css](#)", et mettre dans l'en-tête de la page html (entre les balises `<head></head>`) :

Exemple :

```
<head>
```

```
  <link href = "style.css" rel = "stylesheet">
```

```
</head>
```

Notion de cascades

On peut déclarer les styles à différents endroits,

- ❑ Endroit / priorité : **styles plus au moins prioritaires**,
- ❑ Déclaration en externe : **la plus pratique**, mais c'est celle qui a le **moins de poids**,
- ❑ Déclaration en interne, dans l'en-tête de la page : des styles particuliers à une page. Les styles déclarés auront plus de poids que ceux de la feuille de style externe.
- ❑ Déclaration des styles en attributs des balises html : à faire pour des mises en forme ponctuelles. c'est celle qui a le **plus de poids**,

Structure de base

Pour chaque déclaration, la structure est toujours la même :

```
sélecteur {  
  Propriété1 : valeur1 ;  
  Propriété2 : valeur2 ;  
  Propriété3 : valeur3 ;  
  ...  
}
```

Le sélecteur : c'est la **balise** HTML (**body** ; **h1** ; **p**, **etc.**), l'**identifiant** (**id**) ou la **classe** (**class**),

La propriété : c'est l'**attribut** qu'on veut appliquer (font ; background ; margin ; etc.),

La valeur : précise les **caractéristiques** de la propriété.

Structure de base

balise1

```
{  
  propriete1: valeur1;  
  propriete2: valeur2;  
  propriete3: valeur3;  
}
```

balise2

```
{  
  propriete1: valeur1;  
  propriete2: valeur2;  
  propriete3: valeur3;  
  propriete4: valeur4;  
}
```

balise3

```
{  
  propriete1: valeur1;  
}
```

body

```
{  
  background-color: white;  
  font-family : Verdana, sans-serif;  
  font-size : 100%;  
}
```

h3

```
{  
  color : red;  
  text-align : center;  
}
```

p, ul, li, td

```
{  
  color : black;  
}
```

Activité : Changer la couleur de texte des paragraphes en utilisant les 3 méthodes d'application des styles **CSS**.

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
    <meta charset="utf-8" />
```

```
    <title>Premiers tests du CSS</title>
```

```
  </head>
```

```
  <body>
```

```
    <h1>Ma première activité Style</h1>
```

```
    <p>Lorem ipsum dolor sit amet.</p>
```

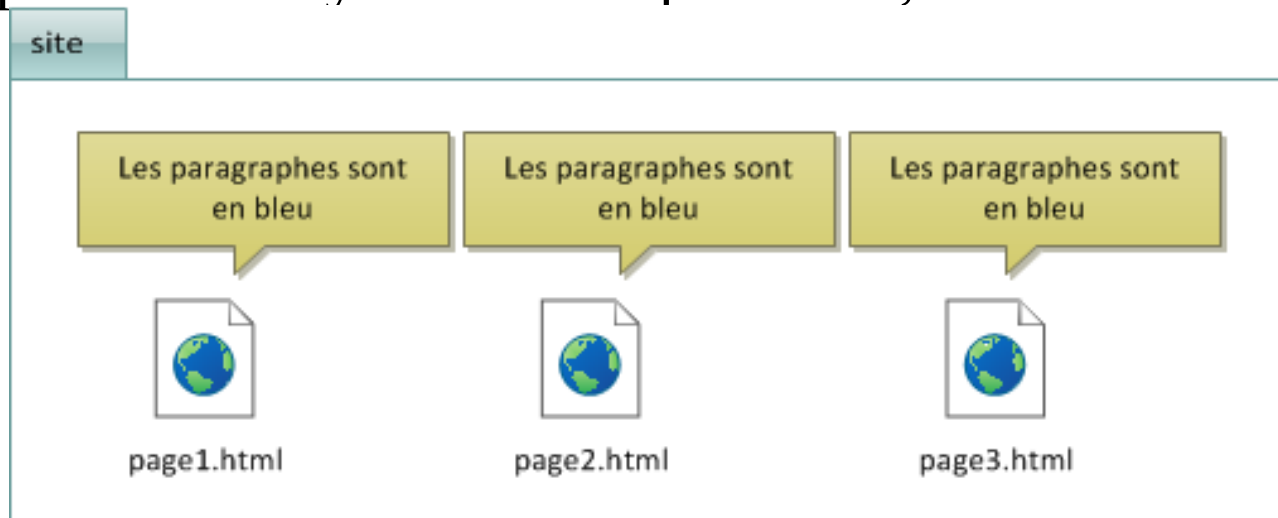
```
    <p>Lorem ipsum dolor sit, amet consectetur adipisicing elit. Et, ipsam.  
      Nesciunt facilis.</p>
```

```
  </body>
```

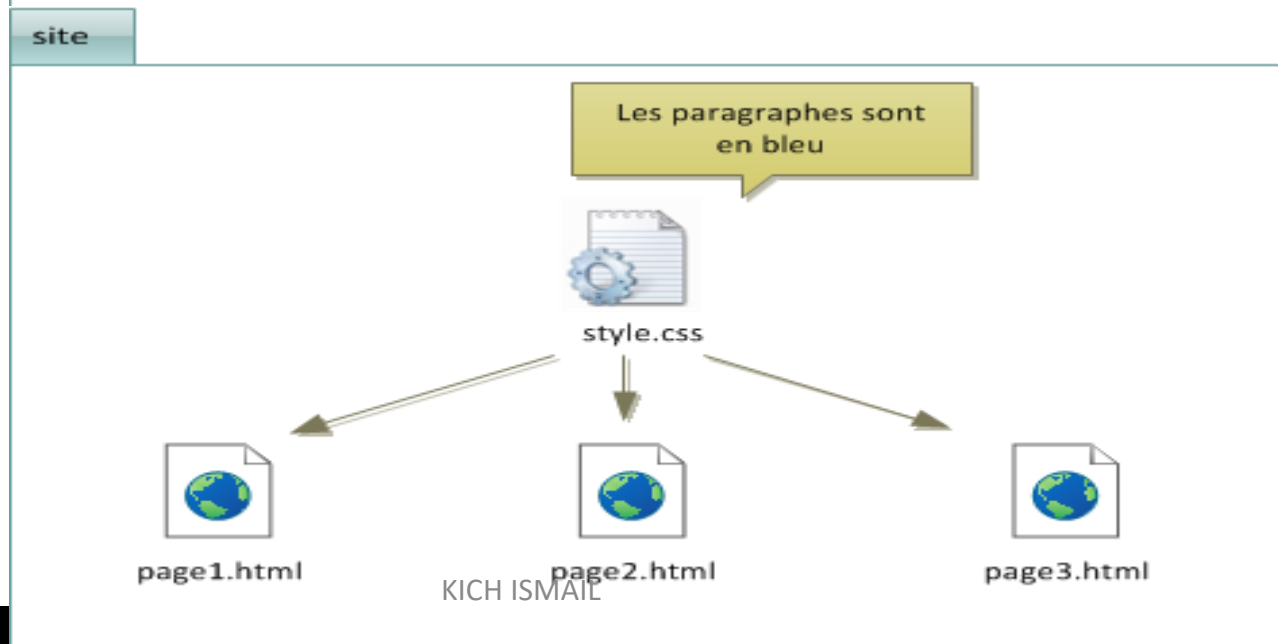
```
</html>
```


Style inline : répétition de style dans chaque balise,

Style interne :



Style externe :



Sélecteurs CSS *class* et *id* ?

Définir des styles particuliers que l'on voudra **reproduire** de façon **ponctuelle** ou **récurrente** dans les pages HTML.

.nom_class

```
{  
    propriété : valeur ;  
}
```

#nom_id

```
{  
    propriété : valeur ;  
}
```

les sélecteurs avancés

balise1

```
{
  propriete1 : valeur1 ;
  propriete2 : valeur2 ;
  propriete3 : valeur3 ;
}
```

[Link](#)

balise1, balise2

```
{
  propriete1 : valeur1 ;
  propriete2 : valeur2 ;
  propriete3 : valeur3 ;
}
```

[Link](#)

```
{
  propriété : valeur ;
}
```

[Link](#)

#id

```
{
  propriete1 : valeur1 ;
  propriete2 : valeur2 ;
  propriete3 : valeur3 ;
}
```

[Link](#)

.class

```
{
  propriete1 : valeur1 ;
  propriete2 : valeur2 ;
  propriete3 : valeur3 ;
}
```

[Link](#)
KICH ISMAIL

p.class

```
{
  propriete1 : valeur1 ;
  propriete2 : valeur2 ;
  propriete3 : valeur3 ;
}
```

[Link](#)

les sélecteurs avancés

balise1 balise2

```
{
  propriete1 : valeur1 ;
  propriete2 : valeur2 ;
  propriete3 : valeur3 ;
}
```

[Link](#)

Balise1 > balise2

```
{
  propriete1 : valeur1 ;
  propriete2 : valeur2 ;
  propriete3 : valeur3 ;
}
```

[Link](#)

balise1 + balise2

```
{
  propriete1 : valeur1 ;
  propriete2 : valeur2 ;
  propriete3 : valeur3 ;
}
```

[Link](#)

balise1 ~ balise2

```
{
  propriete1 : valeur1 ;
  propriete2 : valeur2 ;
  propriete3 : valeur3 ;
}
```

[Link](#)

balise1[attribut]

```
{
  propriete1 : valeur1 ;
  propriete2 : valeur2 ;
  propriete3 : valeur3 ;
}
```

[Link](#)
KICH ISMAIL

balise1 : first-child

```
{
  propriete1 : valeur1 ;
  propriete2 : valeur2 ;
  propriete3 : valeur3 ;
}
```

[Link](#)

les sélecteurs avancés

balise1 : last-child

```
{
  propriete1 : valeur1 ;
  propriete2 : valeur2 ;
  propriete3 : valeur3 ;
}
```

[Link](#)

balise1 : nth-child(3)

```
{
  propriete1 : valeur1 ;
  propriete2 : valeur2 ;
  propriete3 : valeur3 ;
}
```

[Link](#)

[CSS Selector Reference](#)

Balise :: after

```
{
  content: "text";
}
```

[Link](#)

Balise :: before

```
{
  content: "text";
}
```

[Link](#)

les sélecteurs avancés

Exemple :

h3 em

```
{
  propriete1 : valeur1 ;
}
```

<h3>Titre avec texte important</h3>

H3 + p

```
{
  propriete1 : valeur1 ;
}
```

**<h3>Titre</h3>
<p>Paragraphe</p>**

a[title]

```
{
  propriete1 : valeur1 ;
}
```

D'autres sélecteurs existent !

Mettre en forme un texte en CSS

❑ La taille

- **taille absolue** : en pixels, en centimètres ou millimètres

Exemple :

```
p
{
    font-size: 14px ; /* Paragraphes de 14 pixels */
}
```

```
h1
{
    font-size : 40px ; /* Titres de 40 pixels */
}
```

Mettre en forme un texte en CSS

❑ La taille

- **taille relative** : le texte s'adapte plus facilement aux préférences de tous les visiteurs. (**em**, **rem**, **%**, **vw**, ...)

Exemple :

```
p
{
  font-size: 0.87em ;
}

h1
{
  font-size : 2.5em ;
}
```


Mettre en forme un texte en CSS

❑ La taille

Exemple :

```
xx_small : minuscule;  
x-small  : très petit;  
small    : petit;  
medium   : moyen;  
large    : grand;  
x-large  : très grand;  
xx-large : gigantesque;
```

Mettre en forme un texte en CSS

❑ La police

```
balise
{
  font-family : police;
}
```

Ou bien :

```
balise
{
  font-family : police1, police2, police3, police4;
}
```

En général :

```
balise
{
  font-family : police1, police2, police3, police4, serif;
}
```

Mettre en forme un texte en CSS

❑ La police

Exemple :

```
p
{
  font-family : Impact, "Arial Black", Arial, Verdana, sans-serif;
}
```

Mettre en forme un texte en CSS

❑ Italique, gras, souligné...

Exemple :

```
balise
{
    font-style : normal | italic | oblique ;
    font-weight : normal | bold ;
    font-variant : normal | small-caps
}
```

Mettre en forme un texte en CSS

❑ Soulignement :

Exemple :

```
.ligne {
    text-decoration : underline | line-through | overline | underline overline ;
    text-decoration-line : underline | line-through | overline | underline overline ;
    text-decoration-style : solid | wavy | dotted | dashed | double;
    text-decoration-color : brown;
}

h1 {
    text-decoration : underline overline dotted red ; /* Shorthand properties*/
}
```

Mettre en forme un texte en CSS

❑ La couleur et autres

Exemple :

balise

{

Background-color : ColorName | #ColorCode | rgb(v1,v2,v3) ;

color : ColorName | #ColorCode | rgb(v1,v2,v3);

text-transform : none | capitalize | uppercase | lowercase

}

Mettre en forme un texte en CSS

❑ L'alignement

Exemple :

balise

{

text-align : center / justify / right;

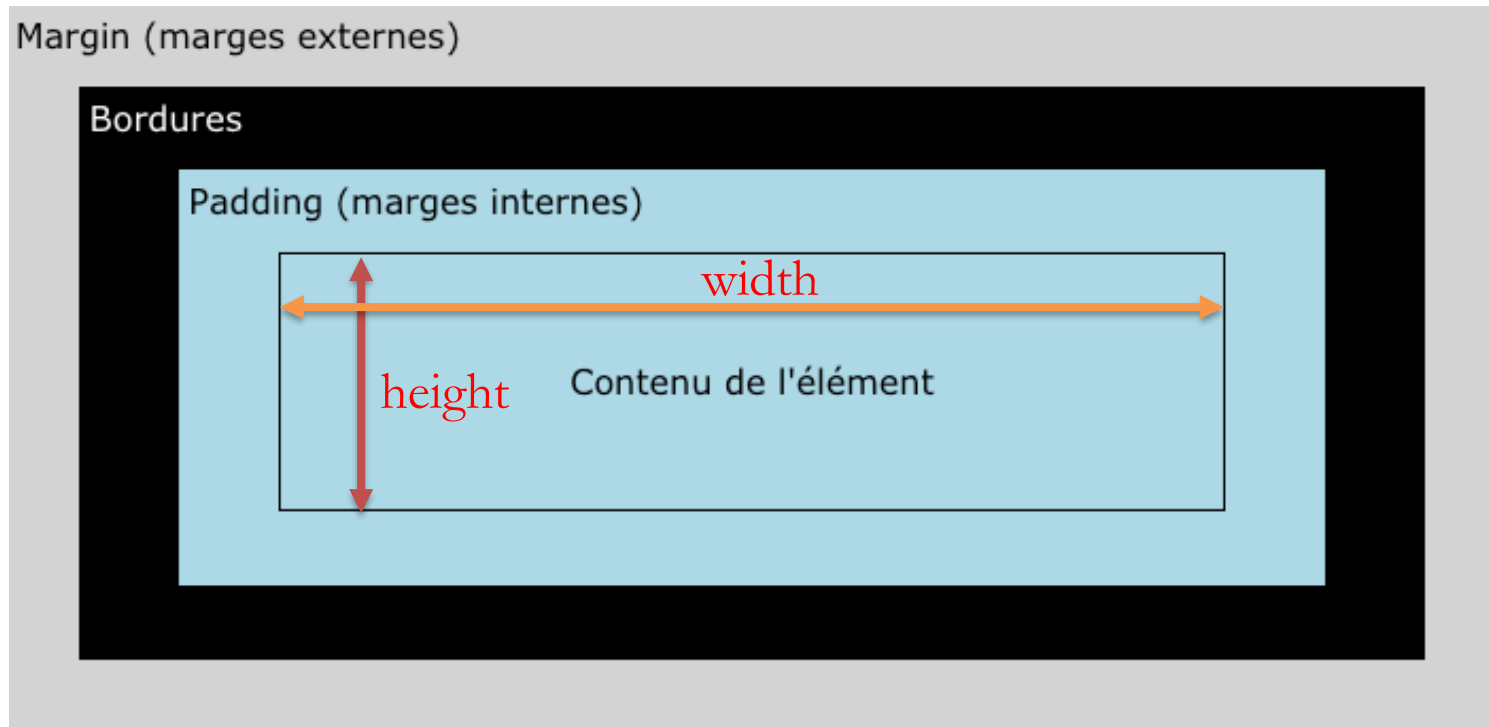
line-height : normal | nombre | %;

letter-spacing : 0,5 em ;

word-spacing : 0,5 em ;

}

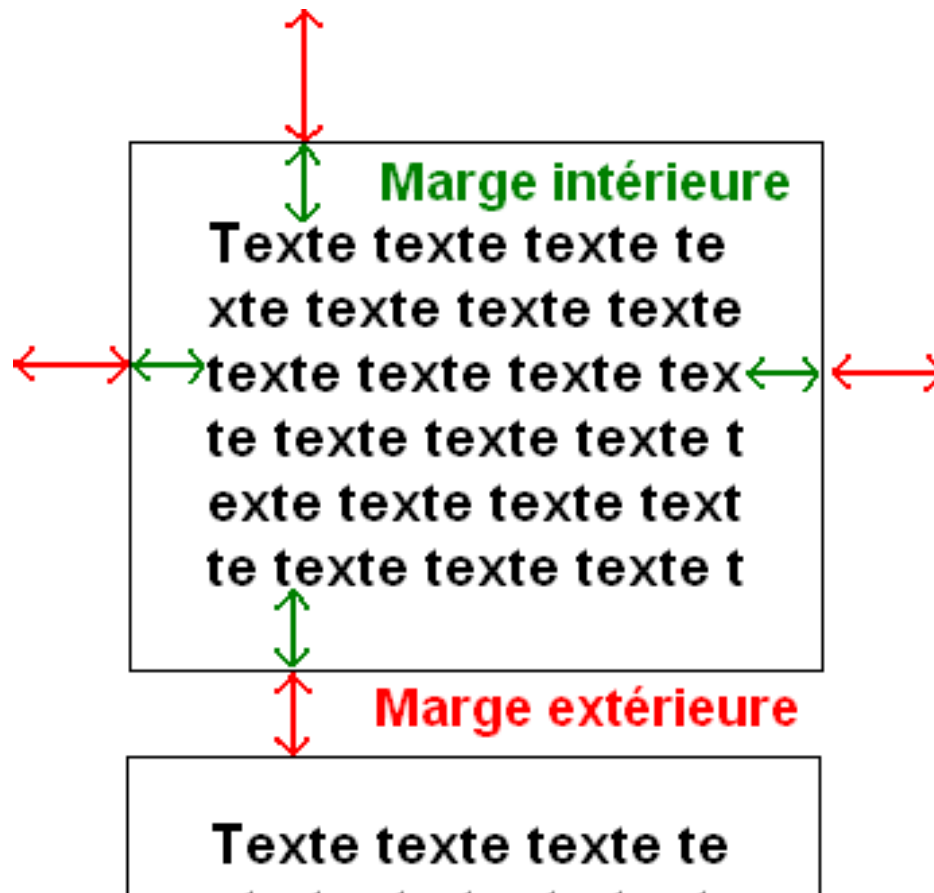
Modèle des boîtes



Attributs :

- ☐ Width, Height : définir la largeur et la hauteur de la boîte « contenu » ;
- ☐ Padding : définir la taille des marges internes ;
- ☐ Border : définir des bordures pour notre élément ;
- ☐ Margin : définir la taille des marges externes.

Modèle des boîtes



Modèle des boites

Balise

{

Width : 200px | 50% ;

Height : 100px | 20% ;

Overflow : auto | scroll | hidden;

}

Gérer les marges en CSS

Marges interne

`padding : 2px 5px 2em 0 ;`



`padding-top : 2px;`
`padding-right : 5px;`
`padding-bottom : 2em;`
`padding-left : 0;`

`padding : 2px 5px`



`padding-top : 2px;`
`padding-right : 5px;`
`padding-bottom : 2em;`
`padding-left : 5px;`

Gérer les marges en CSS

Marges externe

Margin : 2px 5px 2em 0 ;



margin-top : 2px;
margin-right : 5px;
margin-bottom : 2em;
margin-left : 0;

Margin : 2px 5px



margin-top : 2px;
margin-right : 5px;
margin-bottom : 2em;
margin-left : 5px;

Gérer les marges en CSS

Box-sizing

Box-sizing : content-box | border-box ;

Modèle des boîtes

```
div
{
    max-width : 500px ;
    margin : auto;
}
```

[Link](#)

```
span
{
    min-width : 500px ;
    display: inline-block ;
}
```

[Link](#)

Les bordures et les ombres

Les Bords :

```

balise
{
  border-width : épaisseur ;
  border-color : couleur ;
  border-style : type de bordure;
  Border-radius : 10px
}

```

```

h1
{
  border-width : 3px ;
  border-color : blue ;
  border-style : dashed ;
}

```

Ou bien

```

h1
{
  border : 3px blue dashed;
}

/* l'ordre n'est pas important */

```

border-width : "thin | medium | thick | 10px"

border-style : "none | solid | dotted | dashed | double | groove | ridge | inset | outset"

border-radius : 10px 5px 10px 5px;

Les bordures et les ombres

Les Bordeurs :

balise

{

border-top : 5px dotted black;

border-right : 10px dashed rgba(247, 16, 47, 0.308);

border-bottom : 10px double rgb(46, 12, 238);

border-left : 5px groove red;

border-top-left-radius : 20px ;

border-bottom-right-radius : 20px / 10px;

}

Les bordures et les ombres

les ombres des boîtes :

```

balise
{
    box-shadow : 6px 6px 0px black;
}
    
```

- Le décalage horizontal de l'ombre.
- Le décalage vertical de l'ombre.
- L'adoucissement du dégradé.
- La couleur de l'ombre.

```

balise
{
    box-shadow : 6px 6px 0px black inset;
}
    
```

```

box-shadow: 0px 0px 2px 5px red,
            0px 0px 2px 10px green,
            0px 0px 2px 15px black;
    
```

Les bordures et les ombres

l'ombre du texte :

```
balise
{
  text-shadow : 2px 2px 4px black;
}
```

Images de fond

Pour changer la couleur et l'image de fond de la page web, il faut travailler sur la balise `<body>`

Exemple (CSS) :

/ On travaille sur la balise body, donc sur TOUTE la page */*

Balise {

color : white | #000000 | rgb(255, 255, 255) ;

background-color : black | #000000 | rgb(255, 255, 255) ;

background-image : url(images/houx2.gif) ;

background-repeat : no-repeat | repeat-x | repeat-y | repeat ;

background-size : auto | 50px | contain | cover ;

background-attachment : fixed | scroll ;

background-position : 0% 50% ;

background-origin : padding-box | border-box | content-box

}

Images de fond

Combiner les propriétés

```
body
{
    background : url("soleil.png") fixed no-repeat 100% 0%;
}
```

Plusieurs images de fond

```
body
{
    background: url("soleil.png") fixed no-repeat top right, url("neige.png") fixed;
}
```

La transparence

```
p
{
    opacity : 0.6 ;
}
```

Apparences dynamiques

➔ Modifier l'apparence des éléments de façon dynamique

Le **CSS** nous permet de changer l'apparence des éléments :

- Au survol " : **hover** "
- lors du clic " : **active** "
- lors du focus (élément sélectionné) " : **focus** "
- lorsqu'un lien a été consulté " : **visited** "

Affichage et position des élément HTML

La propriété CSS display est une propriété très puissante puisqu'elle va nous permettre de modifier la façon dont un élément va s'afficher dans la page : en ligne, sous forme de bloc, etc. et donc la façon dont il va se comporter avec ses voisins.

Balise

```
{  
display : none | block | inline | inline-block | flex ;  
}
```

Affichage et position des élément HTML

La propriété **position** est une propriété **CSS** très **puissante** qui va nous permettre de définir un type de positionnement pour nos éléments (balises).

Balise

```
{  
    position : static| relative| absolute| fixed| sticky;  
}
```

Positions d'éléments HTML

La propriété **position** est une propriété **CSS** très **puissante** qui va nous permettre de définir un type de positionnement pour nos éléments (balises).

- **static** : valeur par défaut. Les éléments sont rendus dans l'ordre, tels qu'ils apparaissent dans le flux de documents.
- **relative** : l'élément est positionné par rapport à sa position normale,
- **fixed** : l'élément est positionné par rapport à la fenêtre du navigateur,
- **absolute** : l'élément est positionné par rapport à son premier élément ancêtre positionné (non statique).
- **sticky** : l'élément est positionné en fonction de la position de défilement de l'utilisateur.

Mise en page des boîtes flexibles

La mise en page des boîtes flexibles facilite la conception d'une structure de page web flexible et réactive sans utiliser de **float** ou de **positionnement**.

Les codes de flexbox se devise en deux parties :

- Code pour l'élément parent :
 - ❑ **Display** : **flex**;
 - ❑ **flex-direction** : **row**;
 - ❑ **flex-wrap** : **wrap**;
 - ❑ **flex-flow** : **row wrap**;
 - ❑ **justify-content**: **center**;
 - ❑ **align-content**: **center**;

- Code pour les éléments fils
 - ❑ **flex-grow** : **0**;
 - ❑ **flex-shrink** : **1**;
 - ❑ **flex-basis** : **auto**;
 - ❑ **Flex** : **0 1 auto**;
 - ❑ **Order** : **0**;

Mise en page des boites flexibles

Elément parent :

- ❑ `Display : flex | inline-flex ;`
- ❑ `direction: rtl | ltr ;`
- ❑ `flex-direction: row | column | row-reverse | column-reverse ;` [link](#)
- ❑ `flex-wrap: nowrap | wrap ;`
- ❑ `flex-flow: flex-direction flex-wrap; //shorthand declaration`
 - ✓ `flex-flow: row nowrap;`
- ❑ `justify-content: left | center | right;` [link](#)
- ❑ `align-content : center | ... |` [link](#)
- ❑ `align-item : center | ... |` [link](#)

Mise en page des boites flexibles

Éléments fils :

- ❑ `flex-grow : 0 | 1 ;`
- ❑ `flex-shrink : 0 | 1 ;`
- ❑ `flex-basis: 120px | auto ;`
- ❑ `flex: flex-grow flex-shrink flex-basis ;` **//shorthand declaration**
 - ✓ `flex: 0 1 auto ;`
- ❑ `order: 0 | N ;` [link](#)

transform & transition

La propriété **CSS transform** vous permet de faire **pivoter**, de **mettre à l'échelle**, **d'incliner** ou de **traduire** un élément. Elle modifie l'espace de coordonnées du modèle de formatage visuel **CSS**.

❑ **transform**: **translate(x, y)**; // Définit une translation 2D

[Link](#)

❑ **transform**: **scale(x, y)**; // Définit une transformation d'échelle 2D

[Link](#)

❑ **transform**: **rotate(deg)**; // Définit une rotation 2D, l'angle est spécifié dans le paramètre.

[Link](#)

[button](#)

KICH ISMAIL

transform & transition

Les **transitions CSS** permettent de contrôler la vitesse d'animation lors de la modification des propriétés CSS.

transition: property duration timing-function delay ;

- **transition-property** : spécifie le nom de la propriété CSS pour laquelle l'effet de transition est prévu,
- **transition-duration** : indique le nombre de secondes ou de millisecondes nécessaires pour que l'effet de transition soit complet,
- **transition-delay** : définit quand l'effet de transition commencera.
- **transition-timing-function** : spécifie la courbe de vitesse de l'effet de transition,

[Link](#), [Font size](#), [Menu](#), [Multiple transition](#).

Animatable CSS properties : [Link](#)

KICH ISMAIL

Animation

- CSS permet d'animer des éléments HTML sans utiliser JavaScript ou Flash !
- Les animations CSS permettent d'animer les transitions d'une configuration de style CSS à une autre.
- Les sous-propriétés de la propriété **animation** sont :
 - **animation-name** : spécifié le nom de la **@keyframes** décrivant les images clés de l'animation,
 - **animation-duration** : configure la **durée** nécessaire à une animation pour terminer un cycle.

```

@keyframes change-color {
  from {
    background-color: red;
  }
  to {
    background-color: blue;
  }
}

```

[Link](#)

Animation

- Les sous-propriétés de la propriété **animation** sont :
 - **animation-iteration-count** : Configure le nombre de fois que l'animation doit se répéter ; vous pouvez spécifier "**infinite**" afin de répéter l'animation indéfiniment.
 - **animation-timing-function** : courbes d'accélération.
 - **animation-direction** : configure si l'animation doit ou non changer de direction à chaque exécution de la séquence ou être réinitialisée au point de départ et se répéter.
 - **animation-play-state** : vous permet de mettre en pause et de reprendre la séquence d'animation.
 - **animation-delay** : configure le délai entre le moment où l'élément est chargé et le début de la séquence d'animation

[Link](#)

Balises (Tags)

HTML Links

- File
- Section
- Email

HTML Links

- File

`<a href="https://www.google.com/"> click here </a>`

`<a href="Accueil.html"> click here </a>`

`<a href="./test/Accueil.html"> click here </a>`

Balises (Tags)

HTML Links

- Section

`<a href="#sec1"> click here </a>`

`<a name="sec1"> click here </a>`

Balises (Tags)

HTML Links

- Email

`<a href="mailto:exemple@gmail.com"> click here </a>`

`<a href="mailto:exemple@gmail.com?subject=aaaaa & body=bbbbbb"> click here </a>`

Balises (Tags)

HTML Image

- Adding : ajouter une image

```

```

- Formatting : dimension d'image

```

```

- Aligning : Position d'image

```

```

Balises (Tags)

HTML Image

- ImageMap : Liens sur image

```

```

```
<map name="mymap">
```

```
    <area shape="circle" coords="35,35,35" href =  
    "https://www.google.com">
```

```
<area shape="rectangle" coords="472,79,444,273" href =  
    "https://www.google.com">
```

```
</map>
```

Balises (Tags)

HTML iframe

- **iframe** : insertion d'objets

```
<iframe width="560" height="315" src="" ></iframe>
```

Balises (Tags)

HTML Table

```
<table>
  <head>
    <tr>
      <th>Id</th>
      <th>Name</th>
      <th>Adresse</th>
    </tr>
  </head>
  <tbody>
    <tr>
      <td>1</td>
      <td>ismail</td>
      <td>ismail adresse</td>
    </tr>
  </tbody>
</table>
```

Plan

- ❑ Introduction
- ❑ Partie I – Les Bases en HTML et en CSS
- ❑ Formater du Texte & Positionner des Eléments
grâce au CSS
- ❑ Fonctionnalités Avancées

5 Reasons to Learn HTML and CSS

- ✓ Better Understand the Web
- ✓ Create Websites
- ✓ Start a Web Career
- ✓ Learn Another Languages with Greater Ease
- ✓ Earn Money

Formulaire Type

Coordonnees

Nom

Prenom

Je désire recevoir une documentation ☐

Pays

Sexe ☒ Masculin ☐ Féminin

Domaine(s) d'activite

Vous pouvez en choisir plus qu'un...

☐ Pedagogie ☐ Psychologie ☐ Informatique ☐ Autre

Commentaire

Si vous le voulez, vous pouvez decrire vos activites ou interets

Effacer les données

Enregistrer les donnees

Les Formulaires

- ❑ Un formulaire fournit un espace sur la page HTML où l'utilisateur peut entrer des données,
- ❑ Ces données peuvent être envoyées au serveur pour être éventuellement traitées,
- ❑ Un formulaire est composé d'un ou plusieurs éléments d'entrée englobés par la balise **<form>**
- ❑ La balise **<form>** devra avoir les attributs "**method**" et "**action**" permettant le traitement du formulaire.

Les Formulaires

- ❑ Ecrire le code HTML qui permet de réaliser le Formulaire suivant:

Balises (Tags)

Balises de Head

```
<head>
```

```
  <meta charset="utf-8"/>
```

```
  <title>Ma première page WEB</title>
```

```
  <meta name="description" content=
    "information afficher par le
    moteur de recherche" />
```

```
  <style></style>
```

```
  <script></script>
```

```
</head>
```

Introduction au JavaScript

1995 : Brendan Eich (Netscape Communication Corporation)

1996 : Netscape intègre le **moteur JavaScript**,

...

2020 : 11^{ème} édition standard.

Le JavaScript est :

- Un langage de programmation,
- Orienté objet,
- Principalement utilisé coté client,
- Permet de manipuler d'une manière dynamique de HTML et CSS. [Exemple](#)

Où Coder le JavaScript ?

- ❖ Directement dans une balise ouvrante de HTML

```
<button onclick="code javascript">click</button>
```

- ❖ Dans la balise <head>

```
<head>  
    <script>  
        code JavaScript  
    </script>  
</head>
```

- ❖ Dans la balise <body>

```
<body>  
    <script>  
        code JavaScript  
    </script>  
</body>
```

- ❖ Dans un fichier séparé "test.js".

Où Coder le JavaScript ?

- ❖ Code JavaScript dans une balise ouvrante de HTML

```
<!DOCTYPE html>
<HTML>
<head>
    <title>Test JAVASCRIPT</title>
</head>
<body>
    <button onclick="alert('Hi from button script');">Click1</button>
    <button onclick="alert('Good by from button script');
        window.close();"> Click 2</button>
</body>
</HTML>
```

Où Coder le JavaScript ?

- ❖ Code JavaScript dans une balise **<SCRIPT>**, au sein de la page **HTML**

<HTML>

<head>

<title>Test JAVASCRIPT**</title>**

<script> **function** **f1()**{**alert**('Hi from Head tag');}**</script>**

</head>

<body>

<button **onclick**="**f1()**" **>** Click 1**</button>**

<script> **function** **f2()**{**alert**('Good by from Body tag');
window.close();}**</script>**

<button **onclick**="**f2()**"**>**Click 2**</button>**

</body>

</HTML>

Où Coder le JavaScript ?

Remarque :

- ❖ On peut placer la balise **<SCRIPT>** n'importe où dans la page **HTML**, aussi bien dans **<HEAD>...</HEAD>** qu'au sein de l'élément **<BODY> </BODY>**.
- ❖ On peut également ajouter plusieurs balises **<SCRIPT>** dans la même page **HTML** pour placer plusieurs bouts de code JavaScript à différents endroits de la page

Où Coder le JavaScript ?

- ❖ Code JavaScript dans un fichier séparé (**test.js**)

```
function f1(){  
    alert('Hi from JavaScript file') ;  
}  
function f2(){  
    alert('Good by from JavaScript file') ;  
    window.close();  
}
```

test.js

Où Coder le JavaScript ?

- ❖ Code JavaScript dans un fichier séparé (**test.js**)
- ❖ Au sein de la page Html, on fait appel au code du fichier **JavaScript**:

<HTML>

<head>

<title>Test JAVASCRIPT**</title>**

<script **src="** test.js**"></script>**

</head>

<body>

<button **onclick="**f1()**"** > Click 1**</button>**

<button **onclick="**f2()**"**>Click 2**</button>**

</body>

</HTML>

Syntaxe de base : Règles générales

- ❖ Les commentaires :
 - `//` pour un commentaire mon-ligne,
 - `/*` pour un commentaire multi-lignes.`*/`
- ❖ Un code JavaScript est un ensemble d'instruction :
- ❖ Chaque instruction doit être terminer par un point virgule ;
- ❖ Le langage JavaScript est sensible à la casse.
- ❖ Un nombre à virgule est séparé par un point (.) et non par une virgule (11.5)

Types et déclaration de variables.

Une variable est un conteneur servant à stocker des informations de manière temporaire, comme une chaîne de caractères (un texte) ou un nombre par exemple.

Déclaration et affectation

- Le mot-clé **let** permet de déclarer une ou plusieurs variables.
- Lors de la déclaration de la variable, il est possible de lui affecter une valeur par l'opérateur d'affectation (**=**).
- Si une valeur est affectée à une variable sans que cette dernière ne soit déclarée, elle sera déclarée automatiquement.
- La lecture d'une variable non déclarée provoque une erreur.

Types et déclaration de variables.

Exemple :

```
//Déclaration de i, de j et de k.  
    let i, j, k;  
//Affectation  
    i = 1; j = 5, k = i + j  
//Déclaration et affectation  
    let prix = 500;  
    let caractere = "a";  
// variable non déclarée  
    nom = "JS" ; //error
```

Le nom d'une variable :

- Commencer par une **lettre** ou un **underscore** (`_`) et **pas** par un **chiffre**;
- Contenir des lettres, des chiffres et des underscores (**pas** de **caractères spéciaux**);
- Ne doit **pas** contenir **d'espace**.

Types et déclaration de variables.

- Une variable correctement déclarée mais dont aucune valeur n'est affectée, est indéfinie (**undefined**).
- Les variables peuvent être **globales** ou **locales**.
- Une variable **globale** est déclarée en **début de script** et est accessible à n'importe quel endroit du programme.
- Une variable **locale** est déclarée à **l'intérieur d'une fonction** et n'est utilisable que dans la fonction elle-même.
- Le nom des variables est **sensible à la casse** en JavaScript (**prix** $\neq$ **Prix** $\neq$ **pRIX**)
- Généralement on utilise la convention **lower camel case** pour définir les noms de variable en JavaScript. Exp : **Let prixTotal ; Let ageEleve ;**

Types et déclaration de variables.

- La syntaxe de déclaration des variables avec **let** correspond à la nouvelle syntaxe.
- La syntaxe avec **var** est l'ancienne syntaxe qui est vouée à disparaître.

Types de données en JS :

En JavaScript, il existe 7 types de valeurs différents :

- String
- Number
- Boolean
- Null
- Undefined
- Symbol
- Object

Remarques : en JavaScript :

- Pas besoin de préciser à priori le type de valeur qu'une variable va pouvoir stocker.
- Le JavaScript va en effet automatiquement détecter quel est le type de la valeur stockée dans telle ou telle variable,
- Stocker différents types de valeurs dans une variable, sans se préoccuper d'une quelconque compatibilité.

Types et déclaration de variables.

- Pour créer ou déclarer une constante en JavaScript, nous allons utiliser le mot clef **const**.

Exemple :

```
const prenom = "isam";
```

```
const age = 21 ;
```

- Notez qu'il faut obligatoirement **initialiser** une constante lors de sa déclaration, c'est-à-dire lui passer une valeur immédiatement faute de quoi une erreur sera retournée.

Exercice

- Ecrire un script javascript qui affiche " Bonjour Tous le monde !! " sur la page html. Utiliser la méthode `write` de **l'objet** `document`
`document.write()`

JS: Syntaxe de base

Opérateurs

Un opérateur est un symbole qui va être utilisé pour effectuer certaines actions notamment sur les variables et leurs valeurs.

Opérateurs de calcul

let x = 11 ;

Signe	Nom	Signification	Exemple	Résultat
+	Plus	Addition	x+3	14
-	Moins	Soustraction	x-3	8
*	Multiplié par	Multiplication	x*2	22
**	Exposant	Puissance	x**2	1331
/	Divisé	Division	x/2	5.5
%	Modulo	Reste de la division	x%5	1
=	A la valeur	Affectation	x=5	5

Opérateurs

Un opérateur est un symbole qui va être utilisé pour effectuer certaines actions notamment sur les variables et leurs valeurs.

Opérateurs de comparaison

let x=11 ;

Signe	Nom	Exemple	Résultat
==	Egal	x==11	True
<	Inférieur	x<11	False
<=	Inférieur ou égal	x<=11	True
>	Supérieur	x>11	False
>=	Supérieur ou égal	x>=11	True
!=	Différent	x!=11	false

Opérateurs

Un opérateur est un symbole qui va être utilisé pour effectuer certaines actions notamment sur les variables et leurs valeurs.

Opérateurs de associatifs

let x=11 ;

Signe	Description	Exemple	Signification	Résultat
+=	Plus égal	x+=y	x=x+y	16
-=	Moins égal	x-=y	x=x-y	6
=	Multiplié égal	x=y	x=x*y	55
/=	Divisé égal	x/=y	x=x/y	2.2

JS: Syntaxe de base

Opérateurs

Un opérateur est un symbole qui va être utilisé pour effectuer certaines actions notamment sur les variables et leurs valeurs.

Opérateurs associatifs

let x = 11, y = 5 ;

Signe	Description	Exemple	Signification	Résultat
+=	Plus égal	x+=y	x=x+y	16
-=	Moins égal	x-=y	x=x-y	6
=	Multiplié égal	x=y	x=x*y	55
/=	Divisé égal	x/=y	x=x/y	2.2
x++	incrémentatation		x=x+1	12
y--	décrémentatation		y=y-1	4

Opérateurs

Un opérateur est un symbole qui va être utilisé pour effectuer certaines actions notamment sur les variables et leurs valeurs.

Opérateurs logiques

Signe	Nom	Exemple	Signification
&&	Et	Condition 1 && Condition 2	Condition 1 et Condition 2
 	Ou	Condition 1 Condition2	Condition 1 ou Condition2

Structures de contrôle

On appelle « structure de contrôle » un ensemble d'instructions qui permet de contrôler l'exécution du code.

Structures conditionnelles : if ... else

```
if (condition)
{
    bloc1 d'instructions
}
else
{
    bloc2 d'instructions
}
```

Remarque :

- La condition doit être toujours entourée de ()
- La séquence **else** (optionnelle) est exécutée si la condition est fausse
- les { } obligatoires qu'en cas d'instructions multiples

Structures de contrôle

Structures conditionnelles : switch (plusieurs choix)

```
<script>
  switch (Expression){
    case x:
      bloc_instructions_x;
      break;
    case y:
      bloc_instructions_y;
      break;
    ...
    case z:
      bloc_instructions_z;
      break;
    default:
      bloc_instructions_d;
  }
</script>
```

Structures de contrôle

Structures conditionnelles : switch (plusieurs choix)

Exemple :

```
<script>
  let jour = 'samedi';
  switch (jour) {
    case "vendredi":
      document.write("Enfin vendredi");
      break;
    case "samedi":
      document.write("Super Samedi");
      break;
    case "dimanche":
      document.write("Dimanche repos");
      break;
    default:
      document.write("Vive le weekend!");  }
</script>
```

Structures de contrôle

On appelle « **structure de contrôle** » un ensemble d'instructions qui permet de contrôler l'exécution du code.

Structures itératives : Boucle for

for permet d'exécuter un bloc d'instructions un certain nombre de fois :

```
for (valeur initiale; condition; step)
{
    bloc d'instructions
}
```

La condition qui suit **for** est composée de **3** éléments :

- une **valeur_initiale** portant sur une variable entière appelée compteur,
- une **condition** : tant qu'elle est vraie, la répétition est poursuivie,
- une **progression (step)** qui consiste en la mise à jour du compteur.

Structures de contrôle

Structures itératives : Boucle for

Exemple 1 : Affichage des nombres de 0 à 10 dans le document HTML.

```
<script>
    for (i=0; i<=10; i++)
    {
        document.write("Le nombre est : " + i);
        document.write("<br/>");
    }
</script>
```

Structures de contrôle

Structures itératives : Boucle for

Exemple 2 : Affichage des nombres de 0 à 15 et leurs carrés dans le document HTML.

```
<script>
```

```
    document.write("Calcul des carrés :<BR>");
```

```
    for (var i = 0; i <15; i++)
```

```
    {
```

```
        document.write("Le carrée de " + i + " est : " + i*i + "<BR>");
```

```
    }
```

```
</script>
```

Structures de contrôle

Structures itératives : Boucle while (tant que)

while permet de répéter un bloc d'instructions tant qu'une condition donnée est vraie,

```
while(condition vraie)
{
    bloc d'instructions
}
```


Structures de contrôle

Structures itératives : Boucle while

Exemple : Affichage d'un nombre dans le document HTML sous **while** :

```
<script>
    i=0;
    while(i<=10)
    {
        document.write("Le nombre est : " + i + "<BR>") ;
        i++;
    }
</script>
```

Structures de contrôle

Structures itératives : Boucle `do ... while`

while permet de répéter un bloc d'instructions tant qu'une condition donnée est vraie,

```
do
{
    bloc d'instructions
}while(condition vraie)
```

Structures de contrôle

Structures itératives : Boucle do ... while

Exemple : Affichage d'un nombre dans le document HTML sous **do ... while** .:

```
<script>
  i=0;
  do{
    document.write("Le nombre est : " + i + "<BR>") ;
    i++;
  } while(i<=10)
</script>
```

JS: Syntaxe de base

Exercices :

1. Ecrire un script qui affiche la table de multiplication du nombre 6 sur la page html.

Pour un retour à la ligne penser à ajouter la balise `<br/>` à l'affichage.
2. Modifier l'exercice précédent pour afficher le résultat dans un tableau !
3. Modifier l'exercice 2 pour que l'utilisateur puisse choisir le nombre via un prompt :


```
var nb = prompt("Saisir un nombre entier entre 1 et 10");
```
4. Afficher les tables de multiplication des nombres 1 à 10 .
5. Ecrire un script qui calcule la factorielle d'un nombre entier positif n choisi par l'utilisateur.

JS: Syntaxe de base

Correction 1 et 2 :

```
// table de multiplication de nombre 6
document.write("<h2>Table de multiplication de nombre 6</h2>");
var x;
for(x=0; x<=10; x++){
    document.write(x + " * " + "6" + " = " + x*6 + " <br/>");
}
```

```
// table de multiplication de nombre 6 formaté sur une table

var x=0;
document.write("<table border='1'>");
for(x=0; x<=10; x++){
    document.write("<tr><td>" + x + " * " + "9" + " = </td><td>"
        + x*9 + " </td></tr>");
}
document.write("</table>");
```

JS: Syntaxe de base

Correction 3 :

```
//table de multiplication d'un nombre saisié par l'utilisateur
do{
    var nb= parseInt(prompt("saisir un nombre entier entre 1 et 10"));
}while((nb<1) || (nb>10));

document.write("La table de multiplication de nombre "+ nb + " est: ");

var x=0;
document.write("<table border='1'>");
for(x=0; x<=10; x++){
    document.write("<tr><td>"+ x +" * "+ nb + " = </td><td>" + x*nb +"
</td></tr>");
}
document.write("</table>");
```

JS: Syntaxe de base

Correction 4 :

```
// les tables de multiplications des nombres de 1 à 10 :
for(var j=1;j<=10;j++){
  document.write("<h4>La table de multiplication du nombre: "+j+"</h4>")
  document.write("<table border >");
    for(let i=1;i<=10;i++){
      document.write("<tr>");
      document.write("<td>"+j+" x "+i+" =</td>");
      document.write("<td>"+j*i+"</td>");
      document.write("</tr>");
    }
  document.write("</table>");
}
```

JS: Syntaxe de base

Correction 5 :

// Calcul de factoriel d'un nombre entré par l'utilisateur :

```
let x = prompt("saisir un nombre");  
let f=1;  
for (let i=1; i<=x; i++){  
    f*=i;  
}  
alert("Factorielle de " +x + " = " +f);
```


Les Fonctions

Une **fonction** JavaScript est un **bloc de code** conçu pour effectuer une **tâche particulière**.

Une **fonction** JavaScript est **exécutée** lorsque "quelque chose" **l'invoque (l'appelle)**.

En JavaScript, il existe deux types de fonctions :

- ❑ **Fonctions propres à JavaScript** : on les appelle des **méthodes**. Elles Sont associées à un objet bien particulier

Exemple :

Méthode ***write()*** de l'objet **document**.

Méthode ***alert()*** de l'objet **window**.

- ❑ **Fonctions propres à l'utilisateur** : Ecrites par l'utilisateur.

Les Fonctions

Déclaration : fonction sans valeur de retour,

```
function    ma_fonction () {  
    ...variables et instructions  
}
```

Appel :

```
// ...  
    ma_fonction(); // Appel de la fonction maFonction  
// ...
```

Les Fonctions

Déclaration : fonction avec valeur de retour,

```
function    ma_fonction () {  
    ...variables et instructions  
    return valeurRetour;  
}
```

Appel :

```
// ...  
let valeur = ma_fonction();  
// ...
```

Les Fonctions

Déclaration : fonction avec des arguments,

```
function    ma_fonction (param1, param2, ...) {  
    instructions pouvant utiliser param1, param2, ...  
    return valeurRetour;  
}
```

Appel :

```
// ...  
let valeur = ma_fonction();  
// ...
```

Les Fonctions

Exemple : Fonction appelée ***message()*** qui affiche le texte « page is loaded ». Cette fonction sera appelée au chargement de la page .

```
<head>
  <SCRIPT>
    function message() {
      alert("page is loaded");
    }
  </SCRIPT>

</head>
<body onload="message()">
  <p>Lorem, ipsum dolor sit amet.</p>
</body>
</html>
```

Les Fonctions

Exemple : Fonction avec arguments.

```
<head>
  <SCRIPT>
    function message(texte) {
      alert(texte);
    }
  </SCRIPT>
</head>
<body>
  <button onclick="message('Bonjours à tous!!')">click</button>
  <p>Lorem, ipsum dolor sit amet.</p>
</body>
</html>
```

Les Fonctions

Exemple : Fonction avec retour.

```
<head>
  <script >
    function Carre(Nombre)
    {   var   carre = Nombre * Nombre ;
        return carre ;
    }
  </script >
</head>
<body>
  <script>
    let n = 6;
    let carre = Carre(n);
    document.write("le carre de nombre " + n + " est de " + carre)
  </script>
  <p>Lorem, ipsum dolor sit amet.</p>
</body></html>
```

Les Fonctions

Fonctions prédéfinies

- **Eval** : Cette fonction exécute un code JavaScript à partir d'une chaîne de caractères.

```
<script>  
    let x = 10;  
    let y = 20;  
    let text = "x * y";  
    let result = eval(text);  
</script>
```


Les Fonctions

Fonctions prédéfinies

- **isFinite** : Détermine si le paramètre est un nombre fini. Renvoie false si ce n'est pas un nombre ou l'infini positif ou infini négatif.

```
isFinite(240) ;  
isFinite("240") ;  
isFinite(-1.23) ;  
isFinite(5-2) ;  
isFinite("Hello") ;  
isFinite("2005/12/12");
```

```
//retourne true  
//retourne true  
//retourne true  
//retourne true  
//retourne false  
//retourne false
```

Les Fonctions

Fonctions prédéfinies

- **isNaN** : détermine si le paramètre n'est pas un nombre (NaN : Not a Number).

```
isNaN(123);
```

```
isNaN(-1.23);
```

```
isNaN(5-2);
```

```
isNaN(0);
```

```
isNaN('123');
```

```
isNaN('Hello');
```

```
isNaN('2005/12/12');
```

```
//retourne false
```

```
//retourne false
```

```
//retourne false
```

```
//retourne false
```

```
//retourne false
```

```
//retourne true
```

```
//retourne true
```

Les Fonctions

Fonctions prédéfinies

- **parseInt** : analyse une chaîne de caractères et retourne un nombre entier de la base spécifiée. La base peut prendre les valeurs 16 (hexadécimal) 10 (décimal), 8 (octal), 2 (binaire).

```
parseInt("10");  
parseInt("10.00");  
parseInt("10.33");  
parseInt("34 45 66");  
parseInt(" 60 ");  
parseInt("40 years");  
parseInt("He was 40");  
parseInt("100", 8) ;  
parseInt("F", 16) ;  
parseInt("0xF", 16) ;  
parseInt("11", 2) ;
```

```
// retourne 10  
// retourne 10  
// retourne 10  
// retourne 34  
// retourne 60  
// retourne 40  
// retourne NaN  
// retourne 64  
// retourne 15  
// retourne 15  
// retourne 3
```

Les Fonctions

Fonctions prédéfinies

- **parseFloat** : analyse une chaîne de caractères et retourne un nombre décimal.
 - ✓ Si l'argument évalué n'est pas un nombre, renvoie **NaN** (**Not a Number**).

```
parseFloat(10);  
parseFloat("10");  
parseFloat("10.33");  
parseFloat("34 45 66");  
parseFloat("40 years");  
parseFloat("He was 40");
```

```
// retourne 10  
// retourne 10  
// retourne 10.33  
// retourne 34  
// retourne 40  
// retourne NaN
```

JS: Syntaxe de base

Exercices :

6. Écrivez un programme JavaScript pour trouver le plus grand des trois entiers donnés entrés par l'utilisateur .
7. Écrivez un programme JavaScript pour renvoyer la division et le reste de deux nombres entrés par l'utilisateur (pensez à écrire deux fonctions).
8. Écrivez un programme JavaScript pour récupérer l'URL d'un site Web et l'affichez sur le console de navigateur. (Correction : `console.log(document.URL);`)
9. Créez une fonction qui renvoie **TRUE** si un entier est divisible par 5, sinon retournez **FALSE**.

JS: Syntaxe de base

Correction 6 :

```
function getMax(a, b, c) {  
    max = 0;  
    if (a > b){ max = a; }  
    else{ max = b; }  
    if (c > max){ max = c; }  
    return max; }  
  
let nb1 = parseInt(prompt("Entrez le nombre 1 :"));  
let nb2 = parseInt(prompt("Entrez le nombre 2 :"));  
let nb3 = parseInt(prompt("Entrez le nombre 3 :"));  
  
let max_nb = getMax(nb1, nb2, nb3);  
document.write("le plus grand nombre est : " + max_nb);
```

JS: Syntaxe de base

Correction 7 :

```
function Div(a, b) {  
    return a / b; }  
  
function resteDiv(a, b) {  
    return a % b; }  
  
let nb1 = parseInt(prompt("Entrez le nombre 1 :"));  
let nb2 = parseInt(prompt("Entrez le nombre 2 :"));  
  
let div = Div(nb1, nb2);  
let reste = resteDiv(nb1, nb2);  
  
document.write("La division des 2 nombres est : " + div + "<br>");  
document.write("Le reste des 2 nombres est : " + reste);
```

JS: Syntaxe de base

Correction 9 :

```
function isDivisible(n) {  
    return n % 5 === 0;  
}
```

```
let nb = parseInt(prompt("Entrez un nombre :"));
```

```
etat = isDivisible(nb);
```

```
if(etat == true)
```

```
    document.write("le nombre entré est divisible par 5");
```

```
else
```

```
    document.write("le nombre entré n'est pas divisible par 5");
```


JS: Syntaxe de base

Les tableaux

Un tableau est une variable spéciale, qui peut contenir plus d'une valeur.

Déclaration :

```
const nom_tableau = ["V1", " V2", "V3", ... ];
```

Exemple :

```
const saisons = ["printemps", "été", "automne", "hiver"];
```

Ou bien

```
const saisons = [  
  "printemps",  
  "été",  
  "automne",  
  "hiver"  
];
```

```
const saisons = [ ] ;  
  
Saison[0]= "printemps" ;  
Saison[1]= "été" ;  
Saison[2]= "automne" ;  
Saison[3]= "hiver" ;
```

Les tableaux

Un tableau est une variable spéciale, qui peut contenir plus d'une valeur.

Déclaration : en utilisant le mot clé **new**

```
const nom_tableau = new Array(dimension)
```

Exemple :

```
const saisons = new Array("printemps", "été", "automne", "hiver");
```

```
const saisons = new Array(4) ; | const saisons = new Array() ;
```

```
const saisons = new Array(4) ;
```

```
Saison[0]= "printemps" ;
```

```
Saison[1]= "été" ;
```

```
Saison[2]= "automne" ;
```

```
Saison[3]= "hiver" ;
```

Les tableaux

- pour accéder à un élément du tableau nous référons à son numéro d'index,
 - ✓ L'indice commence à **zéro**,

```
const saisons = ["printemps", "été", "automne", "hiver"];
```

```
Document.write(saisons[0]); // "printemps "
```

```
Document.write(saisons[1]); // "été "
```

```
Document.write(saisons[2]); // "automne "
```

```
Document.write(saisons[3]); // "hiver "
```

```
Document.write(saisons[4]); // "undefined "
```

- Autres déclarations :

```
var colors = ["red", "yellow", "green", "blue"];
```

```
var colors = new Array("red", "yellow", "green", "blue");
```

Les tableaux

Propriétés :

- La longueur d'un tableau est donnée par **length** de l'objet **Array** :

```
const colors = new Array("red", "green", "blue");  
let dim = colors.length; // dim vaut 3.
```

- La longueur peut être changée en ajoutant des éléments :

```
colors[3]="yellow";  
dim = colors.length; // dim vaut 4
```

Méthodes :

- reverse()* : change l'ordre des éléments,
- sort()* : trie suivant l'ordre croissant, ou suivant le modèle indiqué en paramètre,
- concat()* : permet de concaténer 2 tableaux,
- join()* : converti un tableau en chaîne de caractères,
- push(element)* : ajoute élément à la fin du tableau,
- pop()* : retourne le dernier élément du tableau et le retire du tableau.

Les tableaux

Exemple : Un carnet d'adresse avec 3 personnes

```
<script>
    carnet = new Array(3);
    //ajout des données :
    carnet[0]="ALAMI";
    carnet[1]="ALOUI";
    carnet[2]="SEMLALI";
    //pour accéder un élément, on emploie :
    document.write(carnet[2]+"<br>"); // donne SEMLALI
    document.write(carnet.join()+"<br>");
    // donne comme résultat : ALAMI, ALOUI, SEMLALI.
    document.write(carnet.join("-")+ "<br>");
    //a comme résultat : ALAMI- ALOUI- SEMLALI.
    document.write(carnet.reverse().join("-")+ "<br>");
    //donne : SEMLALI-ALAOUI-ALAMI.
</script>
```

Les tableaux

Exemple 2 :

```
<body>
  <h2>JavaScript Arrays</h2>
  <p id="demo"></p>
</body>
<script>
  const fruits = ["Banana", "Orange", "Apple", "Mango"];
  let fLen = fruits.length;

  let text = "<ul>";
  for (let i = 0; i < fLen; i++) {
    text += "<li>" + fruits[i] + "</li>";
  }
  text += "</ul>";
  document.getElementById("demo").innerHTML = text;
</script>
```

Les tableaux

JavaScript permet de créer aussi des tableaux de plusieurs dimensions : **tableau des tableaux** :

Exemple :

```
var voitures = new Array();  
var Peugeot = new Array("208", "308", "508");  
var Renault = new Array("clio", "Megane");  
var Fiat = new Array("Uno", "palio", "punto");  
voitures[0]=Peugeot; voitures[1]=Renault; voitures[2]=Fiat;  
document.write("<br>" + voitures);  
  
// 208,308,508,clio,Megane,Uno,palio,punto
```

Les tableaux

JavaScript permet de créer aussi des tableaux de plusieurs dimensions : **tableau des tableaux** :

Exemple :

```
<script type="text/javascript">
  tab=new Array(3);
  tab[0]= new Array(1,2,3); tab[1]= new Array(4,5,6);
  tab[2]= new Array(7,8,9);

  for (i=0;i<3;i++) {
    for (j=0;j<3;j++)
      document.write(tab[i][j], " " );
    document.write("<br> " );
  }
</script>
```


Les Objets (JavaScript Objects)

- Les variables sont des conteneurs pour les valeurs de données.
- Les **objets** sont aussi des variables. Mais les objets peuvent **contenir plusieurs valeurs**.

```
let car = { type:"Fiat", model:"500", color:"white" };
```

Définition de l'objet

```
let person = {firstName:"John", lastName:"Doe", age:50,  
              eyeColor:"blue"};
```

```
let person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50,  
  eyeColor: "blue"  
};
```

Accès aux propriétés des objets

```
person.firstName;    // John  
person["firstName"]; // John
```

Les Objets (JavaScript Objects)

Méthodes des objets

- Les objets peuvent aussi avoir des méthodes.
- Les méthodes sont des actions qui peuvent être exécutées sur des objets.
- Les méthodes sont stockées dans les propriétés en tant que définitions de fonctions.

```
const person = {  
  firstName: "John",  
  lastName : "Doe",  
  id        : 5566,  
  fullName : function() {  
    return this.firstName + " " + this.lastName;  
  }  
};
```

Accès aux propriétés des objets

```
person.fullName();    // John Doe
```

Les Objets (JavaScript Objects)

Création d'un objet JavaScript

```
let person = {firstName:"John", lastName:"Doe", age:50,  
              eyeColor:"blue"};
```

```
let person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50,  
  eyeColor: "blue"  
};
```

```
let person = {};  
person.firstName = "John";  
person.lastName = "Doe";  
person.age = 50;  
person.eyeColor = "blue";
```

```
const person = new Object();  
person.firstName = "John";  
person.lastName = "Doe";  
person.age = 50;  
person.eyeColor = "blue";
```

Création avec
`new Object()`.

Les Objets (JavaScript Objects)

Example : [link](#)

```
<body>
```

```
  <h2>Information personnel</h2>
```

```
  <p id='p1'></p>
```

```
  <p id='p2'></p>
```

```
  <p id='p3'></p>
```

```
  <button onclick="person.bonjour();">Click me !!</button>
```

```
  <button onclick="person.apres1ans();">Après un ans!!</button>
```

```
</body>
```

Les Objets (JavaScript Objects)

Example :

```
<script>
let person = { //nom, age et mail sont des propriétés de l'objet "person"
  nom : ['Ali', 'Amir Dine'],
  age : 29,
  mail : 'ali.amirdine@edhec.com',
  //Bonjour est une méthode de l'objet pierre
  bonjour: function(){
    alert('Bonjour, je suis ' + this.nom[0] + ', j\'ai ' + this.age + '
ans');
  },
  apres1ans: function(){
    this.age=30;
    alert('Après un ans j\'aurais ' + this.age + ' ans');
  }
};

document.getElementById('p1').innerHTML = 'Nom : ' + person.nom[0];
document.getElementById('p2').innerHTML = 'Prénom : ' + person.nom[1];
document.getElementById('p3').innerHTML = 'Age : ' + person.age;
</script>
```

Les Objets (JavaScript Objects)

Exemple (Suite) :

```
<body>
```

```
  <button onclick="person.prez();">Ma taille !!</button>
```

```
</body>
```

```
<script>
```

```
  person.taille = 170;
```

```
  person.prez = function(){
```

```
    alert('Bonjour, je suis ' + this.nom[0] + ', j\'ai ' +
```

```
    this.age + ' ans et je mesure ' + this.taille + 'cm');}
```

```
</script>
```

Refaire le même Exemple en utilisant les crochets `[]` au lieu de point `.`

Les Objets (JavaScript Objects)

Les objets prédéfinis

Objet **String** :

Propriété :

- **length** : retourne la longueur de la chaîne de caractères; [Exemple](#)

Méthodes :

- **includes()** : permet de déterminer si une chaîne de caractères est incluse dans une autre, (sensible à la casse). [Exemple](#)
- **startsWith()** / **endsWith()** : permet de déterminer si une chaîne commence/termine par une certaine sous chaîne (ou expression).
- **Substring()** : retourne une sous-chaîne de la chaîne courante à partir d'un indice de départ.
- **indexOf()** : permet de déterminer la position de la première occurrence d'un caractère ou d'une chaîne de caractères dans une chaîne de caractères de base.
- **lastIndexOf()** : la position de la dernière occurrence de l'expression cherchée qui va être renvoyée.
- **slice()** : extrait une section d'une chaîne de caractères et la retourne comme une nouvelle chaîne de caractères.

Les Objets (JavaScript Objects)

Objet **String** :

Méthodes :

- **replace()** : permet de rechercher une expression dans une chaîne de caractères et de la remplacer par une autre.
- **toLowerCase()** / **toUpperCase()** : retourne une chaîne de caractères en minuscules / majuscules.
- **trim()** : supprime les espaces ou les « blancs » en début et en fin de chaîne, [link](#)
- **match()** : La méthode `match()` recherche dans une chaîne de caractères une correspondance avec une expression régulière, et renvoie les correspondances sous la forme d'un objet de type tableau. [link](#)
- **search()** : renvoie la position de la première occurrence d'un texte spécifié dans une chaîne de caractères [link](#)
- **split()** : divise une chaîne de caractères en un tableau de sous-chaînes. [link](#)

[Autres méthodes](#)

Les Objets (JavaScript Objects)

Objet ***String*** :

Méthodes :

- ***includes()*** : permet de déterminer si une chaîne de caractères est incluse dans une autre, (sensible à la casse). [Exemple](#)
- ***startsWith()* / *endsWith()*** : permet de déterminer si une chaîne commence/termine par une certaine sous chaîne (ou expression).
- ***Substring()*** : retourne une sous-chaîne de la chaîne courante à partir d'un indice de départ.
- ***charAt()*** : renvoie le caractère se trouvant à une certaine position;
- ***charCodeAt()*** : renvoie le code du caractère se trouvant à une certaine position;
- ***concat()*** : permet de concaténer 2 chaînes de caractères;

Les Objets (JavaScript Objects)

Exemple d'opérations :

```
str = "Bonjour";  
str.length;  
/* Retourne 7 */  
str.charAt(0)/str[0]  
/* Retourne B */  
str.indexOf("o");  
/* Retourne 1 */  
str.indexOf("o",2);  
/* Retourne 4 */  
str.indexOf("x");  
/* Retourne -1 */
```

```
str.substring(0,3);  
/* Retourne Bon */  
str.substring(3,6);  
/* Retourne jou */  
tab=str.split("o");  
tab.length;  
/*Retourne 3 */  
tab.join("-");  
/*Retourne B-nj-ur */  
Mystring = str + ' ts le monde'  
/*Retourne Bonjour ts le monde*/
```

Les Objets (JavaScript Objects)

Les objets prédéfinis

Objet *Array* :

Propriété :

- **length** : retourne le nombre d'éléments du tableau;

Méthodes :

- **concat()** : permet de concaténer 2 tableaux; [link](#)
- **join()** : converti un tableau en chaîne de caractères; [link](#)
- **sort()** : permet le classement des éléments du tableau; [link](#)
- **reverse()** : inverse le classement des éléments du tableau; [link](#)
- **slice(debut, fin)** : retourne une section du tableau; [link](#)

Les Objets (JavaScript Objects)

Les objets prédéfinis

Objet **Math** :

Propriété :

- **E** : renvoie la valeur de la constante d'Euler (~ 2.718);
- **LN2** : renvoie le logarithme népérien de 2 (~ 0.693);
- **LN10** : renvoie le logarithme népérien de 10 (~ 2.302);
- **LOG2E** : renvoie le logarithme en base 2 de e (~ 1.442);
- **LOG10E** : renvoie le logarithme en base 10 de e (~ 0.434);
- **PI** : renvoie la valeur du nombre pi (~ 3.14159);
- **SQRT1_2** : renvoie 1 sur racine carrée de 2 (~ 0.707);
- **SQRT2** : renvoie la racine carrée de 2 (~ 1.414).

Les Objets (JavaScript Objects)

Les objets prédéfinis

Objet **Math** :

Méthodes :

- *abs(), exp(), log(), sin(), cos(), tan(), asin(), acos(), atan(), max(), min(), sqrt()* sont les opérations mathématiques habituelles;
- *atan2()* : retourne la valeur radian de l'angle entre l'axe des abscisses et un point;
- *ceil()* : retourne le plus petit entier supérieur à un nombre;
- *floor()* : retourne le plus grand entier inférieur à un nombre;
- *pow()* : retourne le résultat d'un nombre mis à une certaine puissance;
- *random()* : retourne un nombre aléatoire entre 0 et 1;
- *round()* : arrondi un nombre à l'entier le plus proche.

Les Objets (JavaScript Objects)

Les objets prédéfinis

Objet **Date** :

Propriété : *aucune*;

Méthodes :

- ***getFullYear()***, ***getYear()***, ***getMonth()***, ***getDay()***, ***getDate()***, ***getHours()***, ***getMinutes()***, ***getSeconds()***, ***getMilliseconds()***:
retournent respectivement l'année complète, l'année (2chiffres), le mois, le jour de la semaine, le jour du mois, l'heure, les minutes, les secondes et les millisecondes stockés dans l'objet Date;
- ***getUTCFULLYear()*** : retournent l'année complète stockés dans l'objet Date en temps universel; [link](#)
- ***setFULLYear()*** : remplacent l'année complète dans l'objet Date; [link](#)

Les Objets (JavaScript Objects)

Les objets prédéfinis

Objet **Date** :

Méthodes :

- ***getTime()*** : retourne le nombre de millisecondes depuis le 1er janvier 1970; [link](#)
- ***getTimezoneOffset()*** : retourne la différence entre l'heure du client et le temps universel; [link](#)
- ***toUTCString()*** : convertissent la date en chaîne de caractère, selon la convention du temps universel; [link](#)
- ***toLocaleString()*** : convertissent la date en chaîne de caractère, selon la convention du temps locale; [Link](#)
- ... [Link](#)

Les Objets (JavaScript Objects)

L'opérateur new

Utilisé pour créer une nouvelle instance **défini par l'utilisateur** ou de l'un des types d'objets prédéfinis, **Array**, **Boolean**, **Date**, **Function**, **Image**, **Number**, **Object**, ou **String**.

Syntaxe

```
unObjet = new type_objet(parametres) ;
```

Exemple

```
text = new String("Bonjour ts le monde");  
text.length ;    // retourne 19  
text.substring(0,7) ; // retourne Bonjour
```

...

Les Objets (JavaScript Objects)

Exemple : `new Date()`

- ✓ `Let d = new Date();`
 `// retourne la date actuelle,`
 `// Wed Mar 02 2022 18:04:19 GMT+0100 (UTC+01:00)`
- ✓ `Let d = new Date(0);`
 `// Thu Jan 01 1970 00:00:00 GMT+0000 (UTC)`
- ✓ `Let d = new Date(15489732654);`
 `// la date après 15489732654 ms (millisecondes) à partir du 1er janvier 1970.`
 `// Mon Jun 29 1970 06:42:12 GMT+0000 (UTC)`
- ✓ `let d = new Date(2021,6,15,12,30,28,110);`
 `// Thu Jul 15 2021 12:30:28 GMT+0100 (UTC+01:00)`
- ✓ `let d = new Date(2021,6,15);`
 `// Thu Jul 15 2021 00:00:00 GMT+0100 (UTC+01:00)`

Les Objets (JavaScript Objects)

Exemple : `new Date()`

- ✓ `let d = new Date("2021-03-15");`
✓ Mon Mar 15 2021 01:00:00 GMT+0100 (UTC+01:00) /* Format ISO courte */
- ✓ `let d = new Date("2021-03-15T12:36:50");`
✓ Mon Mar 15 2021 12:36:50 GMT+0100 (UTC+01:00) /*Format ISO complète*/
- ✓ `let d = new Date("03/15/2021");`
✓ Mon Mar 15 2021 00:00:00 GMT+0100 (UTC+01:00) /* Format courte */
- ✓ `let d = new Date("Mars 15 2021 12:36:50");`
✓ Mon Mar 15 2021 12:36:50 GMT+0100 (UTC+01:00) /* Format longue */
- ✓ `let d = new Date("Mon Mars 15 2021 12:36:50 GMT+0100");`
✓ Mon Mar 15 2021 12:36:50 GMT+0100 (UTC+01:00) /* Format Complète */

Les Objets (JavaScript Objects)

Exemple : `new Date()`

```
var date=new Date();  
var mois = date.getMonth();  
mois++;  
console.log("Nous somme le "+date.getDate()+" - "+ mois);  
console.log("année "+date.getYear());  
console.log("année "+date.getDay());  
console.log("année "+date.getHours());  
console.log("année "+date.getMinutes());  
console.log("année "+date.getFullYear());
```

Les Objets (JavaScript Objects)

Les objets du navigateur

- JavaScript divise la page du navigateur en objets.
- Pour accéder à un objet, on commence par le plus haut, puis on descend dans la hiérarchie, jusqu'à l'objet voulu.
- L'objet le plus grand est l'objet fenêtre (**window**).
- Dans la fenêtre **window** s'affiche une page, c'est l'objet **document**. Cette page peut contenir plusieurs **objets**, comme des **formulaires**, des **images**, ...
- Pour accéder à un objet, il faudra donner le chemin complet de cet objet :

Exemple : **window.document.form[].radio[];**

Ou bien

document.form[].radio[];

// window n'est pas obligatoire

KICH ISMAIL

Les Objets (JavaScript Objects)

Les objets du navigateur

Objet **Window** :

Propriété :

- **closed** : indique que la fenêtre a été fermée;
- **defaultStatus** : indique le message par défaut dans la barre de status;
- **document** : retourne l'objet document de la fenêtre;
- **frames** : retourne la collection de cadres dans la fenêtre;
- **history** : retourne l'historique de la session de navigation;
- **location** : retourne l'adresse actuellement visitée;
- **name** : indique le nom de la fenêtre;
- **navigator** : retourne le navigateur utilisé;
- **opener** : retourne l'objet window qui a créé la fenêtre en cours;
- **parent** : retourne l'objet window immédiatement supérieur dans la hiérarchie;
- **self** : retourne l'objet window correspondant à la fenêtre en cours;
- **status** : indique le message affiché dans la barre de status;
- **top** : retourne l'objet window le plus haut dans la hiérarchie.

Les Objets (JavaScript Objects)

Les objets du navigateur

Objet **Window** :

Méthodes :

- **blur()** : enlève le focus de la fenêtre;
- **close()** : ferme la fenêtre;
- **focus()** : place le focus sur la fenêtre;
- **moveBy()** : déplace d'une distance;
- **moveTo()** : déplace la fenêtre vers un point spécifié;
- **open()** : ouvre une nouvelle fenêtre;
- **print()** : imprime le contenu de la fenêtre;
- **resizeBy()** : redimensionne d'un certain rapport;
- **resizeTo()** : redimensionne la fenêtre;
- **setTimeout()** : évalue une chaîne de caractère après un certain laps de temps.

Les Objets (JavaScript Objects)

Les objets du navigateur

Objet **document** :

Propriété :

- **applets** : retourne la collection d'applets java présente dans le document;
- **cookie** : permet de stocker un cookie;
- **domain** : indique le nom de domaine du serveur ayant apporté le document;
- **forms** : retourne la collection de formulaires présents dans le document;
- **images** : retourne la collection d'images présentes dans le document;
- **links** : retourne la collection de liens présents dans le document;

Méthodes :

- **close()** : ferme le document en écriture;
- **open()** : ouvre le document en écriture;
- **write()** : écrit dans le document;
- **writeln()** : écrit dans le document et effectue un retour à la ligne.

Les objets prédéfinis : **WINDOW**

L'objet **window** possède plusieurs séries de méthodes: boîtes de dialogue, minuteries, ouverture et fermeture des fenêtres...

Les boîtes de dialogues

- La méthode **alert()** qui permet d'afficher le message en paramètres dans une boîte de dialogue (objet **window**).
- La méthode **confirm()** qui affiche une boîte de dialogue de confirmation qui donne la possibilité de choisir entre 'OK' ou 'Annuler'. Cette méthode est très utile pour confirmer les actions de l'utilisateur sur le navigateur.
- La méthode **prompt()** qui affiche une boîte de dialogue avec une zone de texte qui peut être remplie par l'utilisateur. La valeur de **prompt()** est ensuite transmise au navigateur.

Les objets prédéfinis : **WINDOW**

Exemple :

```
/* Récupérer le nom saisi sur la boîte de dialogue prompt dans une variable nom */  
  
var nom = prompt("Quel est votre nom?");  
  
var conf = confirm("Vous vous appelez vraiment "+nom);  
  
/* confirm renvoie true si l'utilisateur clique sur ok et false dans le cas contraire */  
  
if(conf)  
    alert("Bonjour " + nom);  
  
else  
    alert("Bonjour inconnu");
```

Les objets prédéfinis : **WINDOW**

La minuterie

La syntaxe de la minuterie est la suivante:

Compteur=**setTimeout**("fonction()", **Durée_en_milliseconde**)

- À la fin de la durée spécifiée en milliseconde la fonction sera exécutée.
- La variable '**Compteur**' est l'identificateur du compteur qui permet d'arrêter prématurément celui-là à l'aide de la fonction:

ClearTimeout(Compteur)

Les objets prédéfinis : **WINDOW**

Exemple :

Soit la fonction

```
function lancementDiffere() {  
  
    setTimeout("alert('lancement différé');",5000);  
  
}
```

- Dans la page HTML ajouter la fonction à l'événement **onload** de l'élément **body**:

```
<body onload="lancementDiffere()">
```

- Lancer la page et attendre 5 secondes.

Les objets prédéfinis : **WINDOW**

Ouverture

La méthode **open()** permet d'ouvrir une nouvelle fenêtre (Pop-Up) entièrement personnalisable.

- La syntaxe est la suivante:

Open("URL", "nom_de_la_page", "options_de_la_fenêtre")

Options	Description
Location=yes/no	Affiche ou non la barre d'adresse
Menubar=yes/no	Affiche ou non la barre de menu
Resizable=yes/no	Permet ou non le redimensionnement de la page
Scrollbar=yes/no	Affiche ou non les ascenseurs
Status=yes/no	Affiche ou non la barre d'état
Toolbar=yes/no	Affiche ou non la barre d'outils
Width=largeur en px	Largeur de la fenêtre

[Link](#)

Les objets prédéfinis : **WINDOW**

Exemple :

```
<html>
```

```
<body>
```

```
  <h1>The Window Object</h1>
```

```
  <h2>The open() and close() Methods</h2>
```

```
  <button onclick="openWin()">Open "myWindow"</button>
```

```
  <button onclick="closeWin()">Close "myWindow"</button>
```

```
</body>
```

```
</html>
```

Les objets prédéfinis : **WINDOW**

Exemple :

```
<script>
let myWindow;

function openWin() {
    myWindow = window.open("", "test", "width=200,height=100");
}

function closeWin() {
    myWindow.close();
}
</script>
```

[link](#)

Les objets prédéfinis : **WINDOW**

Fermeture

La méthode **close()** permet de fermer une fenêtre,

- Le plus souvent, la fenêtre courante en utilisant **self.close()**;

Exemple :

```
<input type="button" value="Fermer cette fenetre"  
      onclick="self.close();" />
```

Les objets prédéfinis : **DOCUMENT**

- ❑ Se trouve sous l'objet **window**.
- ❑ Plusieurs méthodes et propriétés existent:
 - **write()**: méthode qui permet d'écrire dans le document.
 - **writeln()**: méthode qui permet d'écrire dans le document avec un retour à la ligne (écriture ligne par ligne).
 - **getElementById()**: méthode DOM (Document Object Model) qui permet l'accès aux éléments par l'attribut id.
 - **getElementByName()**: méthode DOM qui permet l'accès aux éléments par l'attribut name.

Les objets prédéfinis : **DOCUMENT**

- ❑ Se trouve sous l'objet **window**.
- ❑ Plusieurs méthodes et propriétés existent:
 - **images**: sous objet de l'objet document qui permet d'avoir l'accès aux graphiques (images) du fichier HTML. [Link](#) , [link](#)
 - **cookie**: propriété qui permet de déposer ou récupérer un cookie chez le client. Les cookies (biscuits) sont des fichiers que des sites web déposent chez le client en général pour sauvegarder des préférences.
 - **lastModified**: propriété qui permet de récupérer la date de la dernière modification de la page en question.

Les objets prédéfinis : **DOCUMENT**

Exemple :

```
document.cookie=document.lastModified;
```

On ajoute au cookie déposé chez le client la date de la dernière mise à jour du site.

[link](#)

JS: Syntaxe de base

Exercice :

1. Ecrire une fonction javascript qui permet d'afficher, lors du démarrage de la page, "BONJOUR" si l'heure est entre 6H et 12H, "Happy Afternoon" si l'heure courante est entre 13H et 17H et finalement "Heureuse soirée" sinon.
2. Ecrire une fonction qui déclare un tableau de noms de pays et qui permet de tirer au sort un de ces pays. Utiliser la fonction `Math.random()` qui fournit un nombre réel entre 0 et 0,999

JS: Syntaxe de base

Solution 1 :

```
<script>
```

```
function hello(){
```

```
    var date = new Date();
```

```
    var heure =date.getHours();
```

```
    if ((heure<13) && (heure>5)) alert ("GOOD MORNING");
```

```
    else if ((heure<18) && (heure>12))
```

```
        alert("GOOD AFTERNOON");
```

```
    else alert("GOOD NIGHT");
```

```
}
```

```
</script>
```

JS: Syntaxe de base

Solution 2:

```
<script>
```

```
var pays=['MAroc','Algerie','Tunisie','Lybie','Egypte','Soudan'];
```

```
var x = Math.floor(Math.random() * pays.length);
```

```
// Returns a random integer from 0 to 5
```

```
    alert("indice aléatoire  = " + x);
```

```
    alert("pays choisi au hasard est "+ pays[x]);
```

```
</script>
```

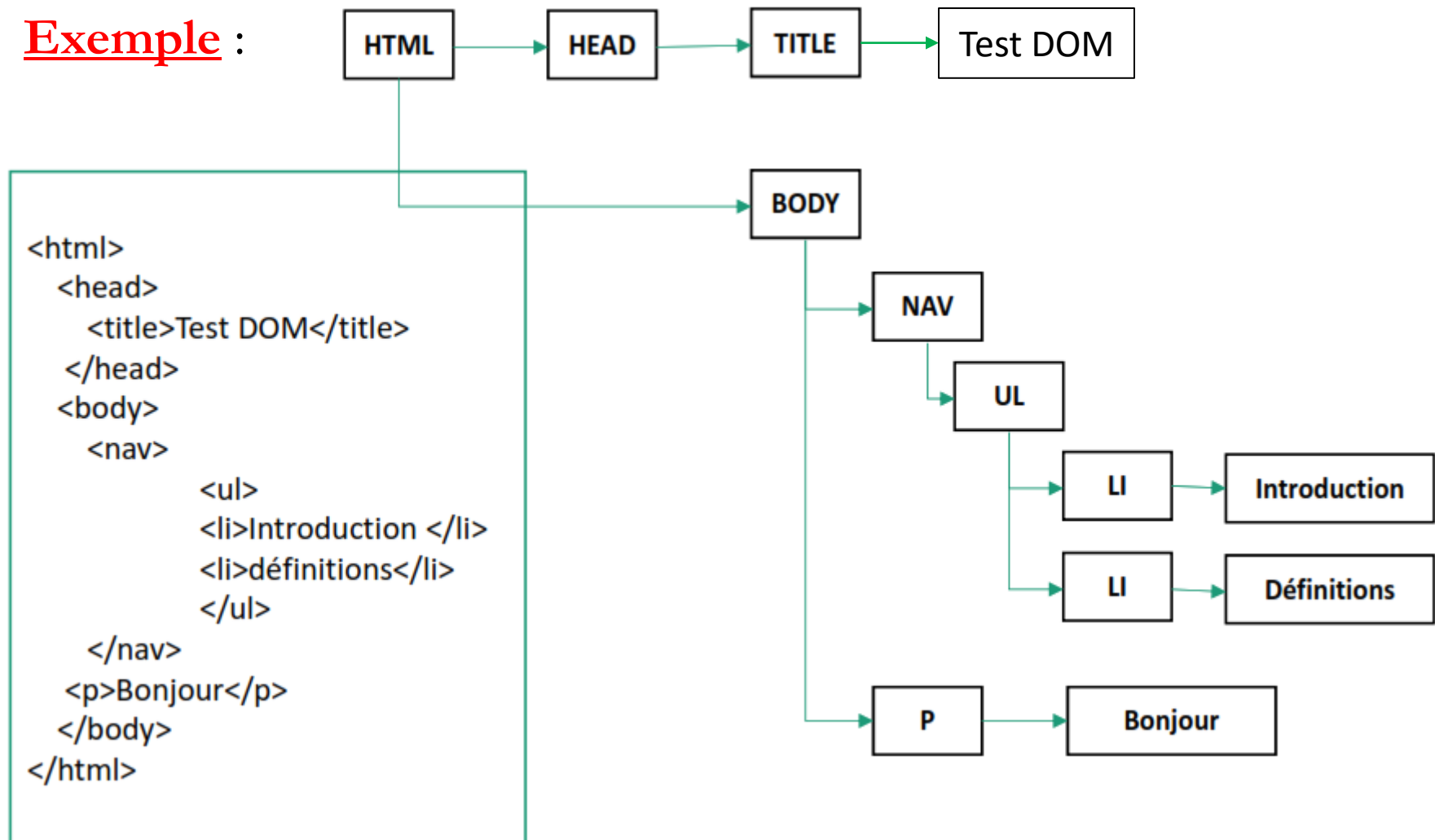
Document Object Model

- **Document Object Model** ou "modèle d'objet de document" est une interface créée automatiquement par le navigateur lors du chargement d'une page HTML.
- Représentation d'une page web sous la forme d'une hiérarchie d'objets, où chaque objet correspond à un nœud de l'arborescence de la page.
- Les objets du **DOM** disposent de propriétés et de méthodes permettant de les manipuler avec JavaScript.
- Le **DOM** est une interface de programmation pour des documents **HTML** ou **XML** qui représente le document sous une forme qui permet aux langages de **script** comme le **JavaScript** d'y accéder et d'en manipuler le contenu et les styles.

JS: Syntaxe de base

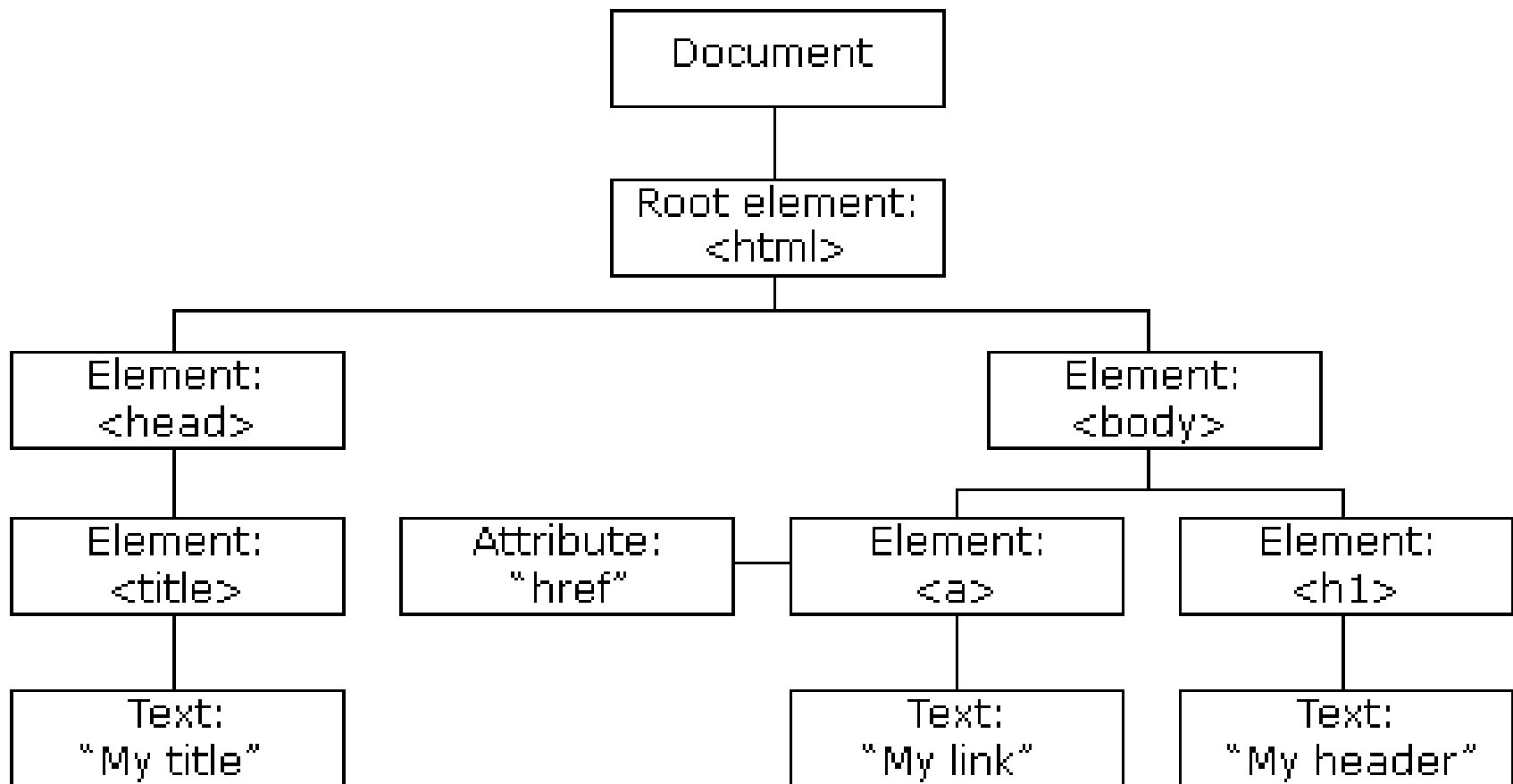
Document Object Model

■ Exemple :



Document Object Model

- Exemple :



Document Object Model

- ❑ Chaque entité de l'arborescence est appelée un **nœud** (node).
- ❑ On distingue deux types de nœuds :
 - Les **éléments HTML**, comme `<body>` ou `<p>`. Ces nœuds peuvent avoir des **sous-nœuds**, appelés fils ou enfants (**children**).
 - Le **contenu contextuel**.

Document Object Model

Avec le **modèle objet**, JavaScript obtient toute la puissance dont il a besoin pour créer du

HTML **dynamique** :

- JavaScript peut **modifier** tous les **éléments HTML** de la page.
- JavaScript peut **modifier** tous les **attributs HTML** de la page.
- JavaScript peut **modifier** tous les **styles CSS** de la page.
- JavaScript peut **supprimer** des **éléments** et **attributs HTML** existants.
- JavaScript peut **ajouter** de nouveaux **éléments** et **attributs HTML**.
- JavaScript peut **réagir** à tous les **événements HTML** existants dans la page.
- JavaScript peut **créer** de **nouveaux événements HTML** dans la page

Parcours avec Javascript : DOM

L'objet **document** est le point de départ du DOM. Il représente votre page web (votre document) entière.

- ☐ L'accès à un élément du DOM se fait via l'objet **document**.
- ☐ Il possède de nombreuses **propriétés** et **méthodes**.

Parcours avec Javascript : DOM

`document.getElementById("id")`

Cette méthode permet de sélectionner un élément **d'identifiant** donné dans une page.

Exemple : `<p id="intro">Contenu</p>`

- `document.getElementById("intro")` : sélectionne précisément l'élément **p** en question.

Parcours avec Javascript : DOM

Exemple :

```
<script>
```

```
  let sayHello = function(){  
    document.getElementById("intro").innerHTML="Hello every Body";  
  }
```

```
</script>
```

```
<body>
```

```
  <p>paragraphe 1</p>
```

```
  <p id="intro">paragraphe 2</p>
```

```
  <button onclick="sayHello()">click</button>
```

```
</body>
```

Parcours avec Javascript : DOM

`document.getElementsByClassName("className")`

Cette méthode fonctionne de la même manière que la précédente, mais fera sa recherche sur la **class** des éléments et retournera la liste des éléments qui correspondent.

Exemple : `<p class="intro">Contenu 1</p>`
`<p class="intro">Contenu 2</p>`

- `document.getElementsByClassName("intro")` : retrouve la liste des éléments ayant la classe **intro**.

Parcours avec Javascript : DOM

Exemple :

<body>

<p class="intro">paragraphe 1</p>

<p class="intro">paragraphe 2</p>

<p class="intro">paragraphe 3</p>

<p>le nombre des élément avec class "intro" est: </p>

**
**

<button onclick="sayHello()">click</button>

<button onclick="numbElement()">number elements</button>

</body>

Parcours avec Javascript : DOM

Exemple :

```
<script>

let sayHello = function(){

let contenu = document.getElementsByClassName("intro");

    contenu[2].innerHTML="Hellow every Body";}

let numbElement = function(){

    let num = document.getElementsByClassName("intro");

    document.getElementById("demo").innerHTML= num.length;

    return num;}

</script>
```

Exemple

Parcours avec Javascript : DOM

`document.getElementsByTagName("name")`

Avec cette méthode, vous rechercherez tous les éléments avec un **nom de balise** bien précis (par exemple tous les **liens** (`a`), tous les **boutons** (`button`)...).

De la même manière que la méthode précédente, vous récupérerez la liste des éléments correspondants.

Exemple : `<p class="intro">Contenu 1</p>`
`<p class="intro">Contenu 2</p>`

- `document.getElementsByTagName("p")` : retrouve la liste des

éléments de type `p`

Parcours avec Javascript : DOM

Exemple :

```
<script>
```

```
    const articles = document.getElementsByTagName('article');
```

```
    const thirdArticle = articles[2];
```

```
</script>
```

```
<body>
```

```
    <div>
```

```
        <article>Contenu 1</article>
```

```
        <article>Contenu 2</article>
```

```
        <article>Contenu 3</article>
```

```
    </div>
```

```
</body>
```

`articles.length`

3

Parcours avec Javascript : DOM

document.querySelector()("selector")

Cette méthode est plus complexe, mais aussi beaucoup plus puissante car elle vous permet de faire une recherche complexe dans le DOM, en mélangeant plusieurs procédés. Il s'agit en fait d'un sélecteur qui permet de cibler certains éléments..

Exemple :

Par exemple, **document.querySelector("#myId p.article > a")** fera une recherche dans l'élément ayant pour id **#myId**, les éléments de type **<p>** qui ont pour classe **article**, afin de récupérer le lien **<a>** qui est un enfant direct (pas des enfants de ses enfants).

Parcours avec Javascript : DOM

Exemple :

```
document.querySelector("#myId p.article > a")
```

```
<div id="myId">
  <p>
    <span><a href="#">Lien 1</a></span>
    <a href="#">Lien 2</a>
    <span><a href="#">Lien 3</a></span>
  </p>
  <p class="article">
    <span><a href="#">Lien 4</a></span>
    <a href="#">Lien 5</a>
  </p>
  <p>
    <a href="#">Lien 6</a>
    <span><a href="#">Lien 7</a></span>
  </p>
</div>
```

Exemple 2

```
const elt = document.querySelector("#myId p.article > a");
```

Lien 5

Parcours avec Javascript : DOM

`document.getElementsByName()("name")`

Cette méthode permet de sélectionner les éléments portant un nom (attribut **name**) donné dans une page

- Elle retourne un tableau d'éléments portant le même nom (**name**)

Exemple :

```
<p class="intro" name="parg1">Bonjour</p>  
<p id="id1" name="parg1">tous le monde</p>
```

- `document.getElementsByName("parg1")` retrouver la liste des éléments avec le nom **parg1**

Parcours avec Javascript : DOM

Exemple :

```
<script>
```

```
function testDom(){  
    var x = document.getElementsByName('parg1');  
    alert("le nombre d\'élément est : " + x.length);  
    for(var i=0; i<x.length; i++)  
        alert("element numero "+i+" : "+x[i].innerHTML) ;}
```

```
</script>
```

```
<body>
```

```
<p class="intro" name="parg1">Bonjour</p>  
<p id="id1" name="parg1">tous le monde</p>  
<button onclick="testDom()">click</button>
```

```
</body>
```

Exemple

JS: Syntaxe de base

Parcours avec Javascript : DOM

innerHTML vs innerText vs textContent

```
<div id='blog test'>
```

```
  This element is <strong>strong</strong> and      has some super fun  
<code>code</code>!  
</div>
```

```
const getValue = document.getElementById('blog-test');
```

getValue.innerHTML

This element is strong and has some super fun
<code>code</code>!

getValue.innerText

This element is strong and has some super fun code!

getValue.textContent

This element is strong and has some super fun code!

Exemple

Parcours avec Javascript : DOM

Exercice :

```
<body>
<label><input type="radio" name="check" value="1"/>Case n°1</label>
<br />
<label><input type="radio" name="check" value="2"/>Case n°2</label>
<br />
<label><input type="radio" name="check" value="3"/>Case n°3</label>
<br />
<label><input type="radio" name="check" value="4"/>Case n°4</label>
<br>
<input type="button" value="Afficher la case cochée"
onclick="check();" />
</body>
```


Parcours avec Javascript : DOM

```
<script>
```

```
function check() {  
    var inputs = document.getElementsByTagName('input');  
    var nbElements = inputs.length;  
    for (var i = 0 ; i < nbElements ; i++) {  
        if (inputs[i].checked) {  
            alert('La case cochée est la n°'+ inputs[i].value);  
        }  
    }  
}
```

```
</script>
```

[LINK](#)

Parcours avec Javascript : DOM

- La propriété **parentNode** de l'interface Node renvoie le parent du nœud spécifié dans l'arborescence du **DOM** ou **null** si le nœud ne possède pas de parent.
- La propriété **childNodes** de cette même interface renvoie une liste des nœuds enfants de l'élément donné.

```
<body>
  <h1>Titre principal</h1>
  <p id='p1'>Un paragraphe <span>avec un span</span</p>
<div>
  <p id='p2'>Un paragraphe dans le div</p>
  <p>Un autre paragraphe dans le div</p>
</div>
<p>Un autre paragraphe</p>
</body>
```

[Link](#)

Parcours avec Javascript : DOM

```
<script>
  let p1 = document.getElementById('p1');
  let p2 = document.getElementById('p2');

  p2.parentNode.style.backgroundColor='RGBA(240,160,000,0.5)';

  let enfantsP1 = p1.childNodes;
  console.log(enfantsP1);

  enfantsP1[1].style.backgroundColor = 'yellow';

  let enfantsEltP1 = p1.children;
  console.log(enfantsEltP1);

  enfantsEltP1[0].style.textDecoration = 'underline';
</script>
```

JS: Syntaxe de base

Ajouter, modifier ou supprimer des éléments du DOM

- ❑ **createElement()** : créer un nouvel élément HTML en JavaScript via l'interface Document.

```
let newP = document.createElement('p');  
newP.textContent = 'Paragraphe créé et inséré grâce au JavaScript'
```

- ❑ **createTextNode()** : créer directement un nœud de type texte via l'interface Document.

```
let newTexte = document.createTextNode('Texte écrit en JavaScript');
```

- ❑ **prepend()** : permet d'insérer un nœud ou de texte avant le premier enfant d'un nœud.
- ❑ **append()** : permet d'insérer un nœud ou de texte après le dernier enfant d'un nœud.

[Exemple](#)

Ajouter, modifier ou supprimer des éléments du DOM

- ❑ **insertBefore()** : permet pour sa part d'insérer un nœud en tant qu'enfant d'un autre nœud juste avant un certain nœud enfant donné de ce parent.

```
let b = document.body;  
let p1 = document.getElementById('p1');  
let newP = document.createElement('p');  
newP.textContent = 'Paragraphe créé et inséré grâce au JavaScript';  
b.insertBefore(newP, p1);
```

[Exemple](#)

- ❑ **insertBefore()** & **appendChild()** : permettent de déplacer un nœud dans le DOM, leur passant en argument un nœud qui existe déjà et qui est déjà placé dans le DOM.

```
let b = document.body;  
let p1 = document.getElementById('p1');  
let p4 = b.lastElementChild; //On accède au dernier paragraphe  
b.insertBefore(p1, p4); //On déplace p1 juste avant p4 dans le DOM
```

JS: Syntaxe de base

Ajouter, modifier ou supprimer des éléments du DOM

❑ **remove()** : permet supprimer totalement un nœud du DOM.

```
let b = document.body;  
let p1 = document.getElementById('p1');  
let p2 = document.getElementById('p2');
```

//Supprimer p1 et p2 du DOM

```
p1.remove();
```

```
p2.remove();
```

[Exemple](#)

❑ **appendChild()** : permet d'ajouter un nœud (élément) comme dernier enfant d'un élément. [Exemple](#)

[Autres : HTML DOM Elements](#)

Validation du formulaire : Rappel

Rappel : La validation d'un formulaire côté client est la validation effectuée dans le navigateur, avant que les données n'aient été soumises au serveur. Il est possible de la subdiviser en :

- ✓ la validation de formulaire intégrée avec les fonctions de validation de formulaire **HTML5** (non personnalisable),
- ✓ la validation **JavaScript**, codée en **JavaScript** (entièrement personnalisable).

Validation du formulaire : Rappel

HTML Input Attributes

- L'élément **<input>** peut être affiché de plusieurs façons, en fonction de l'attribut type :

<input type="text"> (default value)

<input type="button">

<input type="checkbox">

<input type="color">

<input type="date">

<input type="datetime-local">

<input type="email">

<input type="file">

<input type="hidden">

<input type="image">

<input type="month">

<input type="number">

<input type="password">

<input type="radio">

<input type="range">

<input type="reset">

<input type="search">

<input type="submit">

<input type="tel">

<input type="time">

<input type="url">

<input type="week">

https://www.w3schools.com/tags/tag_input.asp

KICH ISMAIL

Validation du formulaire : Rappel

HTML Input Attributes

- **placeholder** : spécifie une courte indication décrivant la valeur attendue d'un champ de saisie,
- **value** : Specifies the value of an **<input>** element
- **disabled** : spécifie que l'élément d'entrée doit être désactivé,
- **readonly** : spécifie qu'un champ de saisie est en lecture seule,
- **size** : spécifie la largeur (en caractères) d'un champ de saisie,
- **multiple** : spécifie que l'utilisateur est autorisé à saisir plus d'une valeur dans un champ de saisie,
- **autofocus** : spécifie qu'un champ de saisie doit automatiquement être mis en évidence lors du chargement de la page.
- **list** : refers to a **<datalist>** element that contains pre-defined options for an **<input>** element. [link](#)
- ...

https://www.w3schools.com/html/html_form_attributes.asp

Validation du formulaire : Rappel

HTML Input Attributes

- **required** : rendre une entrée **obligatoire**,
- **max** / **min** : Spécifie la valeur **maximale** / **minimale** d'un élément d'entrée
- **minlength** et **maxlength** : le champ sera invalide si sa taille est inférieure à la valeur **minlength** ou supérieure la valeur **maxlength**,
- **pattern** : qui attend une expression régulière comme valeur,

Validation du formulaire

Validation selon une expression régulière :

- On utilise l'attribut **pattern**, qui attend une **expression régulière** comme valeur,
- Une **expression régulière** (**regex**) est un modèle qui peut être utilisé pour faire correspondre des combinaisons de caractères dans des chaînes de texte,

Vous trouverez ci-dessous quelques exemples pour vous donner une idée de base de leur fonctionnement :

- ✓ **a** : correspond à un caractère qui doit être un a (ni b, ni aa, etc.)
- ✓ **abc** : correspond à a, suivi de b, suivi de c.
- ✓ **a*** : correspond au caractère a, absent ou présent plusieurs fois (+ correspond à un caractère une ou plusieurs fois).
- ✓ **[^a]** : correspond à un caractère qui n'est pas un a.
- ✓ **a|b** : correspond à un caractère qui est a ou b.
- ✓ **[abc]** : correspond à un caractère qui est a, b ou c.
- ✓ **[^abc]** : correspond à un caractère qui n'est pas a, b ou c.

Validation du formulaire

- ✓ **[a-z]** : correspond à tout caractère de la plage a–z, en minuscules seulement (utilisez **[A-Za-z]** pour minuscules et majuscules et **[A-Z]** pour les majuscules uniquement).
- ✓ **a.c** : correspond à a, suivi par n'importe quel caractère, suivi par c.
- ✓ **a{5}** : correspond à a, 5 fois.
- ✓ **a{5,7}** : correspond à a, 5 à 7 fois, mais ni plus, ni moins.
- ✓ **[\s-]** : correspond à une espace ou un tiret.
- ✓ **[0-9]** : correspond à tout nombre compris entre 0 et 9.
- ✓ **[Ll].*k** : Un seul caractère L en majuscules ou minuscules, suivi de zéro ou plusieurs caractères de n'importe quel type, suivis par un k minuscules.
- ✓ **[A-Z][A-Za-z' -]+** : Un seul caractère en majuscules suivi par un ou plusieurs caractères en majuscules ou minuscules, un tiret, une apostrophe ou une espace,
- ✓ **[0-9]{3}[\s-][0-9]{3}[\s-][0-9]{4}** : Un schéma pour un numéro de téléphone intérieur américain — trois chiffres, suivis par une espace ou un tiret, suivis par trois nombres, suivis par une espace ou un tiret, suivis par quatre nombres.

Validation du formulaire

Exemple :

```
<body>
  <form>
    <table>
      <tr><td><label for="choose">Nom complet :</label></td>
      <td><input id="choose" name="i_like" required pattern=[A-Za-
z\s]+></td></tr>

      <tr><td><label for="Nai">Date de naissance :</label></td>
      <td><input id="Nai" name="DN" required pattern=[0-9]{2}[\/][0-
9]{2}[\/][0-9]{4}><br></td></tr>

      <tr><td><label for="tel">Numéro de téléphone :</label></td>
      <td><input id="tel" name="numTel" required pattern=[0-9]{2}[\s-
][0-9]{2}[\s-][0-9]{2}[\s-][0-9]{2}></td></tr>
```

Validation du formulaire

```
<tr><td><label for="email">Adresse Email :</label></td>
  <td><input id="email" name="email" required pattern="^[a-zA-Z0-9+_.-]+@[a-zA-Z0-9.-]+$></td></tr>

<tr><td><label for="choose">Le langage C# ou Python :</label></td>
  <td><input id="choose" name="i_like" required
pattern="C#|Python"></td></tr>

<tr><td><label for="niv">Niveau en programmation ?</label></td>
  <td><input type="number" id="niv" name="niveau" required max="5"
min="0" step="1"></td></tr>

<tr><td><button>Soumettre</button></td></tr>
</table>
</form>
</body>
```

[LINK](#)

JS : Validation du formulaire

- JavaScript peut exploiter les objets du navigateur selon des méthodes Définies.
- Les objets sont classés hiérarchiquement, pour accéder à un objet, il faut d'abord passer par l'objet parent.

Exemple :

```
<html>
<head></head>
<body>
  <form name="mon_form">
    <input type="text" name="mon_text"/>
  </form>
</body>
</html>
```

- Pour accéder à la valeur du champ texte **"mon_text"**:
- L'objet **mon_text** se trouve dans **form**, **form** se trouve dans **document** (body) et **document** se trouve dans **window** (la fenêtre de la page html)
- On doit écrire:

window.document.mon_form.mon_text

- Puisque le premier objet est « **window** » on peut toujours ne pas le mentionner lors de l'appel hiérarchique aux objets:

document.mon_form.mon_text

JS : Validation du formulaire

Exemple 1 :

```
<body>
  <h2>JavaScript Validation</h2>
  <form name="myForm" action="/action.php" onsubmit="validateForm();" "
method="post">
    <label for="">Entrez votre Nom :</label>
    <input type="text" name="fname">
    <input type="submit" value="Submit">
  </form>
  <script>
    function validateForm() {
      let x = document.forms["myForm"]["fname"].value;
      if (x == "") {
        alert("Name must be filled out");
        return false;
      }
    }
  </script>
</body>
```

[LINK](#)

JS : Validation du formulaire

Exemple 2 :

```
<body>
  <p>Please input a number between 1 and 10:</p>
  <input id="numb">
  <button type="button" onclick="myFunction()">Submit</button>
  <p id="demo"></p>
<script>
function myFunction() {
  // Obtenez la valeur du champ de saisie avec id="numb".
  let x = document.getElementById("numb").value;
  // Si x n'est pas un nombre ou est inférieur à un ou supérieur à 10
  let text;
  if (isNaN(x) || x < 1 || x > 10) {
    text = "Input not valid";
  } else {
    text = "Input OK";
  }
  document.getElementById("demo").innerHTML = text;
}
</script>
```

[LINK](#)

Exercice : Validation du formulaire

Soit le formulaire suivant :

Saisir votre nom	
Saisir votre email	
Saisir votre age	
Valider	Initialiser

A. Ecrire le script pour les fonctions suivantes :

- Une fonction **verifNom()** pour vérifier le pseudo (on vérifie juste la longueur : entre 2 et 25 caractères),
- Une fonction **verifEMail()** pour l'adresse email (on vérifie si l'adresse contient le caractère '@')
- Une fonction **verifAge()** pour l'âge (on va dire qu'il doit être compris entre 5 et 100 ans) ;
- Une fonction **Valider()** (à partir des précédentes) qui vérifie tout et retourne **vrai** si tout va bien et **false** sinon.

Remarque : la fonction **Valider()** est appelée lors de l'événement **onSubmit** du bouton **submit** du formulaire. Si elle retourne **true**, le formulaire est alors envoyé à l'adresse de l'action. Si elle retourne **false**, le formulaire n'est pas envoyé !!

Solution : Validation du formulaire (HTML)

HTML File

```
<html>
<head> <script src="code.js"></script>
</head>
<body>
  <form name="f">
    <table>
      <tr><td><label for="nom" > Saisir votre nom</label></td>
        <td><input type="text" name="txtnom" id="nom" /></td></tr>
      <tr><td><label for="prenom" > Saisir votre prenom</label></td>
        <td><input type="text" name="txtemail" id="email" /></td></tr>
      <tr><td><label for="age" > Saisir votre age</label></td>
        <td><input type="text" name="txtage" id="age" /></td></tr>
      <tr><td><input type="submit" id="btnsubmit" value="Valider"
onclick="valider()" /></td>
        <td><input type="reset" id="btnreset"
value="Initialiser" /></td></tr>
    </table>
  </form>
</body>
</html>
```

Solution : Validation du formulaire (JavaScript)

JS File

```
function verifNom(){
    let txtnom = document.getElementById('nom').value;
    if ((txtnom.length<2)|| (txtnom.length>25)) {
        alert("Le nom doit être de type texte et avoir plus de 2
caractères et moins de 25 caractères");
        return false;
    }
    return true;
}

function verifEmail(){
    let text = /^[a-zA-Z0-9+_.-]+@[a-zA-Z0-9.-]+$ /;
    let txtemail = document.getElementById('email').value;
    if(txtemail.match(text))
        return true;
    else{
        alert("Entrez un Email Valide")
        return false;
    }
}
```

Solution : Validation du formulaire (JavaScript)

JS File

```
function verifAge() {
    var txtage = document.getElementById('age').value;
    var age = parseInt(txtage);

    if ((age < 5) || (age > 100) || isNaN(age)) {
        return false;
    }
    else
        return true;
}

function valider() {
    if (verifNom() & verifEmail() & verifAge()) return true;
    else {
        alert("formulaire non bien renseigné");
        return false;
    }
}
```

Exercice(suite) : Validation du formulaire

Soit le formulaire suivant :

Saisir votre nom	
Saisir votre email	
Saisir votre age	
Valider	Initialiser

B. Modifier les scripts pour :

Chaque champ non valide doit être coloré une fois qu'il perd le focus (l'utilisateur change de champ) ceci est possible grâce à la méthode **onBlur**.

Pour changer la couleur d'un champ, il faut modifier la couleur de l'arrière plan grâce à :
element.style.backgroundColor = "nouvelle couleur";

Pour enlever la coloration d'un champ , il suffit d'affecter aucune valeur à
element.style.backgroundColor.

Solution : Validation du formulaire (HTML)

HTML File

```

<html>
<head> <script src="code.js"></script>
</head>
<body>
  <form name="f">
    <table>
      <tr><td><label for="nom" > Saisir votre nom</label></td>
        <td><input type="text" name="txtnom" id="nom"
onblur="changeColor_nom()" /></td></tr>
      <tr><td><label for="prenom" > Saisir votre prenom</label></td>
        <td><input type="text" name="txtemail" id="email"
onblur="changeColor_email()" /></td></tr>
      <tr><td><label for="age" > Saisir votre age</label></td>
        <td><input type="text" name="txtage" id="age"
onblur="changeColor_age()" /></td></tr>
      <tr><td><input type="submit" id="btnsubmit" value="Valider"
onclick="valider()" /></td>
        <td><input type="reset" id="btnreset" value="Initialiser" />
        </td></tr>
    </table>
  </form>

```

Solution : Validation du formulaire (JavaScript)

JS File

```
function changeColor_nom(){
    var txtnom = document.getElementById('nom');
    if(txtnom.value.length < 2 || txtnom.value.length > 25){
        txtnom.style.backgroundColor="yellow";
        return false;
    }else{
        txtnom.style.backgroundColor="";
        return true;
    }
}

let text = /^[a-zA-Z0-9+_.-]+@[a-zA-Z0-9.-]+$/;
function changeColor_email(){
    var txtemail = document.getElementById('email');
    if(txtemail.value.match(text)){
        txtemail.style.backgroundColor="";
        return true;
    }else{
        txtemail.style.backgroundColor="yellow";
        return false;
    }
}
```


Solution : Validation du formulaire (JavaScript)

JS File

```
function changeColor_age(){
    var txtage = document.getElementById('age');
    var age = parseInt(txtage.value);
    if ((age<5) || (age>100) || isNaN(age)) {
        txtage.style.backgroundColor="yellow";
        return false;
    }else
        txtage.style.backgroundColor="";
        return true;
    }
function valider(){
    if ( changeColor_nom() & changeColor_email() &
changeColor_age() ) return true;
    else{
        alert("formulaire non bien renseigné");
        return false;
    }
}
```

JS: Les événements

Propriétés de l'API de validation des contraintes en HTML5

Propriétés	Description
validationMessage	Un message (dans la langue locale) décrivant les contraintes de validation que le contrôle ne satisfait pas (si c'est le cas), ou une chaîne vide si le contrôle n'est pas soumis à validation (willValidate est alors false), ou bien la valeur de l'élément satisfait ses contraintes.
validity	Un objet <u>ValidityState</u> qui décrit l'état de validité de l'élément.
validity.customError	Renvoie true si l'élément à une erreur personnalisée, false a contrario.
validity.patternMismatch	Renvoie true si la valeur de l'élément ne correspond pas au motif fourni, false dans le cas contraire. Si la méthode renvoie true, l'élément fera partie de la pseudo-classe CSS <u>:invalid</u>
validity.rangeOverflow	Renvoie true si la valeur de l'élément est supérieure au maximum défini, false dans le cas contraire. Si le retour est true, l'élément fera partie des pseudo-classes CSS <u>:invalid</u> et <u>:out-of-range</u>
validity.rangeUnderflow	Renvoie true si la valeur de l'élément est plus petite que le minimum défini, false dans le cas contraire. Si le retour est true, l'élément fera partie des pseudo-classes CSS <u>:invalid</u> et <u>:out-of-range</u>

JS: Les événements

Propriétés	Description
validity.stepMismatch	Renvoie true si la valeur de l'élément ne correspond pas aux règles définies par l'attribut step, false a contrario. Si le retour est true, l'élément fera partie des pseudo-classes CSS :invalid et :out-of-range .
validity.toolong	Renvoie true si la taille de l'élément est supérieure à la longueur maximum définie, false dans le cas contraire. Si le retour est true, l'élément fera partie des pseudo-classes CSS :invalid et :out-of-range .
validity.typeMismatch	Renvoie true si la syntaxe de la valeur de l'élément n'est pas correcte ; false dans le cas contraire. Si le retour est true, l'élément sera de la pseudo-classe CSS :invalid .
validity.valid	Renvoie true si la valeur de l'élément n'a pas de problème de validité, sinon false. L'élément sera de la pseudo-classe CSS :valid si le retour est true ; de la pseudo-classe CSS :invalid si le retour est false.
validity.valueMissing	Renvoie true si l'élément n'a pas de valeur alors que le champ est requis, sinon false. L'élément sera de la pseudo-classe CSS :invalid si le retour est true.
validity.stepMismatch	Renvoie true si la valeur de l'élément ne correspond pas aux règles définies par l'attribut step, false a contrario. Si le retour est true, l'élément fera partie des pseudo-classes CSS :invalid et :out-of-range .

JS: Les événements

Propriétés	Description
willValidate	Retourne true si l'élément est validé lorsque le formulaire est soumis, false dans le cas contraire.
checkValidity()	Renvoie true si la valeur de l'élément n'a pas de problème de validation, false autrement. Si l'élément est invalide, cette méthode déclenche aussi un événement invalid sur cet élément.
setCustomValidity(message)	Ajoute un message d'erreur personnalisé à l'élément; si vous définissez un message d'erreur personnalisé, l'élément est considéré comme invalide, et le message spécifié est affiché. Cela vous permet d'utiliser du code JavaScript pour établir une erreur de validation autre que celles offertes par l'API standard des contraintes de validation. Le message est affiché à l'utilisateur lorsque le problème est rapporté. Si l'argument est une chaîne de caractères vide, l'erreur personnalisée est considérée comme effacée.

HTMLSelectElement.setCustomValidity()

La méthode **HTMLSelectElement.setCustomValidity()** définit :

- Le message de validation personnalisé de l'élément sélectionné avec le message renseigné.
- Utiliser une chaîne de caractère vide indique que l'élément n'a pas de message d'erreur de validation personnalisé.

Syntaxe

```
ElmSelectionne.setCustomValidity("message");
```

Exemple

JS: Les événements

Exemple 1 :

```
<body>
  <form action="#" method="GET">
    <!--Le nombre doit être entre 0 et 20 et ne doit pas être dans la liste noire-->
      <label for="number1">Nombre (0, 10, 12 dans la liste noire)</label>
      <input id="number1" type="number" onchange="checkNumberBlackList(this)"
min="0" max="20" />
    <!-- Téléphone doit respecter l'expression régulière (pattern regex)
numéro de téléphone commençant par 0 suivi de 9 chiffres -->
      <label for="phone">Téléphone</label>
      <input id="phone" type="tel" pattern="0[0-9]{9}" />
      <!-- Le nombre doit être entre 0 et 10 ou 90 et 100 -->
      <label for="number">Nombre (entre 0-10 ou 90-100)</label>
      <input id="number" type="number" onchange="checkNumber(this)" />
      <!-- La date de naissance doit avoir l'année égale à 1980 ou 1990 -->
      <label for="date">Date de naissance</label>
      <input id="date" type="date" min="1980-01-01" max="1990-12-31"
onchange="checkDate(this)" />
    <!-- Bouton pour envoyer le formulaire -->
      <input type="submit" />
  </form>
</body>
```

[Link](#)

JS: Les événements

Exemple 1 : <script>

```
const blacklistNumber = ["0", "10", "12"];

function checkNumberBlackList(element) {
    const number = element.value;
    /* Si le numéro est dans la liste */
    if (blacklistNumber.includes(number)) {
        element.setCustomValidity("Ce numéro est dans la liste
noire");
    } else {
        /* valeur valide - reset du message */
        element.setCustomValidity("");
    }
}
```

JS: Les événements

Exemple 1 : <script>

```
function checkNumber(element) {  
    const number = element.value;  
    /* Réglage beaucoup plus fin qu'avec les attributs max et  
min  
    * le nombre doit être entre 0 et 10 ou 90 et 100  
    */  
    if ((number >= 0 && number <= 10) || (number >= 90 &&  
number <= 100)) {  
        /* valeur valide - reset du message */  
        element.setCustomValidity("");  
    } else {  
        element.setCustomValidity("Le nombre doit être entre  
0 et 10 ou 90 et 100 (inclus)");  
    }  
}
```


JS: Les événements

Exemple 1 : <script>

```
function checkDate(element) {  
    /* Convertit la valeur en date et en année */  
    const year = new Date(element.value).getFullYear();  
  
    /* L'année doit être égale à 1980 ou 1990 */  
    if (year == 1980 || year == 1990) {  
        /* valeur valide - reset du message */  
        element.setCustomValidity("");  
    } else {  
        element.setCustomValidity("L'année doit être égale à  
1980 ou 1990");  
    }  
}
```

Les événements

Un **événement** est associé à une **action** de l'utilisateur :

- ☐ appui sur une touche,
- ☐ click ou déplacement de la souris,
- ☐ click sur un lien, un bouton,
- ☐ Chargement de la page web,
- ☐ Le survol d'un élément par la souris,
- ☐ ...

Javascript permet de gérer d'autres **événements** non gérables par l'HTML classique

Les événements

Exemples d'événements

- L'utilisateur clique sur un bouton, un lien ou tout autre élément : **Click**
- La page est chargée par le navigateur : **Load**
- L'utilisateur quitte la page : **Unload**
- L'utilisateur place le pointeur de la souris sur un élément : **MouseOver**
- Le pointeur de la souris quitte un lien ou tout autre élément : **MouseOut**
- Un élément de formulaire a le focus (devient la zone d'entrée active) : **Focus**
- Un élément de formulaire perd le focus : **Blur**

Les événements

Exemples d'événements

- La valeur d'un champ de formulaire est modifiée : **Change**
- L'utilisateur sélectionne un champ dans un élément de formulaire : **Select**
- L'utilisateur clique sur le bouton **submit** pour envoyer un formulaire : **Submit**
- L'utilisateur appuie sur une touche : **Keydown**
- Autres événements : **Abort, Error, Move, Resize, KeyPress, KeyUp, Db1Click, MouseDown, MouseUp, MouseMove, Reset.**

Les événements

Aujourd'hui, en JavaScript, il existe trois grandes façons d'implémenter un **gestionnaire d'évènements** :

- On peut utiliser des **attributs HTML** de type évènement;
- On peut utiliser des **propriétés JavaScript** liées aux évènements ;
- On peut utiliser la méthode **addEventListener()** (recommandé).

Les événements

❑ Utiliser les attributs HTML pour gérer un événement :

Principe : Associer à chaque événement une action : une fonction

➤ le gestionnaire d'événement (**EVENT HANDLER**).

Syntaxe du gestionnaire d'événement :

OnEvenement=fonction()

Exemple 1 :

```
<FORM>
```

```
<INPUT TYPE="button" VALUE="Cliquez ici"
```

```
onClick="alert('Vous avez bien cliqué ici !');">
```

```
...
```

```
</FORM>
```

Les événements

Exemple 2 :

```
<<body>  
  <h1>Titre principal</h1>  
  <p>Un premier paragraphe</p>  
  <button onclick="alert('Bouton cliqué')">Cliquez moi !</button>  
  <div onmouseover="this.style.backgroundColor='orange'"  
    onmouseout="this.style.backgroundColor='#EEE'">  
    <p>Un paragraphe dans un div</p>  
    <p>Un autre paragraphe dans le div</p>  
  </div>  
</body>
```

[LINK](#)

Les événements

- ❑ Utiliser les **propriétés** JavaScript pour gérer un évènement :

Exemple :

```
let b1 = document.getElementById('input');  
b1.onclick = function(){  
  alert('Vous avez bien cliqué ici !')  
};
```

```
<FORM>  
  <INPUT TYPE="button" VALUE="Cliquez ici" id="input">  
  ...  
</FORM>
```

Exemple

JS: Les événements

Exemple :

```
<body>
  <h1>Titre principal</h1>
  <p>Un premier paragraphe</p>
  <button>Cliquez moi !</button>
  <div>
    <p>Un paragraphe dans un div</p>
    <p>Un autre paragraphe dans le div</p>
  </div>
</body>
```

```
<script>
//On sélectionne le premier button et le premier div du document
let b1 = document.querySelector('button');
let d1 = document.querySelector('div');
//On utilise les propriétés gestionnaires d'évènement avec nos
éléments
b1.onclick      = function(){alert('Bouton cliqué')};
d1.onmouseover  = function(){this.style.backgroundColor = 'orange'};
d1.onmouseout   = function(){this.style.backgroundColor=' #eee'};
</script>
```

[LINK](#)

Les objets prédéfinis : **DOCUMENT**

- ❑ Utiliser la méthode **addEventListener()** pour gérer un évènement :

Exemple :

```
let b1 = document.getElementById('input');  
b1.addEventListener('click', function(){alert('Vous avez  
bien cliqué ici !')});
```

```
<FORM>  
  <INPUT TYPE="button" VALUE="Cliquez ici" id="input">  
  ...  
</FORM>
```

JS: Les événements

Exemple :

```
<body><h1>Titre principal</h1>
  <p>Un premier paragraphe</p>
  <button>Cliquez moi !</button>
  <div>
    <p>Un paragraphe dans un div</p>
    <p>Un autre paragraphe dans le div</p>
  </div></body>
```

```
<script>
//On sélectionne le premier button et le premier div du document
let b1 = document.querySelector('button');
let d1 = document.querySelector('div');
//On utilise la méthode addEventListener pour gérer des évènements
b1.addEventListener('click', function(){alert('Bouton cliqué')});
d1.addEventListener('mouseover',
function(){this.style.backgroundColor = 'orange'});
d1.addEventListener('mouseover', function(){this.style.fontWeight
= 'bold'});
d1.addEventListener('mouseout',
function(){this.style.backgroundColor='white'});
</script>
```

[LINK](#)

Les événements

Liste des événements:

- **Click** : se produit lorsque l'utilisateur clique sur un élément associé à l'événement.

```
<element onclick="myScript">
```

[link](#)

```
object.onclick = function(){myScript};
```

[link](#)

```
object.addEventListener("click", myScript);
```

[link](#)

- **MouseOver**: c'est lorsque l'utilisateur survole (met le curseur de la souris sur) l'élément associé à l'événement,

```
<element onmouseover="myScript">
```

[link](#)

```
object.onmouseover = function(){myScript};
```

```
object.addEventListener("mouseover", myScript);
```

[link](#)

- **MouseOut**: c'est lorsque le curseur de la souris quitte l'élément. (**onmouseout**).

Les événements

Liste des événements:

- **Load** : se produit lorsque le navigateur de l'utilisateur charge la page en cours.
- **Unload** : se produit lorsque le navigateur de l'utilisateur quitte la page en cours.
- **Focus**: se produit lorsque l'utilisateur donne le focus à un élément, c'est-à-dire que l'élément devient actif.

```
<element onfocus="myScript">
```

```
object.onfocus = function(){myScript}; link
```

```
object.addEventListener("focus", myScript);
```

- **Blur**: c'est lorsque l'élément perd le focus (il était actif et il ne l'est plus, c'est à ce moment que cet événement est déclenché).

```
<element onblur="myScript">
```

```
object.onblur = function(){myScript}; link
```

```
object.addEventListener("blur", myScript);
```

Les événements

Liste des événements:

- **Change** : se produit lorsque l'utilisateur modifie le contenu d'un champs donné (souvent une liste de sélection).

`<select onchange="myFunction()">` [Link](#)

`object.addEventListener("change", myScript);` [link](#)

- **Select** : se produit lorsque l'utilisateur sélectionne un texte (ou une partie d'un texte) dans un champs de type "text" ou "textarea".

`<input type="text" onselect="myFunction()">` [link](#)

`object.addEventListener("select", myScript);` [link](#)

Les événements

Liste des événements:

- **Submit (**onsubmit**)** : se produit lorsque l'utilisateur clique sur le bouton de la soumission d'un formulaire (bouton "submit").

```
<form onsubmit="myFunction()">
```

```
  Enter name: <input type="text">      link
```

```
  <input type="submit">
```

```
</form>
```

```
object.addEventListener("submit", myScript); link
```

Les événements

Liste des événements:

- **KeyDown** : se produit lorsque l'utilisateur enfonce une touche du clavier. [Link](#)
- **KeyUp** : c'est lorsque l'utilisateur relâche une touche du clavier. [Link](#)
- **DbClick** : se produit lorsque l'utilisateur clique deux (double clique) fois sur un élément du navigateur. [Link](#)
- **Resize** : quand l'utilisateur redimensionne le navigateur. [Link](#)

Les événements

Quelques évènements associés aux objets les plus courants:

- Lien hypertexte: **onClick**, **onMouseOver**, **onMouseOut**
- Page du navigateur: **onLoad**, **onUnload**
- Boutons: **onClick**
- Liste de selection: **onBlur**, **onFocus**, **onChange**
- Champs de texte: **onBlur**, **onFocus**, **onChange**, **onSelect**, **onKeyUp**, **onKeyDown**
- Images: **onMoseOver**, **onMouseOut**.

JS: Les événements

Exemple :

```
<style>
```

```
  div{
```

```
    border: 1px solid red;
```

```
    margin:1em;
```

```
    padding:1em;
```

```
  }
```

```
  #texte{
```

```
    display:none; /* permet de ne pas afficher l'élément */
```

```
  }
```

```
</style>
```

```
<body>
```

```
<div>
```

```
<h2 id="titre">Le théorème de Pythagore</h2>
```

```
<p id="texte">
```

Pour tout triangle ABC, si ABC est rectangle en A alors
 $AB^2 + AC^2 = BC^2$.

```
</p>
```

```
</div>
```

```
</body>
```

[Link](#)

JS: Les événements

Exemple :

```
<script>
```

```
let texte = document.getElementById("texte");
```

```
let titre = document.getElementById("titre");
```

```
titre.addEventListener(
```

```
  'click',
```



```
  ()=>{
```

```
    texte.style.display = "block";
```

```
  }
```

```
);
```

```
</script>
```

Essayez de changer l'évènement '**click**' par d'autres évènements ('**mouseover**', '**dblclick**', ...)

[Link](#)

jQuery "Faire plus en écrivant moins"

- ❑ jQuery est une « librairie » ou une « bibliothèque » **JavaScript**,
- ❑ Permet de simplifier l'utilisation **JavaScript** en fournissant un ensemble de codes déjà prêts à l'emploi,
- ❑ Développé par **John Resig** : développeur d'outils JavaScript pour Mozilla Co,
- ❑ **jQuery** ne doit pas être vu comme une alternative au JavaScript mais plutôt comme un ajout, on peut utiliser les deux dans un même script,
- ❑ Pour tirer avantage de **jQuery**, il est fortement recommandé d'avoir déjà une bonne connaissance et une bonne maîtrise du langage JavaScript et de son fonctionnement général.

Ajouter jQuery

jQuery est disponible dans deux formats différents : le format de développement, et celui de production.

☐ Le Format de développement : "jquery-3.6.0.js"

- Plus gros,
- Annoté de manière à faciliter la compréhension du fonctionnement du code,
- Des commentaires dans le code.

☐ Le Format de production : "jquery-3.6.0.min.js"

- Plus petit,
- Plus rapide à utiliser,
- Le contenu est compressé,
- Comporte tout son code sur une seule ligne pour le rendre plus petit.

☐ Téléchargement :

- <https://jquery.com/download/>

Ajouter le fichier "jQuery"

- Télécharger une copie et l'inclure dans les fichiers,
- Mettez le fichier sur le serveur qui héberge votre site,
- Ajouter le fichier via un **CDN** (Content Delivery Network) : Google, Microsoft, ou bien le **CDN** proposé par **jQuery**,
 - ✓ Recommandé !
 - ✓ Une meilleure performance,
 - ✓ La possibilité qu'un utilisateur ait déjà le fichier en cache.

Ajouter le fichier "jQuery"

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>Page Title</title>
```

```
  <script src="jquery-3.6.0.js"></script>
```

```
</head>
```

Sélectionner des éléments

- Sélecteurs CSS : permet de sélectionner des éléments afin de leur appliquer des règles de style, définir leur taille, leur couleur, etc.
 - ✓ `h1{ /* règles de styles */ }` //sélection par type d'élément
 - ✓ `.intro{ /* règles de styles */ }` //sélection par class
 - ✓ `#demo{ /* règles de styles */ }` //sélection par id
- Avec jQuery, on peut sélectionner des éléments de la même façon qu'en CSS en entrant le fameux **signe** de dollar de **jQuery** ! : `$()`
- `$()` : est un raccourci vers une fonction appelée `jQuery()`
 - ✓ `$('p')` : sélectionne tous les éléments **p**,
 - ✓ `jQuery('p')` : même chose, mais plus long.
- La méthode `$()` transforme les éléments du **DOM** en objet **jQuery**, sur lesquels vous pouvez appeler des méthode **jQuery**,

Sélection par relation

- **`$('ancêtre descendant')`** : sélectionne tous les éléments qui sont au sein d'un élément ancêtre,

jQuery :

```
$("ol li")
```

HTML :

```
<ol>
  <li>Premier élément</li> <!-- sélectionné -->
  <li>Second élément</li> <!-- sélectionné -->
  <ul>
    <li>Élément sans ordre</li> <!-- sélectionné -->
  </ul>
</ol>
```

Sélection par relation

- **`$('parent > enfant')`** : sélectionne tous les enfants qui sont descendants immédiats d'un élément parent,

jQuery :

```
$("ol > li")
```

HTML :

```
<ol>
  <li>Premier élément</li> <!-- sélectionné -->
  <li>Second élément</li> <!-- sélectionné -->
  <ul>
    <li>Élément sans ordre</li>
  </ul>
</ol>
```

Sélection par relation

- `$('élément ~ frère')` : sélectionne tous les frères du premier élément spécifié,

jQuery :

```
$("#li#premier ~ li")
```

HTML :

```
<ul>  
  <li id="premier">Un</li>  
  <li id="deuxième">Deux</li>    <!-- sélectionné -->  
  <li id="troisième">Trois</li> <!-- sélectionné -->  
</ul>
```

Sélection par relation

- `$('élément + frère')` : sélectionne l'élément frère qui suit immédiatement l'élément spécifié,

jQuery :

```
$("#li#premier + li")
```

HTML :

```
<ul>
  <li id="premier">Un</li>
  <li id="deuxième">Deux</li>    <!-- sélectionné -->
  <li id="troisième">Trois</li>
</ul>
```

Sélection par filtres

:first : sélectionne la première occurrence d'un élément,

jQuery :

```
$("p:first")
```

HTML :

```
<p>Premier texte</p> <!-- sélectionné -->  
<p>Deuxième texte</p>  
<p>Troisième texte</p>
```

Sélection par filtres

:last : sélectionne la dernière occurrence d'un élément,

jQuery :

```
$("#p:last")
```

HTML :

```
<p>Premier texte</p>  
<p>Deuxième texte</p>  
<p>Troisième texte</p> <!-- sélectionné -->
```

Sélection par filtres

:eq(indice) : sélectionne l'élément à un indice donné (c'est là que les tableaux deviennent bien pratiques !)

jQuery :

```
$("#li:eq(1)")
```

HTML :

```
<p>Premier texte</p>  
<p>Deuxième texte</p> <!-- sélectionné -->  
<p>Troisième texte</p>
```

Sélection par filtres

:gt(indice) : sélectionne le ou les élément(s) situé(s) à un indice **supérieur** au nombre spécifié,

jQuery :

```
$("#p:gt(0)")
```

HTML :

```
<p>Premier texte (donc indice 0)</p>  
<p>Deuxième texte (donc indice 1)</p> <!-- sélectionné -->  
<p>Troisième texte (donc indice 2)</p> <!-- sélectionné -->
```


Sélection par filtres

:lt(indice) : sélectionne le ou les élément(s) situé(s) à un indice **plus petit** au nombre spécifié,

jQuery :

```
$("#p:lt(2)")
```

HTML :

```
<p>Premier texte (donc indice 0)</p>    <!-- sélectionné -->  
<p>Deuxième texte (donc indice 1)</p>   <!-- sélectionné -->  
<p>Troisième texte (donc indice 2)</p>
```

Sélection par filtres

:not(sélecteur) : sélectionne les éléments qui ne sont pas... eh bien, le sélecteur,

jQuery :

```
$("#li:not('.legume')")
```

HTML :

```
<ul id="courses">
  <li class="legume">Aubergine</li>
  <li class="legume">Carotte</li>
  <li class="fruit">Pomme</li>    <!-- sélectionné -->
</ul>
```

Sélection par filtres

:hidden : sélectionne les éléments cachés (**display : none**),

:visible : sélectionne les éléments visibles,

:contains("texte") : éléments qui contiennent le texte spécifié,

:has("element") : éléments qui contiennent l'élément spécifié,

[attribut] : éléments qui ont l'attribut spécifié, par exemple **\$("[align]")**

[attribut = "valeur"] : éléments qui ont l'attribut et la valeur spécifiés, par exemple **\$("[align = center]")**,

[attribut != "valeur"] : Les éléments qui n'ont pas l'attribut et la valeur spécifiés, par exemple **\$("[align!=center]")**,

[Exemple](#)

Introduction JQuery

Exemple

```
<head>
  <script src="../../jquery-3.6.0.js"></script>
  <style>
    .souligne{text-decoration: underline;}
  </style>
</head>
<body>
  <h1>Cours jQuery</h1>
  <p id="p1">Un paragraphe <span class="souligne">écrit en
HTML</span></p>
  <span>Un élément span non contenu dans un élément p</span>
  <p id="p2">Un deuxième paragraphe</p>
  <p>Un <span>troisième</span> paragraphe</p>
</body>
```

Introduction JQuery

```
<script>
    $(document).ready(function(){

        $('p').has('span').css('color', 'orange');

        $('span').filter('.souligne').css("color", "green");

        $('span').not('.souligne').css("color", "blue");

        $('span').first().css("color", "orange")

        $('span').last().css("color", "blue")

        $('span').eq(1).css("color", "green");

    });
</script>
```

Sélection par filtres : formulaire

✓ Sélection des éléments du formulaire:

`$("[type=element]")` ou bien `$(":element")`

:input	sélectionne les éléments input,
:password	sélectionne les zones de mot de passe,
:text	sélectionne les zones de texte,
:checkbox	sélectionne les cases à cocher,
:radio	sélectionne les boutons d'option,
:submit	sélectionne les boutons de validation,
:checked	sélectionne tous les éléments cochés

Sélection par filtres : formulaire

:disabled	sélectionne tous les éléments non actifs
:selected	sélectionne tous les éléments sélectionnés
:enabled	sélectionne tous les éléments actifs
:image	sélectionne tous les éléments de type image
:focus	sélectionne tous les éléments qui ont le focus
:file	sélectionne tous les éléments de type file
:reset	sélectionne tous les boutons d'initialization

Autres

<https://api.jquery.com/category/selectors/basic-filter-selectors/>

<https://api.jquery.com/category/selectors/form-selectors/>

<https://api.jquery.com/category/selectors/hierarchy-selectors/>

KICH ISMAIL

Introduction JQuery

Example :

```
<head>
  <meta charset="UTF-8">
  <title>Document</title>
</head>

<body>
<!-- on inclut jquery --->
  <script src="../jquery-3.6.0.js"></script>
  <script src="jquery/testJQ.js"></script>
  <ol>
    <li id="elt1">Element 1.1</li>
    <li id="elt2"><span></span>Element 1.2</li>
    <li id="elt3" class="element3">Element 1.2</li>
  <ul>
    <li id="elt4">Element 2.1</li>
    <li id="elt5">Element 2.2</li>
  </ul>
</ol>
</body>
```

[Link](#)

Introduction JQuery

Exemple : Fichier "testJQ.js", Essayez tous les codes block par block

```
$(document).ready(function() {  
    $("li").css("color","red"); //Tous les éléments li,  
});
```

```
$(document).ready(function() {  
    $("ul li").css("color","red"); //les éléments li qui sont au sein  
de ul,  
});
```

```
$(document).ready(function() {  
    $("ol > li").css("color","blue"); //Tous les li descendants  
immédiats de ol,  
});
```

```
$(document).ready(function() {  
    $("li#elt1 ~ li").css("color","blue");//Tous les li frères du premier  
de li#elt1,  
});
```

Introduction JQuery

```
$(document).ready(function() {  
    $("li#elt1 + li").css("color","blue"); // li frère qui suit  
    immédiatement li#elt1,  
});
```

```
$(document).ready(function() {  
    $("li:first").css("color","blue"); //la première occurrence de li,  
});
```

```
$(document).ready(function() {  
    $("li:last").css("color","blue"); //la dernière occurrence de li,  
});
```

```
$(document).ready(function() {  
    $("li:eq(1)").css("color","blue"); //l'élément li d'indice 1,  
});
```

```
$(document).ready(function() {  
    $("li:not('#elt2')").css("color","magenta"); //l'élément autre  
    que li#elt2  
});
```

Introduction JQuery

```
$(function(){  
    $("li:contains(2.1)").css("background-color", "green"); //l'élément  
    contenant le texte "2.1"  
});
```

```
$(function(){  
    $("li").has("span").css("color", "magenta"); //l'élément contenant la balise  
    "span"  
});
```

```
$(function(){  
    $("[class]").css("background-color", "magenta"); //l'élément contenant  
    l'attribut "class"  
});
```

```
$(function(){  
    $("li[id='elt5']").css("background-color", "magenta"); //l'élément contenant  
    l'attribut "id=elt5"  
});
```

```
$(function(){  
    $("li[id!='elt3']").css("background-color", "magenta"); //tous les éléments  
    autres que l'élément contenant l'attribut "id=elt3"  
});
```

Méthodes jQuery

❑ Format d'une instruction jQuery

`$("élément").methode("paramètres");`

- Sélection d'un **élément** converti en un objet jQuery avec `$()`
- **Point**,
- **Nom de la méthode**,
- **Parenthèses**, vides ou contenant des **paramètres**,

❑ On peut classer les méthode JQuery suivant:

- ✓ Méthodes de manipulation du **contenu**
- ✓ Méthodes de parcours du **DOM**
- ✓ Méthodes de manipulation de la **taille** et de la **position**
- ✓ Méthodes des **effets**

Méthodes jQuery : Manipulation du contenu

Avec les méthodes jQuery de manipulation du contenu :

- ✓ Il est possible de changer le texte, les éléments et les attributs d'une page.
- ✓ Cela peut être utile lorsqu'on souhaite que le contenu change au clic sur un bouton, quand l'utilisateur effectue une action donnée, si quelque chose se produit sur la page, etc.

.css() : récupère ou définit les propriétés CSS d'un élément, même plusieurs propriétés à la fois.

`css("propertyname");` [link](#)

`css("propertyname", "value");` [link](#)

`css({"propertyname": "value", "propertyname": "value", ...});` [link](#)

Méthodes jQuery : Manipulation du contenu

❑ Parcourir un résultat JQuery : **each()**

- Il est possible de parcourir le resultat de JQuery retourné en utilisant la fonction **each()**
- La fonction **each()** prend en paramètres une fonction (callback) qui va être invoquée pour chaque élément dans le résultat de JQuery.

```
$("li").each(function() {  
    // Pour tous les <li>  
    $(this).css("color","red");  
});
```

- Le mot clé **this** référence l'élément en cours

```
1. Element 1.1  
2. Element 1.2  
3. Element 1.2  
   o Element 2.1  
   o Element 2.2
```

Méthodes jQuery : Manipulation du contenu

.html(paramètres) : remplace le contenu HTML de l'élément sélectionné, (sans "paramètres" retourne le contenu HTML du premier élément uniquement).

.text(paramètres) : remplace le texte de l'élément sélectionné, (sans "paramètres" retourne le contenu texte de tous les éléments descendants des éléments sélectionnés) [link](#)

.val() : Définit ou renvoie la valeur des champs de formulaire [link](#)

[Exemple](#)

Méthodes jQuery : Manipulation du contenu

Avec les méthodes jQuery de manipulation du contenu :

- ✓ On peut insérer et remplacer des éléments
- ✓ Plusieurs fonctions jQuery existent permettant d'insérer du contenu à l'intérieur, avant, après ou à la place d'un élément ou de plusieurs éléments HTML
- ✓ Ce contenu peut être du texte, de l'HTML, des objets jQuery, des éléments ou du texte
- ✓ Ces fonctions retournent l'objet jQuery sur lequel elles ont été invoquées,
- ✓ Elles peuvent recevoir en paramètre une fonction qui calcule le contenu à insérer ou qui va remplacer du contenu

.replaceWith() : remplace le(s) élément(s) sélectionné(s) entièrement, pas juste le texte ou le HTML à l'intérieur [link](#)

.remove() : supprime les éléments de l'élément sélectionné [link](#)

.before() : insère du contenu avant le(s) élément(s) sélectionné(s) [link](#)

.after() : insère du contenu après le(s) élément(s) sélectionné(s) [link](#)

Méthodes jQuery : Manipulation du contenu

- **prepend()** : insère du contenu à l'intérieur du/des élément(s) sélectionné(s) (après la balise HTML ouvrante) [link](#)
- **prependTo()** : insère des éléments HTML au début des éléments sélectionnés. [link](#)
- **append()** : insère du contenu à l'intérieur du/des élément(s) sélectionné(s) (avant la balise HTML fermante) [link](#)
- **appendTo()** : insère des éléments HTML à la fin des éléments sélectionnés. [link](#)
- **attr()** : définit un attribut et sa valeur ou simplement récupère sa valeur
 - `$(selector).attr(attribute)`
 - `$(selector).attr(attribute, value)` [link](#)
- **removeAttr()** : supprime un attribut
 - `$(selector).removeAttr(attribute)` [link](#)

Méthodes jQuery : Parcours du DOM

.removeClass() : supprime une classe du/des élément(s) sélectionné(s)

\$(selector).removeClass(classname) [link](#)

.addClass() : ajoute une nouvelle classe à/aux élément(s) sélectionné(s) (sans remplacer sa classe actuelle)

\$(selector).addClass(classname) [link](#)

[Exemple](#)

Méthodes jQuery : Parcours du DOM

.find() : trouve un/des élément(s) correspondant au paramètre dans la sélection actuelle

`$(selector).find(filter)` [link](#)

.parent() : accède au parent direct d'un/des élément(s) ou à ses parents si on utilise `.parents()`

`$(selector).parent(filter)` [Link](#)

.children() : accède aux enfants de(s) élément(s)

`$(selector).children(filter)` [link](#)

[Autres](#)

[Exemple 1](#)

[Exemple 2](#)

Méthodes jQuery : les effets

.show() : affiche le(s) élément(s) sélectionné(s),

\$(selector).show(speed, easing, callback) [link](#)

.hide() : masque le(s) élément(s) sélectionné(s),

\$(selector).hide(speed, easing, callback)

.toggle(): affiche ou masque le(s) élément(s) sélectionné(s) en fonction de son/leur état actuel.

\$(selector).toggle(speed, easing, callback) [link](#)

.fadeIn() : fait apparaître en fondu le(s) élément(s) sélectionné(s).

\$(selector).fadeIn(speed, easing, callback)

.fadeOut() : fait disparaître en fondu le(s) élément(s) sélectionné(s).

\$(selector).fadeOut(speed, easing, callback) [link](#)

Méthodes jQuery : les effets

.fadeTo() : amène progressivement le(s) élément(s) sélectionné(s) à une opacité donnée.

`$(selector).fadeTo(speed, opacity, easing, callback)` [link](#)

.slideUp() : masque le(s) élément(s) sélectionné(s) avec un défilement vers le haut.

`$(selector).slideUp(speed, easing, callback)`

.slideDown() : masque le(s) élément(s) sélectionné(s) avec un défilement vers le bas.

`$(selector).slideDown(speed, easing, callback)` [link](#)

.slideToggle() : affiche ou masque le(s) élément(s) sélectionné(s) avec un défilement en fonction de son/leur état actuel.

`$(selector).slideToggle(speed, easing, callback)` [link](#)

Méthodes jQuery : Taille et position

.height() : hauteur d'un cadre sans les marges intérieures, extérieurs ni les bords

\$(selector).height()

\$(selector).height(value) [link](#)

.width () : largeur d'un cadre sans les marges intérieures, extérieurs ni les bords

\$(selector).width()

\$(selector).width(value) [link](#)

.innerHeight() : hauteur en comptant les marges intérieures

.innerWidth() : largeur en comptant les marges intérieures

.outerHeight() : hauteur en comptant les marges intérieures et les bords

.outerWidth() : largeur en comptant les marges intérieures et les bords

Méthodes jQuery : gestion des événements "Souris"

- **click()** : permet de gérer les événements de type click,
- **dblclick()** : permettre de gérer un événement doubleclick, [link](#)
- **mouseenter()** : pour l'événement "la souris entre dans un élément"
- **mouseleave()** : pour l'événement "la souris sors d'un élément"
- **hover()** : un raccourci pour nous permettre de gérer les deux événements **mouseenter** et **mouseleave** d'un coup. [Link](#)
- **mousedown()** et **mouseup()** pour gérer les événements "le bouton de la souris est pressé sur un élément" et "le bouton de la souris relâché",
- **mousemove()** : permet d'obtenir les coordonnées de la souris en temps réel à partir d'un point de départ. [link](#)

Méthodes jQuery : gestion des événements "Clavier"

.keydown() et **.keyup()** : pour gérer les événements "un bouton du clavier est pressé/relâché par l'utilisateur". Notez que ces deux événements ne vont pouvoir être envoyés qu'à un élément qui possède le focus "les formulaire en générale".

[Link](#)

Méthodes jQuery : gestion des événements "formulaire"

- **focus()** : permet d'attacher un gestionnaire d'événement pour l'événement **focus**.
L'événement **focus** est envoyé à un élément lorsqu'il gagne le focus.
- **focusin()** : permet d'attacher un gestionnaire d'événement pour l'événement **focusin**. L'événement **focusin** est envoyé à un élément lorsque cet élément ou l'un de ses descendants gagne le **focus**.
- **blur()** / **focusout()** : permet d'attacher un gestionnaire d'événement pour l'événement **blur**. L'événement **blur** est envoyé à un élément lorsqu'il perd le **focus**.

[Link](#)

- **change()** : permet d'attacher un gestionnaire d'événement à un élément pour l'événement **change**. L'événement **change** se déclenche lorsque la valeur d'un élément **change**. Juste pour : **input**, **textarea** et **select**.
- **Submit()** : se déclenche dès qu'un utilisateur tente d'envoyer un formulaire

[Link](#)

Les événements en jQuery

Les méthodes vues précédemment sont appelées des méthodes de "type" événement, il s'agit en fait des notations raccourcies de la méthode **jQuery on()** qui va nous permettre de prendre en charge n'importe quel type d'événements.

.on() est la méthode clé pour travailler avec les événements en jQuery .

```
$('élément').on('événement', function(){  
    instructions ;  
})
```

- **événement** : on passe l'événement désiré en tant que paramètre à la méthode **.on()**
- **Function()** : code jQuery qui sera exécuté quand l'événement a lieu.

.on() crée ce qu'on appelle "un écouteur d'événement" ("**addEventListener**"), ce qui signifie que le code attend l'arrivée de l'événement.

Les événements en jQuery

voici une liste des événements courants que vous pouvez passer à la méthode `.on()`

- `.on('click', function() { ... })` [link](#)
- `.on('scroll', function() { ... })`
- `.on('hover', function() { ... })`
- `.on('mouseover', function() { ... })` [link](#)
- `.on('mouseenter', function() { ... })`
- `.on('mouseleave', function() { ... })`
- `.on('keydown', function() { ... })`
- `.on('keyup', function() { ... })`
- `.on('keypress', function() { ... })`
- `.on('focus', function() { ... })`
- `.on('blur', function() { ... })`
- `.on('resize', function() { ... })`

Méthodes JQuery

Exemple : plusieurs événements **on()** à la fois

```
<style> .jaune{background-color: yellow}</style>
<body>   <h1>Cours jQuery</h1>
         <div>
           <p>Un paragraphe dans un div</p>
           <p>Un autre paragraphe dans un div</p>
         </div>
         <p id="p3">Un paragraphe hors div</p> </body>

<script>
$(document).ready(function(){
//On enregistre un gestionnaire pour plusieurs événements pour un élément
$("div").on("mouseenter mouseleave", function(){
    $(this).toggleClass("jaune");
});
//On enregistre plusieurs gestionnaires pour plusieurs événements pour un élément
$("#p3").on({
    click: function(){$(this).text("Paragraphe cliqué");},
    mouseenter: function(){$(this).css("background-color", "lightBlue");},
    mouseleave: function(){$(this).css("background-color", "white");}
});
});
</script>
```

[Link](#)

Les événements en jQuery

```
$('#élément').on('Event1 Event2', function(){  
    instructions ;  
});
```

Ou

```
$('#élément').on({  
    Event1 : function(){ instructions ; },  
    Event2 : function(){ instructions ; },  
    Event3 : function(){ instructions ; },  
});
```

[Link](#)

Les Objets JavaScript (Rappel)

- Les variables sont des conteneurs pour les valeurs de données.
- Les **objets** sont aussi des variables. Mais les objets peuvent **contenir plusieurs valeurs** en utilisant une la notation **clé : valeur**.

```
let car = { type:"Fiat", model:"500", color:"white" };
```

Définition de l'objet

```
let person = {firstName:"John", lastName:"Doe", age:50,  
              eyeColor:"blue"};
```

```
let person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50,  
  eyeColor: "blue"  
};
```

Accès aux propriétés des objets

```
alert(person.firstName); // John  
  
alert(person["firstName"]); // John
```

Les Objets JavaScript (Rappel)

Voici un exemple d'un objet JavaScript :

```
let Etud = {numero :345, nom:"Pierre", age:25, moyenne:14.6 };
```

On peut aussi, ajouter un tableau membre, **objet** membre et une **fonction** membre :

```
let Etud = {  
  numero : 345,  
  nom : "Pierre",  
  age : 25,  
  note : [12.5, 16.8, 14.5],  
  moyenne : 14.6,  
  adresse : {numRue : 1260, nomRue : "Av. Hassan II"},  
  ajouterUnBonus: function(b) {  
    this.moyenne += b;  
  }  
};
```

Qu'est-ce que JSON ?

- ❑ **Format de fichier** permettant de stocker les données de manière organisée et lisible
- ❑ **Format léger** pour le l'échange des données entre le serveur et le client, facile à lire ou à écrire par les humains et pour les langage de programmation,
- ❑ **JSON** est l'abréviation de **JavaScript Object Notation**,
- ❑ **JSON** est dérivé de JavaScript,
- ❑ Les données (**objets**) sont présentées dans un format textuel constitué de paires "**clé : valeur**"
- ❑ Les objets contiennent des **propriétés** et leurs **valeur**,
- ❑ Une alternative au **XML**,
- ❑ Extension : **.json**

Pourquoi JSON ?

- ❑ Facile à lire et à utiliser : simplicité de mise en œuvre et grande vitesse de traitement,
- ❑ Utilisé par la plupart des langages de programmation et leurs Framework,
- ❑ Popularisé par le développement d'Ajax,
- ❑ Vous pouvez convertir un objet **JSON** en **Objet JS**,
- ❑ Vous pouvez convertir un **Objet JS** en **Objet JSON**,
- ❑ **API** Support [Facebook, Twitter, GitHub].

JSON vs XML

❑ JSON

- Format texte,
- Poids léger,
- N'utilise pas des balises,
- Plus court,
- Peut utiliser des tableaux,
- Ne soutien pas les Commentaires.

❑ XML

- Langage de balisage,
- plus lourd,
- Utilise des balises,
- Long,
- N'utilise pas les tableaux,
- Supporte les commentaires.

Introduction JSON

Exemple d'un fichier JSON

```
{
  "currentVersion":10.3,
  "folders":[
    "Canvas",
    "Demographics",
    "Elevation",
    "Ocean"
  ],
  "services":[
    {
      "name":"NatGeo_World_Map",
      "type":"MapServer"
    },
    {
      "name":"USA_Topo_Maps",
      "type":"MapServer"
    }
  ]
}
```

KICHEL ISMAIL

- ✓ Les **clés** avant les **:** ce sont toujours des chaînes de caractères;
- ✓ Les **valeurs** après les **:** elles peuvent être de plusieurs types,
 - une chaîne de caractères,
 - un nombre,
 - un tableau,
 - un objet,
 - Un booléen,
 - une valeur null.

Objet JSON

Un fichier **JSON** est composé aussi d'un ensemble de paires **clé: valeur** organisées de façon hiérarchique :

- Exemple d'une chaîne de caractères **JSON** décrivant un **objet** "compte bancaire" :

```
{ "numero": "B22", "titulaire": "Annie", "solde": 350 }
```

```
{  
  "numero"      : "B22",  
  "titulaire"   : "Annie",  
  "solde"       : 350  
}
```

Objet JSON

- Et voici une chaîne **JSON** décrivant un **tableau d'objets** "cours" :

```
[{  
  "num": "420-343-R0",  
  "nom": "Mise en oeuvre",  
  "duree": 45  
}, {  
  "num": "420-275-R0",  
  "nom": "BD 1",  
  "duree": 75  
}, {  
  "num": "420-035-R0",  
  "nom": "Syst. 1",  
  "duree": 75  
}, {  
  "num": "9956",  
  "nom": "Test des espaces",  
  "duree": 44  
}]
```

JSON et JavaScript

- **JSON** n'est qu'un objet JavaScript : **C'est faux**,
- **JSON** est un format de texte **indépendant** de tout langage : un **format** d'échange de données très populaire,
- De nombreux langages proposent aujourd'hui des outils pour faire la **passerelle** entre le langage en question et **JSON**.
- En **JavaScript** aussi, l'objet JavaScript global **JSON** possède deux méthodes pour interpréter du **JSON** et convertir des valeurs en **JSON** :
 - ✓ **parse()**: analyse une chaîne de caractères JSON et construit la valeur JavaScript ou l'objet décrit par cette chaîne
 - ✓ **stringify()**: convertit une valeur JavaScript en chaîne JSON

JSON to JavaScript : parse()

```
<script>
```

```
//convertir un objet JSON (string) en objet JS
```

```
let myJsonObject = '{"numero":"B22", "titulaire":"Annie",  
"solde":350}'; //chaîne de caractères JSON
```

```
let myJsObject = JSON.parse(myJsonObject);
```

```
console.log(myJsonObject);
```

```
console.log(typeof(myJsonObject));
```

```
console.log(myJsObject);
```

```
console.log(typeof(myJsObject))
```

```
</script>
```

JSON to JavaScript : stringify()

```
<script>
    //convertir un objet JS en objet JSON (string)
let myJsObject = {numero:"B22", titulaire:"Annie",
solde:350};    // Objet JavaScript

    let myJsonObject = JSON.stringify(myJsObject);

    console.log(myJsObject);
    console.log(typeof(myJsObject));

    console.log(myJsonObject);
    console.log(typeof(myJsonObject));
</script>
```

[Link](#)[Link2](#)

Exemple : Manipulation du JSON avec JavaScript

```
<style>
  table, th, td{
    border: 1px solid;
    border-collapse: collapse;
    text-align: center;
  }
  th{width: 100px;}
  tr:nth-child(even){background-color:darkgray;}
</style>
<body>
  <table id="table1">
    <tr><th>Numero de compte</th> <th>Titulaire</th>
      <th>Solde</th></tr>
  </table>
</body>
```

Exemple : Manipulation du JSON avec JavaScript

```
<script>
  let myJsonObject =
    '[{"numero":"B22","titulaire":"Mohamed","solde":350},
    {"numero":"B32","titulaire":"Jhon","solde":-
    15}, {"numero":"A332","titulaire":"Ali","solde":1500}, {"numero":"C14
    5","titulaire":"Ahmed","solde":300}, {"numero":"Z198","titulaire":"A
    nnie","solde":179}]';
    // file JSON (Table of object)
    let myJsObject = JSON.parse(myJsonObject);
    //Afficher les données JSON sur un tableau
    str = '';
    for(let i=0; i<myJsObject.length; i++){
      str += '<tr><td>' + myJsObject[i].numero+ '</td><td>' +
      myJsObject[i].titulaire + '</td><td>' + myJsObject[i].solde +
      '</td></tr>';
    }
    document.getElementById('table1').innerHTML += str;
</script>
```

[Link2](#)

Exemple : Manipulation du JSON avec jQuery : la méthode `getJSON()`

- ❑ La méthode `getJSON()` est utilisée pour obtenir des données JSON à l'aide d'une requête AJAX HTTP GET. (Environnement Client-Serveur)

```
<head> <title>Charger un fichier JSON</title>
  <script src="../../jquery-3.6.0.js"></script> </head>
<body>
  <div id="show"></div>
  <script>
    $(function(){ // équivalent à $(document).ready(function()
      $.getJSON("Data-type.json", function(data){
        console.log(data);
        var content = "";
        content += data.username;
        content += " " + data.lastName;
        $("#show").html(content);});
    });
  </script>
</body>
```

JSON

```
{  
  "firstName" : "Ali",  
  "lastName" : "Ait Ahmed",  
  "location" : "Morocco",  
  "skills" : [  
    "HTML",  
    "JS",  
    "CSS"  
  ],  
  "adresses" : {  
    "Morocco" : "khémisset",  
    "USA" : "Dakota"  
  },  
  "Age" : 38,  
  "Worhing" : true,  
  "identity" : null  
}
```

File JSON : Data-type.json

AJAX

- ❑ **AJAX** : Asynchronous JavaScript and XML.
- ❑ **L'AJAX** n'est pas un langage de programmation,
- ❑ Ensemble de **techniques** utilisant des technologies diverses pour envoyer et récupérer des données vers et depuis un serveur,
- ❑ Echange **asynchrone**, c'est-à-dire sans avoir à recharger la page.

AJAX est le résultat d'un ensemble de technologies du web qui fonctionnent ensemble pour arriver à un objectif simple : **rafraîchir** une partie de page web **sans recharger** la page complète.

AJAX

Une technique qui fait usage des éléments suivants:

- HTML.
- CSS pour la présentation de la page.
- JavaScript pour les traitements locaux,
- DOM (Document Object Model) qui accède aux éléments de la page ou du formulaire
- ASP.NET, PHP ou un autre langage de scripts peut être utilisé coté serveur.

AJAX

"**Asynchronous**" - asynchrone - : l'exécution de JavaScript continue sans attendre la réponse du serveur qui sera traitée quand elle arrivera.

- ☐ En mode synchrone, le navigateur serait gelé en attendant la réponse du serveur.
- ☐ Cela permet de modifier le contenu d'une page selon les commandes de l'utilisateur, à partir de données préalablement fournies ou avec un texte tapé par l'utilisateur.

AJAX : Pourquoi ?

Ajax Modifier partiellement la page affichée par le navigateur pour la mettre à jour sans avoir à **recharger** la page entière.

- Par exemple le contenu d'un champ de formulaire peut être changé, sans avoir à recharger la page avec le titre, les images, le menu, etc.
- Permet ainsi d'effectuer des traitements sur le poste client (avec JavaScript) à partir d'informations prises sur le serveur. Cela répartit la charge de traitement. Auparavant, toutes les modifications de pages étaient faites sur le serveur

AJAX : Comment ?

AJAX permet alors d'appeler des scripts serveur et interagir avec le serveur directement depuis jQuery.

- Le script coté serveur (ASP ou PHP ,...) appelé fait son travail : envoi de mail, insertion en base de données... et surtout, il renvoie un résultat que jQuery va intercepter.
- Ce résultat, c'est ce que jQuery utilisera pour mettra à jour la page ! C'est comme ça que le fil de commentaire de Facebook fonctionne

AJAX : Comment ?

- Grace à **AJAX** toute la puissance des scripts serveurs est à notre disposition.
- On voit de plus en plus d'applications réalisées complètement grâce à **AJAX**, citons **Google Maps** pour l'exemple !
- **XML** se situe au niveau du retour que le script serveur fait et que jQuery intercepte.
- Si on souhaite ajouter un nouveau commentaire, mieux vaut que le retour soit un peu structuré pour le rendre plus facilement exploitable, XML est alors tout indiqué.
- On peut décider de renvoyer du **HTML** ou même du texte pur, ou même un autre format, le **JSON** qui est plus simple à traiter et plus léger.
- **Script Serveur** : le script serveur doit renvoyer quelque chose, et c'est ce quelque chose qu'on va devoir utiliser pour **rafraîchir la page**.

AJAX : Comment ?

Javascript ou JQuery

- La base de chaque appel **AJAX** est une requête **HTTP**.
- **JavaScript** est suffisamment puissant et évolué pour permettre d'envoyer des requêtes **HTTP**.
- **jQuery** ne fait que simplifier énormément l'appel et le suivi de cette requête **HTTP**, et, en prime, de faciliter énormément le traitement du retour que fait le script serveur.
- **jQuery** simplifie la vie, même à tel point que réaliser un appel **AJAX** grâce à **jQuery** est presque une promenade.

AJAX : Fonctionnement

- ❑ **AJAX** utilise un modèle de programmation comprenant d'une part **la présentation**, d'autre part **les évènements**.
- ❑ Les évènements sont les actions de l'utilisateur, qui provoquent l'appel des fonctions associées aux éléments de la page.
- ❑ L'interaction avec l'utilisateur se fait à partir des formulaires ou boutons html.
- ❑ Ces fonctions JavaScript identifient les éléments de la page grâce au **DOM** et communiquent avec le serveur par l'objet **XMLHttpRequest**.

AJAX : Fonctionnement

- ❑ L'objet **XMLHttpRequest** est un objet prédéfini,
 - ✓ disponible dans tous les navigateurs,
 - ✓ permet de faire des requêtes HTTP en JavaScript.
- ❑ Pour effectuer une requête asynchrone au serveur en utilisant l'objet **XMLHttpRequest**, nous allons toujours devoir suivre **4 étapes** :
 - ✓ On crée un objet **XMLHttpRequest** ;
 - ✓ On initialise notre requête, c'est à dire on choisit le mode d'envoi des données, l'URL à demander, etc. ;
 - ✓ On envoie la requête ;
 - ✓ On crée des gestionnaires d'événements pour prendre en charge la réponse du serveur.

AJAX : Fonctionnement

- ❑ Pour recueillir des informations sur le serveur cet objet dispose de deux méthodes:
 - ✓ **open** : établit une connexion.
 - ✓ **send** : envoie une requête au serveur.
- ❑ Les données fournies par le serveur seront récupérées dans les champs de l'objet **XMLHttpRequest**:
 - ✓ **responseXml** : pour un fichier **XML** ou,
 - ✓ **responseText** : pour un fichier de texte brut (**JSON**).

AJAX : Fonctionnement

- ❑ Création et Envoie d'une requête XMLHttpRequest

```
<script>  
    // On crée un objet XMLHttpRequest  
    let xhr = new XMLHttpRequest();  
  
    //On initialise notre requête avec open()  
    xhr.open("GET", 'url');  
  
    //On veut une réponse au format JSON  
    xhr.responseType = "json";  
  
    //On envoie la requête  
    xhr.send();  
</script>
```

AJAX : Fonctionnement

❑ Prendre en charge la réponse renvoyée par le serveur

Une fois notre requête envoyée, nous allons devoir réceptionner la réponse du serveur. Pour connaître l'état d'avancement de notre requête, nous avons deux options :

❖ Utiliser les gestionnaires d'événements **load**, **error** et **progress**

- ✓ L'événement **load** qui se déclenche lorsque la requête a bien été effectuée et que le résultat est prêt ;
- ✓ l'événement **error** qui se déclenche lorsque la requête n'a pas pu aboutir ;
- ✓ L'événement **progress** qui se déclenche à intervalles réguliers et nous permet de savoir où en est notre requête.

❖ Utiliser la propriété **readyState** et le gestionnaire **onreadystatechange**

- ✓ observer la valeur de la propriété **readyState** de l'objet **XMLHttpRequest** pour déterminer l'état d'avancement de la requête

Introduction AJAX

- ❑ Utiliser les gestionnaires d'événements **load**, **error** et **progress**

```
<script>
    // On crée un objet XMLHttpRequest
    let xhr = new XMLHttpRequest();

    //On initialise notre requête avec open()
    xhr.open("GET", 'Data-type.json');

    //On veut une réponse au format JSON
    xhr.responseType = "json";

    //On envoie la requête
    xhr.send();
```

[Exmeples](#)

Introduction AJAX

(suite)

```
//Dès que la réponse est reçue...
    xhr.onload = function(){
//Si le statut HTTP n'est pas 200...
    if (xhr.status != 200){
        //...On affiche le statut et le message correspondant
        alert("Erreur " + xhr.status + " : " + xhr.statusText);
//Si le statut HTTP est 200, on affiche le nombre d'octets
//téléchargés et la réponse
    }else{
        alert(JSON.stringify(xhr.response));
    }
};
```

Introduction AJAX

(suite)

```
//Si la requête n'a pas pu aboutir...
xhr.onerror = function(){
    alert("La requête a échoué");
};

//Pendant le téléchargement...
xhr.onprogress = function(event){
//lengthComputable = booléen; true si la requête a une length
calculable
    if (event.lengthComputable){
        //loaded = contient le nombre d'octets téléchargés
        //total = contient le nombre total d'octets à télécharger
        alert(event.loaded + " octets reçus sur un total de " +
event.total);
    }
};
</script>
```

Introduction AJAX

Les statuts code **HTTP** les plus fréquents sont les suivants :

- **100 Continue** : tout fonctionne jusqu'à présent; le client devrait continuer avec la requête ;
- **200 OK** : La requête a été complétée avec succès ;
- **301 Moved Permanently** : L'identifiant de ressource unique (URI) relatif à la ressource demandée a changé de localisation de façon permanente ;
- **302 Found** : L'identifiant de ressource unique (URI) relatif à la ressource demandée a changé de localisation de façon temporaire ;
- **304 Not Modified** : Indique au client que la réponse n'a pas été modifiée depuis le dernier accès et qu'il peut utiliser la version en cache ;
- **401 Unauthorized** : Indique que le client doit s'identifier s'il veut accéder à la réponse ;
- **403 Forbidden** : Indique que le client n'a pas l'autorisation d'accéder à ce contenu ;
- **404 Not Found** : Le serveur n'a pas pu trouver la ressource demandée ;
- **500 Internal Server Error** : Le serveur a rencontré une situation qu'il ne peut pas gérer.

Introduction AJAX

- ❖ Utiliser la propriété **readyState** et le gestionnaire **onreadystatechange**,
 - Il faut attendre la disponibilité des données, et l'état est donné par l'attribut **readyState** de **XMLHttpRequest**.
 - Les états de **readyState** sont les suivants (seul le dernier est vraiment utile):
 - ✓ **0** : non initialisé.
 - ✓ **1** : connexion établie.
 - ✓ **2** : requête reçue.
 - ✓ **3** : réponse en cours.
 - ✓ **4** : terminé.

Si on choisit d'utiliser **readyState** pour suivre l'avancement de notre requête, on va alors utiliser un gestionnaire d'événement **onreadystatechange** qui va être appelé dès que la valeur de **readyState** change.

Introduction AJAX

❖ Utiliser la propriété **readyState** et le gestionnaire **onreadystatechange**,

```
<body>
  <button onclick="afficher();">afficher comptes</button>
  <div id="mydiv"></div>
<script>
  var myRequest = new XMLHttpRequest();
  myRequest.onreadystatechange = function(){
    console.log(this.readyState);
    if(this.readyState == 4 && this.status == 200){
      //Convert ResponseText to JS Object
      var myJsonObject = JSON.parse(this.responseText);
      document.getElementById("mydiv").innerHTML =
myJsonObject.firstName;
    }
  }

  myRequest.open("GET", "Data-type.json");
  myRequest.send();
</script> </body>
```

[Exmeples](#)

Introduction AJAX

Les propriétés de l'objet **XMLHttpRequest**

- **readyState** = retourne un entier entre 0 et 4 qui correspond à l'état de la requête ;
- **onreadystatechange** = gestionnaire d'événements appelé lorsque la valeur de **readyState** change ;
- **responseType** = chaîne de caractères précisant le type de données contenues dans la réponse ;
- **response** = renvoie un objet **JavaScript**, **ArrayBuffer**, **Blob**, **Document** ou **DOMString**, selon la valeur de **responseType**, qui contient le corps de la réponse ;
- **responseText** = retourne un **DOMString** qui contient la réponse serveur sous forme de texte ou **null** si la requête a échoué ;
- **responseURL** = retourne une **URL** sérialisée de la réponse serveur ou la chaîne de caractères vide si l'URL est **null** ;
- **responseXML** = retourne un **Document** qui contient la réponse serveur ou **null** si la requête a échoué ;
- **status** = retourne de statut code **HTTP** de la réponse ;
- **statusText** = retourne un **DOMString** contenant la réponse complète **HTTP** (statut code + texte de la réponse) . . .

AJAX : objet XMLHttpRequest

- ❑ Aujourd'hui, le **XML** a été largement délaissé au profit du **JSON** qui est une notation qui permet d'échanger des données relativement simplement tandis que l'objet **XMLHttpRequest** est lentement en train de laisser sa place à la nouvelle **API Fetch**.

Ajax et jQuery

- ❑ **AJAX** fait appel à différentes technologies,
 - Son implémentation d'un navigateur à l'autre peut être différente et un même code peut donc provoquer des comportements différents.
 - **jQuery** fournit ici une solution censée efficace de sorte que nos requêtes s'exécutent de la même manière avec tous les navigateurs.
 - **jQuery** met notamment à notre disposition une méthode globale **\$.ajax()** et des méthodes comme **\$.get()**, **\$.getScript()**, **\$.getJSON()** ou **\$.post()** pour effectuer nos différentes opérations.
- ❑ Les deux méthodes les plus courantes pour envoyer une demande à un serveur sont **GET** et **POST**.
 - ✓ **GET** pour effectuer des opérations non destructives : récupérer des données du serveur sans les modifier,
 - ✓ **POST** pour effectuer des opérations destructives : modifier des données stockées sur le serveur.

Ajax et jQuery

❑ La méthode jQuery **`$.ajax()`**

➤ Les options les plus couramment utilisées sont les suivants :

- ✓ **url** : URL de la requête. Seule option strictement obligatoire ;
- ✓ **method** (valeur par défaut : GET) : (GET, POST ou plus rarement PUT, DELETE, etc.) ;
- ✓ **dataType** : Le type de données qu'on attend en réponse du serveur.
- ✓ **data** : Contient les données à envoyer au serveur. Si ces données ne sont pas au format chaîne de caractères, elles seront converties en chaîne ;
- ✓ **async** (valeur par défaut : true) : asynchrone (true) ou synchrone (false) ;
- ✓ **name** : Nom d'utilisateur à utiliser dans le cas où une demande d'identification HTTP est faite;
- ✓ **password** : Mot de passe à utiliser dans le cas où une demande d'identification HTTP est faite;
- ✓ ...

Introduction AJAX

Ajax et jQuery

- Effectuer une requête **GET**

```
<script>
$(document).ready(function(){
    $.ajax({
        //L'URL de la requête
        url: "URL",

        //La méthode d'envoi (type de requête)
        method: "GET",

        //Le format de réponse attendu
        dataType : "json",
    })
</script>
```

Ajax et jQuery

- Une fois la requête envoyée, on va vouloir récupérer la réponse du serveur et prendre en charge les différents cas de succès ou d'échec de cette requête.
- On utilise les événements **done()**, **fail()** et **always()**.
 - ✓ La méthode **done()** va généralement contenir une fonction de rappel qui sera exécutée si la requête aboutit. Cette fonction reçoit les données de réponse (converties en un objet JavaScript si le type de données est **JSON**),
 - ✓ La méthode **fail()** va également contenir une fonction de rappel qui sera exécutée si la requête n'aboutit pas.
 - ✓ Finalement, on passera une nouvelle fois une fonction de rappel à **always()** qui sera exécutée lorsque la requête est terminée, et ceci que ce soit un succès ou un échec.

Introduction AJAX

Exemple

```
<body>
  <h1>jQuery Ajax</h1>
  <div id="res"></div>
</body>
<script>
  $(document).ready(function(){
    $.ajax({
      url: "Data-type.json",
      method: "GET",
      dataType : "json",
    })
    //Ce code sera exécuté en cas de succès - La réponse du serveur est
    passée à done()
    /*On peut par exemple convertir cette réponse en chaine JSON et
    insère
    .done(function(response){
      let data = JSON.stringify(response);
      $("div#res").append(data);
    })
```

Introduction AJAX

Exemple (suite)

```
//Ce code sera exécuté en cas d'échec - L'erreur est passée à fail()  
//On peut afficher les informations relatives à la requête et à  
l'erreur  
    .fail(function(error){  
        alert("La requête s'est terminée en échec. Infos : " +  
JSON.stringify(error));  
    })  
  
//Ce code sera exécuté que la requête soit un succès ou un échec  
    .always(function(){  
        alert("Requête effectuée");  
    });  
});  
</script>
```

Exemple

Ajax et jQuery

Même si il reste généralement recommandé d'utiliser **\$.ajax()**, jQuery met à notre disposition les méthodes (moins puissantes suivantes) :

- ✓ **\$.get()** : permet d'effectuer une requête **GET** à **1'URL** fournie ;
- ✓ **\$.post()** : permet d'effectuer une requête **POST** à **1'URL** fournie ;
- ✓ **\$.getScript()** : permet d'ajouter un script à une page ;
- ✓ **\$.getJSON()** : permet d'effectuer une requête **GET** à **1'URL** fournie et demande à ce que du **JSON** soit renvoyé en réponse.

Introduction AJAX

Exemple

```
<body>
    <div id="res"></div>
</body>

</script>
// la méthode load()
$(document).ready(function(){
//Charge les éléments p depuis la page "une/url" et les insère
dans #res
    $("#res").load("Data-type.json");
// la méthode get()
$(document).ready(function(){
    $.get("Data-type.json", function(response){
        console.log(response); //affiche les éléments sur le console
    });
});
```

Exemple

Introduction AJAX

Exemple

```
// la méthode getJSON()

$(document).ready(function(){
    $.getJSON("Data-type.json", function(data){
        $.each( data, function( key, value ) {
            alert( key + ": " + value );
        });
    });
});
</script>
```

Exemple

Rappels : W3C

- **WORLD WIDE WEB Consortium**,
- Fondé en **1994**,
- Crée des **standards** pour le **Web**,
- Président: **Tim Berners-Lee**, inventeur du **Web**,

Objectifs :

- **Accroître** le potentiel du **Web**
- **Développer** des protocoles communs
- Assurer l'**inter-opérabilité** sur le **Web** entre les différents systèmes

XML

XML : e**X**tensible **M**arkup **L**anguage (langage extensible de balisage)

- **1998** : une spécification proposée par le **W3C**,
- Le langage **XML** est un format général de documents orienté texte,
- **Versions de la norme :**
 - **1.0** 4^{eme} du W3C du 16 août 2006 - recommandation du W3C
 - **1.0** 3^{eme} du W3C du 4 février 2004
 - **1.0** 2nd du W3C du 6 octobre 2000
 - **1.0** du 10 février **1998**

XML

XML comme le **HTML** sont des sous-ensemble de **SGML** (**S**tandard **G**eneralized **M**arkup **L**anguage), permettant de rendre **SGML** utilisable sur le Web.

- **SGML** : (développé dans les années 80)
 - ✓ **HTML**
 - ✓ **XML**
- **XML** est utilisé pour :
 - ✓ L'échange des données entre applications,
 - ✓ La création des documents pour le web, ...
 - ✓ Un site web qui va évoluer très souvent dans son fond et sa forme.

XML

XML est un **Méta-langage** (langage pour définir d'autres langages constitués d'un ensemble d'éléments appelés **balises**).

❑ 2 concepts fondamentaux

- Structure, contenu et présentation sont **séparés**
- Les balises ne sont pas **figées** : Possibilité de concevoir un ensemble de balises personnalisé pour un contexte donné.

Pourquoi XML ?

❑ Le **Web** est confronté à deux problèmes:

- **HTML** n'est pas extensible, il ne peut pas répondre aux besoins spécifiques de tous les domaines (mathématiques, chimie, musique, astronomie...) et ne définit plus le contenu du document ;
- **SGML** qui permettrait de définir de nouveaux langages de balisage spécifiques est **complexe**.

Pourquoi XML ?

- ❑ XML apporte une réponse à ces problèmes:
 - ✓ Richesse sémantique de **SGML** :
 - Compatible **SGML**
 - ✓ Facilité de mise en œuvre de **HTML**
 - **Simple** et **lisible**
 - **Portable** et **facilement** utilisable sur Internet
 - Assurant un **développement aisé**,
 - ✓ International (**ASCII** + **Unicode**)
 - **applicable** à un maximum de **domaines**
 - document **lisible** par un humain
 - documents **XML faciles** à **créer**, même à la main.

Pourquoi XML ?

Exemple : (HTML)

```
<html>
<body>
  <H1> Ma chanson </H1>
  <ul>
    <li> Auteur : Jean DeLacroix</li>
    <li> Producteur : Dupond</li>
    <li> Editeur : Maison Babel</li>
    <li> Duree : 6:20</li>
    <li> Date : 2009</li>
    <li> Artiste : Jean Holydays</li>
  </ul>
</body>
</html>
```


Pourquoi XML ?

Exemple : (XML)

<chanson>

<titre> Ma chanson </titre>

<auteur> Jean DeLacroix </auteur>

<producteur> Dupond</producteur>

<editeur> Maison Babel </editeur>

<duree> 6:20</duree>

<date> 2009</date>

<artiste> Jean Holydays </artiste>

</chanson>

**Balises
compréhensibles**

Pourquoi XML ?

XML c'est presque du **HTML**

Sauf que :

- On crée ces **propres balises**
- On est **contraint** par une **syntaxe** rigoureuse

Et que :

- Ne sait pas comment s'afficher dans un navigateur!!

```
<html>
<body>
<H1> Ma chanson </H1>
<ul>
<li> Auteur : Jean DeLacroix</li>
<li> Producteur : Dupond</li>
<li> Editeur : Maison Babel</li>
<li> Duree : 6:20</li>
<li> Date : 2009</li>
<li> Artiste : Jean Holydays</li>
</ul>
</body>
</html>
```

```
<chanson>
  <titre> Ma chanson </titre>
  <auteur> Jean DeLacroix </auteur>
  <producteur> Dupond</producteur>
  <editeur> Maison Babel </editeur>
  <duree> 6:20</duree>
  <date> 2009</date>
  <artiste> Jean Holydays </artiste>
</chanson>
```

Pourquoi XML ?

Contrairement à HTML, XML permet de structurer des données en utilisant ses propres balises, ce qui fait de lui un langage extensible et en employant un format texte comme support.

Par exemple l'information : L'étudiant **AIT ALI Moussa** habitant **Khémisset** et âgé de **21** ans peut être décrite sous forme d'un document XML comme suit:

<etudiant>

<nom>AIT ALI**</nom>**

<prenom>Moussa**</prenom>**

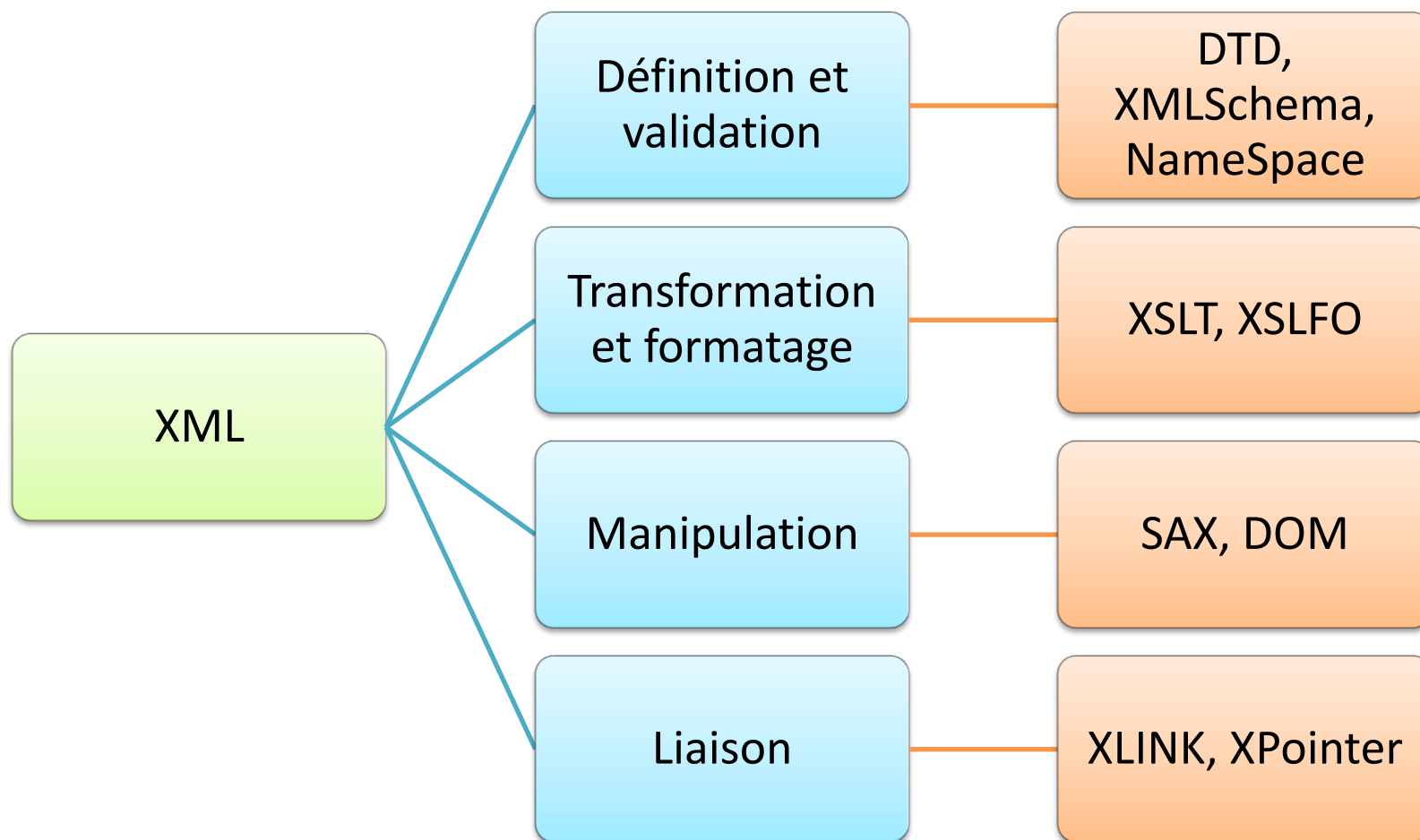
<adresse>Khémisset**</adresse>**

<age>21**</age>**

</etudiant>

XML Technologies liées

XML ne sait pas faire grand chose tout seul, à part servir pour les échanges :



XML : Technologies liées

XML sert de base pour tout une **pile de langages** touchants plusieurs domaines :

- ❑ **Présentation** : Le réseau est hétérogène, les données doivent être représentées indépendamment d'une machine donnée.
- ❑ **Echange** : Il faut pouvoir échanger facilement les données d'un format à un autre.
- ❑ **Description** : Les données doivent être décrites indépendamment de toute implantation.
- ❑ **Applications sectorielles.**

XML : Technologies liées

❑ Applications de présentation :

- ✓ **XHTML** : Extensible HyperText Markup Language basé sur **XML**
- ✓ **XSLT**: XML Stylesheet Language Transformation
- ✓ **XSLFO** : XML Stylesheet Language Formating (**PDF**)
- ✓ **SVG** : Scalable Vector Graphics, description des graphiques vectoriels et basé sur XML
- ✓ **SMIL** Synchronized Multimedia Integration Language (Intégration de contenus multimédias),
- ✓ ...

❑ Applications d'échange :

- ✓ **SOAP** : Simple Object Access Protocol protocole de transmission de messages entre objets distants indépendamment de toute implantation physique,
- ✓ **UDDI** Universal Description, Discovery and Integration : annuaire de services fondé sur XML destiné aux services Web,
- ✓ **XMI** : XML Metadata Interchange, pour la sérialisation et l'échange des modèles.

XML : Technologies liées

❑ Applications de description :

- ✓ **RSS** : Really Simple Syndication format d'échange de données entre les navigateurs.
- ✓ **WSDL** : Web Services Description Language grammaire XML permettant de décrire un service web
- ✓ **OWL** : Web Ontology Language.
- ✓ ...

❑ Applications sectorielles :

- ✓ **MathML** : Mathematical Markup Language. Langage pour l'affichage de symboles mathématiques, notamment sur Internet,
- ✓ **CML** : Chemical Markup Language Langage pour les données chimiques,
- ✓ ...

XML : Domaines d'applications

- ❑ **Interopérabilité de SI hétérogènes :**
 - ✓ Architectures existantes : complexité, dépendances,
 - ✓ Apports d'XML : simplicité, flexibilité, indépendance.
- ❑ **Commerce électronique :** Permet aux entreprises d'échanger facilement des informations,
- ❑ **Moteurs de recherches :** si je sais interpréter les données transmises, je peux les indexer efficacement,
- ❑ **Services en Ligne :** je peux envoyer mes données à un serveur pour leur appliquer un traitement donné (ex : publication),
- ❑ ...

Introduction

Exemple:
une bibliographie

```
<!-- le prologue -->
<?xml version="1.0" encoding="ISO-8859-1"?>
<!-- Élément racine -->
<biblio>
  <!-- Premier enfant -->
  <livre>
    <!-- Élément enfant titre -->
    <titre>Les Misérables</titre>
    <auteur>Victor Hugo</auteur>
    <nb_tomes>3</nb_tomes>
  </livre>
  <livre>
    <titre>L'Assomoir</titre>
    <auteur>Émile Zola</auteur>
  </livre>
  <livre lang="en">
    <titre>David Copperfield</titre>
    <auteur>Charles Dickens</auteur>
    <nb_tomes>3</nb_tomes>
  </livre>
</biblio>
```

Exercice:

Une bouteille d'eau Cristaline de 150 cl contient par litre 71 mg d'ions positifs calcium, et 5,5 mg d'ions positifs magnésium. On y trouve également des ions négatifs comme des chlorures à 20 mg par litre et des nitrates avec 1 mg par litre. Elle est recueillie à St-Cyr la Source, dans le département du Loiret. Son code barre est 3274080005003 et son pH est de 7,45. Comme la bouteille est sale, quelques autres matériaux comme du fer s'y trouvent en suspension.

Une seconde bouteille d'eau Cristaline a été, elle, recueillie à la source d'Aurèle dans les Alpes Maritimes. La concentration en ions calcium est de 98 mg/l, et en ions magnésium de 4 mg/l. Il y a 3,6 mg/l d'ions chlorure et 2 mg/l de nitrates, pour un pH de 7,4. Le code barre de cette bouteille de 50 cl est 3268840001008.

Introduction

<bouteilles>

<bouteille>

<marque>Cristaline</marque>

<composition>

<ion type="positif">calcium 71mg/l</ion>

<ion type="positif">magnésium 5,5mg/l</ion>

<ion type="negatif">chlorure 20mg/l</ion>

<ion type="negatif">nitrate 1mg/l</ion>

<autre type="metal">fer</autre>

</composition>

<source>

<ville>St-Cyr la Source</ville>

<departement>Loiret</departement>

</source>

<code_barre>3274080005003</code_barre>

<contenance unit="cl">150</contenance>

<ph>7,45</ph>

</bouteille>

...

</bouteilles>

Structure de base d'un fichier XML

- Un document XML correct doit être :
 - ✓ **Bien Formé** : respecte les règles d'écritures (**Syntaxiquement correcte**),
 - ✓ **Être valide** : respecte le modèle d'organisation (**Structurellement correcte**),
- Un document XML est un fichier texte dont l'extension est **".xml"**
- Un document XML est constitué de deux parties : le **prologue** et le **corps**,

`<?xml ... ?>`

`...`

`<root-element>`

`...`

`</root-element>`

Prologue (Entête) : ensemble d'information sur le document.

Corps : ensemble d'éléments avec leurs contenus.

Structure de base d'un fichier XML

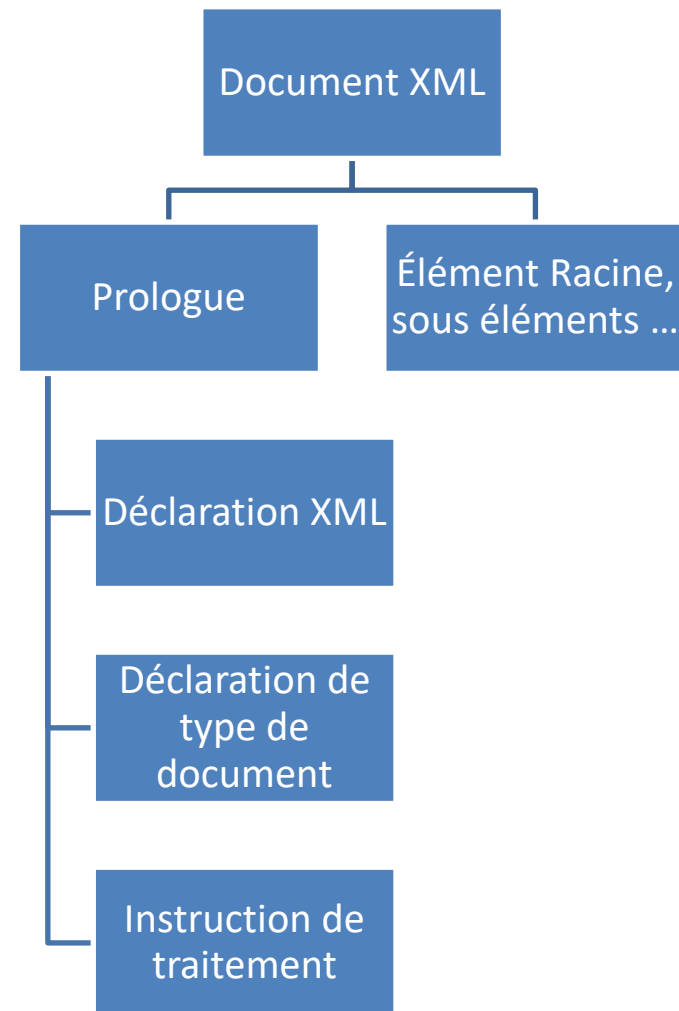
❑ Spécifications du langage XML

- ✓ Un document doit commencer par une **déclaration XML**,
- ✓ Toutes les balises avec un contenu doivent être **fermées**,
- ✓ Toutes les balises sans contenu doivent se terminer par les caractères **/>**
- ✓ Le document doit contenir un et un seul **élément racine**,
- ✓ Les balises ne doivent pas **se chevaucher**,
- ✓ Les valeurs d'attributs doivent être entre **guillemets**,
- ✓ La **casse** doit être **respectée** pour toutes les occurrences de noms de balise,
- ✓ Un document respectant ces critères est dit **"bien formé"**

Structure de base d'un fichier XML

- **Le prologue** n'est pas obligatoire mais lorsqu'il est présent, doit précéder le corps du document XML (la partie délimitée par l'élément racine).
- Il peut contenir la déclaration XML, la déclaration du type de document et des instructions de traitement.

Remarque : il ne faut placer ni commentaire, ni même de simple ligne vide avant le **prologue**.



Structure de base d'un fichier XML

■ Déclaration XML :

```
<?xml version="1.0" encoding="iso-8859-1" standalone="no"?>
```

Cette ligne permet d'indiquer 3 choses :

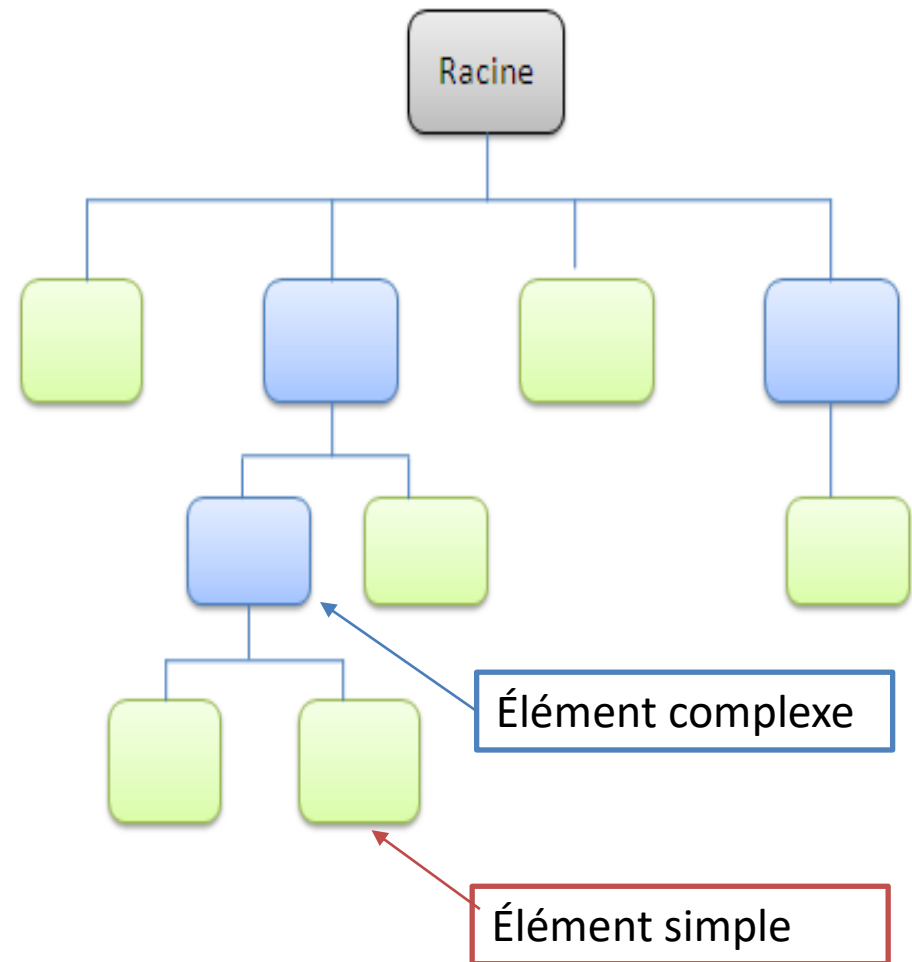
- ✓ **version** : la version de XML : soit **1.0** ou **1.1**
- ✓ **encoding** : Le jeu de caractères employé (UTF-8 est l'Encodage pris par défaut)
- ✓ **standalone**: spécifie que le document est autonome ou pas. Lorsque sa valeur est égale à "**yes**", cela signifie que tout ce qui permet de valider le document se trouve en lui même; "**no**" dans le cas contraire.

Remarques :

- ✓ Les attributs **encoding** et **standalone** sont facultatifs.
- ✓ Le prologue n'est pas obligatoire et, dans ce cas, le comportement par défaut (**version 1.0** et **encoding UTF-8**) est utilisé.

Structure de base d'un fichier XML

- Le reste du document (à partir de l'élément racine) contient l'instance de la base de données XML. C'est la partie la plus dynamique, car à tout moment on peut supprimer, mettre à jour ou ajouter un nouveau élément.
- La structure logique d'un document XML peut être représentée sous forme d'une arborescence hiérarchique et ordonnée.



L'arbre d'éléments

Éléments XML

❑ Élément racine :

- C'est la base du document XML,
- Il est unique et englobe tous les autres éléments,
- Il s'ouvre juste après le prologue, et se ferme à la fin du document.

❑ Les éléments :

- Forment la structure du document : ce sont les branches et les feuilles de l'arborescence,
- Ils peuvent contenir du texte, ou bien d'autres éléments, qui sont alors appelés « éléments enfants »,
- L'élément contenant d'autres éléments est appelé « élément parent »,
- D'autres éléments peuvent être vides : ils ne contiennent pas d'élément-enfant.

Introduction

```
<biblio>      <!-- Élément racine biblio -->
  <livre>      <!-- Élément parent livre, contient d'autres élément -->
    <titre>Les Misérables</titre>  <!-- Élément enfant titre -->
    <auteur>Victor Hugo</auteur>   <!-- Élément enfant auteur -->
    <nb_tomes>3</nb_tomes>         <!-- Élément enfant nb_tomes -->
  </livre>
  <livre>
    <titre>L'Assomoir</titre>
    <auteur>Émile Zola</auteur>
    <couverture couleur="rouge"/>  <!-- Élément vide -->
  </livre>
  <livre lang="en">
    <titre>David Copperfield</titre>
    <auteur>Charles Dickens</auteur>
    <nb_tomes>3</nb_tomes>
  </livre>
</biblio>
```

Exercice :

Donnez l'arbre du document XML ci-dessus :

```
<adresse>  
  <!--Carnet d'adresse -->  
  <personne id="personnel">  
    <type classe="famille"/>  
    <nom>AIT ALI </nom>  
    <prenom> Moussa </prenom>  
    <adr>n 7 Boulevard Hassan II</adr>  
    Anniversaire : le 6 août 1987  
  </personne>  
</adresse>
```

Éléments XML

- ❑ **Les attributs** : Tous les éléments peuvent contenir un ou plusieurs attributs.
 - Chaque élément ne peut contenir qu'une fois le même attribut.
 - Un attribut est composé d'un nom et d'une valeur.

Exemple d'utilisation d'un élément avec attribut :

```
<instrument type="vent">trompette</instrument>
```

Exemple d'utilisation d'un élément vide avec attributs :

```

```

Remarque : L'ordre d'utilisation des attributs au sein d'une balise n'a pas d'importance.

Eléments XML

- ❑ **Les entités** : Certains caractères ayant un sens précis en XML, il est nécessaire de leur trouver un remplaçant lorsque l'on a besoin de les insérer dans un document. On a recours dans ce cas à des entités prédéfinies.
- ❑ Si on en a besoin, on réalise alors un appel d'entités : **&nom_entité;**
- ❑ Ces entités sont :

Caractère	Entité
&	&
<	<
>	>
"	"
'	'
~	~

- ❑ On peut aussi utiliser le segment

<![CDATA[j'&cris_ce_que <je_veux>]]>

KICH ISMAIL

Document XML : Bien Formé

- ❑ **Les Règles de composition** : Un certain nombre de règles de base doivent être respectées :
- Un nom d'élément ne peut commencer par un chiffre.
 - Si le nom n'est composé que d'un seul caractère, ce doit être une lettre comprise entre « a » et « z » pour les minuscules, « A » et « Z » pour les majuscules.
 - S'il est composé d'au moins deux caractères, le premier peut être « _ » ou « : ». Le nom peut ensuite être composé de lettres, chiffres, tirets, tirets bas et deux points.
 - La syntaxe XML est sensible à la casse (le format distingue entre majuscules et minuscules).
 - Toutes les balises portant un contenu non vide doivent être fermées.
 - Les balises n'ayant pas de contenu doivent se terminer par />
 - Les valeurs d'attributs doivent être entre guillemets ;

Un document respectant ces critères est dit **bien formé** (**well formed**).

Introduction

Exercice : Pour chacun des fichiers suivants, déterminez s'il est bien formé, et corrigez les erreurs.

```
<?xml version="1.0" encoding="UTF-8"?>

<Contacts>
  <Person>
    <Firstname>John</Firstname>
    <Lastname>Smith</Lastname>
    <Position>CEO</Position>
  </Person>
  <Person>
    <Firstname>Tom</Firstname>
    <Lastname>Dunne</Lastname>
    <Company>Today FM</Company>
    <Position/>
    <Email>tom.dunne@todayfm.com</Email>
  </Person>
</Contacts>
```

✔ Document XML
bien formé

Introduction

```
<?xml version="1.0"? encoding="UTF-8">
<Contacts>
  <Person>
    <Firstname>John</Firstname>
    <Lastname>Smith</Lastname>
    <Birthday>1965/03/02</Birthday>
    <Company>IBM</Company>
    <Position>CEO</Position>
    <Email>jsmith@ibm.com</Email>
    <Email>jsmith@yahoo.com<Email/>
  </Person>
  <Person>
    <Firstname>Tom</Firstname>
    <Lastname>Dunne</Lastname>
    <Company>Today FM</Company>
    <Position/>
    <Email>tom.dunne@todayfm.com</Email>
  </Person>
</Contacts>
```

❌ **Document XML pas bien formé : balises xml et Email mal fermées**

Introduction

```
<?xml version="1.0" encoding="UTF-8"?>
<Contacts>
  <Person>
    <Firstname>John</Firstname>
    <Lastname>Smith</Lastname>
    <Birthday>19650302</Birthday>
    <Company>IBM</Company>
    <Position>CEO</Position>
    <Email>jsmith@ibm.com</Email>
    <Email>jsmith@yahoo.com</Email>
    <Address type="home">
      <Company>
        <Street>23 Main St</Company>
        </Street>
        <City>Dublin</City>
        <Postcode>4</Postcode>
        <Country/>
      </Address>
    </Contacts>
```

❌ **Document XML pas bien formé** : enlever la première ligne, balises **Company** et **Street** inversées.

[LINK](#)

Eléments XML

- ❑ Il est aussi possible de définir des règles plus strictes indiquant quelles sont les séquences et imbrications de balises ou les attributs autorisés.
- ✓ Cela se fait à l'aide d'une **DTD** ou d'un **Schéma**. On peut alors tester la validité des documents faisant référence à une **DTD** ou un **Schéma XML** pour s'assurer qu'ils respectent bien les règles qui y sont mentionnées.
- ✓ Un document **bien formé** dont la syntaxe est conforme aux règles stipulées dans une **DTD** ou un Schéma **XML** est dit valide.

Construction d'une DTD

- ❑ Les DTD fournissent les règles que le parseur (ou analyseur syntaxique = outil logiciel qui interprète le document) doit suivre pour la validation et l'interprétation correcte du document.
- ❑ Une DTD suit des règles strictes pour assurer que les résultats désirés soient obtenus et les données restituées correctement.
- ❑ Une DTD (ou un schéma XML) est requise pour qu'un document XML soit valide.

Déclarations de Type de Document : DTD

- ❑ Une **DTD** sert à structurer un document XML. On dit qu'elle valide un document XML.
- ❑ Dans un document XML valide apparait une déclaration du type de document DTD,
- ❑ Cette déclaration définit :
 - ✓ Le nom des éléments, leur contenu, le nombre de fois et l'ordre d'apparition,
 - ✓ Les attributs éventuels et leur valeurs par défaut,
 - ✓ Les noms des entités qui peuvent être utilisées.
- ❑ Les documents XML valides doivent respecter les règles données d'une DTD.

Déclarations de Type de Document : DTD

- ❑ Une **DTD** :
 - Est une forme de grammaire relativement ancienne,
 - La syntaxe d'une DTD est basée sur des instructions **<!INSTRUCTION...>** et pas sur des balise,
- ❑ Une **DTD** peut être stockée dans deux endroits différents.
 - Incorporée au document XML : **DTD interne**,
 - Dans un fichier à part : **DTD externe**.
 - Une **DTD** externe peut être partager entre plusieurs documents **XML**,
 - Il est possible aussi de mêler **DTD** interne et externe.
 - La DTD interne est lue avant la DTD externe, il ne doit pas y avoir de conflit.
- ❑ Il existe deux types de **DTD** externes : privé ou public
 - Les **DTD** privées sont accessibles uniquement **en local** (sur la machine de développement),
 - Les **DTD** publiques sont disponibles pour tout le monde, étant accessibles grâce à un **URI** (Uniform Resource Identifier).

Déclarations de Type de Document : DTD

- ❑ Une **DTD** permet essentiellement de spécifier :
 - ✓ Le nom des éléments, leur contenus et le nombre de fois et l'ordre d'apparition,
 - ✓ Les attributs éventuels et leurs valeurs par défaut,
 - ✓ Les noms des entités qui peuvent être utilisées.
 - ✓ des éléments qui composent le fichier XML visé.
- ❑ Une déclaration de type de document **DTD** est de la forme :

<!DOCTYPE elt.racine ... "...">
- ❑ Cette déclaration se place juste après le prologue du document **XML**. L'élément racine du document **XML** rattaché à cette **DTD** est alors obligatoirement **elt.racine**.

Déclarations de Type de Document : DTD

Exemple : `<?xml version="1.0" encoding="ISO-8859-1"?>`

`<!DOCTYPE biblio SYSTEM "bibliographie.dtd">`

`<biblio>`

`<item1>(...)</item1>`

`<item2>(...)</item2>`

`<item3>(...)</item3>`

`</biblio>`

Déclarations de Type de Document : DTD

❑ Syntaxe :

Le contenu ne change pas suivant le type de **DTD**, mais les déclarations d'une **DTD** interne sont écrites à l'intérieur du document XML alors que celles d'une **DTD** externe sont stockées dans un fichier externe.

- ✓ Exemple de déclarations pour une **DTD** interne :

```
<!DOCTYPE biblio[  
  <!ELEMENT biblio (livre)* >  
  <!ELEMENT livre (titre, auteur, nb_pages) >  
    <!ATTLIST livre  
      type (roman | nouvelles | théâtre) #IMPLIED  
      lang CDATA "fr">  
  <!ELEMENT titre (#PCDATA) >  
  <!ELEMENT auteur (#PCDATA) >  
  <!ELEMENT nb_pages (#PCDATA) >  
]>
```


Déclarations de Type de Document : DTD

- ❑ **DTD externe** : Les deux types de **DTD** externes sont les **DTD** de type **PUBLIC** (serveur distant) et les **DTD** de type **SYSTEM** (ordinateur local),
- ❑ Exemple de déclaration de **DTD** externe de type **SYSTEM** :

```
<!DOCTYPE biblio SYSTEM "bibliographie.dtd">
```

- ✓ Le fichier associé est le suivant :

```
<!ELEMENT biblio (livre)* >
```

```
<!ELEMENT livre (titre, auteur, nb_pages) >
```

```
<!ATTLIST livre
```

```
    type (roman | nouvelles) #IMPLIED lang CDATA "fr">
```

```
<!ELEMENT titre (#PCDATA) >
```

```
<!ELEMENT auteur (#PCDATA) >
```

```
<!ELEMENT nb_pages (#PCDATA) >
```

Déclarations de Type de Document : DTD

- ❑ Exemple de déclaration de **DTD** externe de type **PUBLIC** :

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"  
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

Dans cet exemple, la chaîne de caractères après le mot **PUBLIC** fait référence tout d'abord à l'identifiant de la **DTD** (ici -, signifie que la **DTD** n'a pas de numéro d'enregistrement officiel), au propriétaire de la **DTD** (ici le W3C), puis son nom, enfin sa langue.

Déclarations de Type de Document : DTD

❑ La **DTD** contient trois types d'instructions permettant de définir :

✓ Une ou plusieurs définitions d'éléments :

<!ELEMENT nom-élément valeur>

✓ Une ou plusieurs listes d'attributs:

<!ATTLIST nom-élément attribut type défaut>

✓ Une ou plusieurs définitions d'entité introduites par la balise **!ENTITY** :

<!ENTITY nom-entité "valeur">

Déclarations d'éléments

- Une déclaration d'élément est de la forme :

<!ELEMENT nom type_element >

- ✓ **nom** : nom de l'élément,
 - ✓ **type_element** : type auquel il est associé,
- Un élément peut être de type : **texte** | **vide** | **séquence** | **choix d'élément**,
 - Dans le cas de séquence ou choix : on indique la liste des éléments-enfants.

Déclarations d'éléments

- **Élément texte** : Cet élément est le plus répandu, puisque c'est celui qui contient... du texte. Il se déclare ainsi :

<!ELEMENT elt (#PCDATA) >

- **Élément vide** : Un élément vide est, comme son nom l'indique, un élément qui n'a aucun contenu.

Le mot-clef utilisé pour la déclaration de ce type d'élément est **EMPTY** :

<!ELEMENT elt EMPTY >

- ✓ Exemple d'utilisation :

<elt />

- ✓ Un élément vide peut cependant posséder un ou plusieurs attributs.

Par exemple :

Déclarations d'éléments

- **Élément aléatoire :**
 - Il peut contenir tout autre élément défini dans la **DTD**,
 - Il peut être vide ou contenir du texte,
 - Le nombre et l'ordre des éléments-enfants n'est pas important,
 - Il est préférable de ne pas utiliser trop souvent ce type de déclaration,
 - Le mot-clef utilisé pour la déclaration de ce type d'élément est **ANY**.

<!ELEMENT elt ANY >

Exemple de définition d'éléments

✓ Exemple de **DTD** :

```
<!ELEMENT personne (nom, prenom+, tel?, adresse) >
<!ELEMENT nom      (#PCDATA) >
<!ELEMENT prenom    (#PCDATA) >
<!ELEMENT tel       (#PCDATA) >
<!ELEMENT adresse   ANY       >
<!ELEMENT rue       (#PCDATA) >
<!ELEMENT ville     (#PCDATA) >
```

✓ Exemple de document XML:

```
<personne>
  <nom>Hugo</nom>
  <prenom>Victor</prenom>
  <prenom>Charles</prenom>
  <tel>01120243</tel>
  <adresse>
    <rue>victoire</rue> <ville>Paris</ville>
  </adresse>
</personne>
```

Déclarations d'éléments

■ Séquence d'éléments :

- une liste ordonnée des éléments qui doivent apparaître en tant qu'éléments-enfants de l'élément que l'on est en train de définir (**élément parent**),
- L'élément parent ne pourra contenir aucun autre élément que ceux figurant dans la séquence.
- Cette liste est composée d'éléments séparés par des virgules et est placée entre parenthèses.
- Chaque élément-enfant doit de plus être déclaré par ailleurs dans la **DTD** (avant ou après la définition de la liste, peu importe).
- Dans le fichier XML, ils doivent apparaître dans l'ordre de la séquence.

<!ELEMENT elt0 (elt1, elt2, elt3) >

KICH ISMAIL

Déclarations d'éléments

- Séquence d'éléments : `<!ELEMENT elt0 (elt1, elt2, elt3) >`

✓ Exemple d'utilisation valide :

```
<elt0>  
  <elt1>(…)</elt1>  
  <elt2>(…)</elt2>  
  <elt3>(…)</elt3>  
</elt0>
```

✗ Exemple d'utilisation non valide :

```
<elt0>  
  <elt1>(…)</elt1>  
  <elt3>(…)</elt3>  
</elt0>
```

... car l'élément **elt2** est manquant.

```
<elt0>  
  <elt1>(…)</elt1>  
  <elt3>(…)</elt3>  
  <elt2>(…)</elt2>  
</elt0>
```

... car l'ordre des éléments n'est pas respecté.

Déclarations d'éléments

- **Choix d'éléments** : donne le choix dans une liste de plusieurs éléments possibles.
 - L'utilisation précise dépend des indicateurs d'occurrence.
 - Les éléments-enfants doivent être déclarés dans la **DTD**.
 - Cette liste est composée d'éléments séparés par le caractère |
 - `<!ELEMENT elt0 (elt1 | elt2 | elt3) >`

✓ Exemple d'utilisation valide :

```
<elt0>  
  <elt2>(…)</elt2>  
</elt0>
```

✗ Exemple d'utilisation non valide :

```
<elt0>  
  <elt2>(…)</elt2>  
  <elt3>(…)</elt3>  
</elt0>
```

Déclarations d'éléments

- **Élément à contenu mixte :**

- il peut contenir aussi bien du **texte** que des **éléments-enfants**.
- Il se présente comme une liste de choix, avec des indicateurs d'occurrence bien choisis.
- Le texte contenu peut se trouver à n'importe quel endroit dans l'élément, et peut être une section **CDATA** .

<!ELEMENT citation (#PCDATA | auteur)* >

- Exemple d'utilisation :

<citation>

<auteur>Shakespeare</auteur>Être ou ne pas être

</citation>

- La seule syntaxe possible dans la déclaration commence par **#PCDATA**.

<!ELEMENT citation (auteur|#PCADATA)*>

INCORRECT.

KICHLISMAIL

Déclarations d'éléments

✓ **Bref** : le contenu d'un élément peut être un

Texte	#PCDATA (Parced Charater Data)	<!ELEMENT prenom (#PCDATA)>
Vide	Empty	<!ELEMENT BR EMPTY>
Aléatoire	ANY : L'élément peut contenir n'importe quel élément présent dans la DTD	<!ELEMENT adresse ANY>
Séquence d'éléments	Suite de noms d'éléments séparés par virgules. L'ordre doit être respecté	<!ELEMENT nomCompleet (nom,prenom)>
Choix	Suite de noms d'éléments séparés par des barres verticales " "	<!ELEMENT nomCompleet (nom prenom)>
Mixte	Contenu variable	<!ELEMENT alias (#PCDATA nom)*>

Déclarations d'éléments

- **Indicateur d'occurrence** : Lors de la déclaration de séquence ou de choix d'éléments, à chaque élément enfant peut être attribuée une indication d'occurrence (**?**, **+** ou *****).

<!ELEMENT elt0 (elt1, elt2?, elt3+, elt4*) >

- **elt1** ne comprend aucune indication d'occurrence. Il doit donc apparaître une seule et unique fois dans l'élément **elt0**;
- **elt2** a pour indication d'occurrence **?**. Cela signifie que l'élément doit apparaître au maximum une fois (il peut ne pas apparaître du tout);
- **elt3** a pour indication d'occurrence **+**. Cela signifie que l'élément doit apparaître au moins une fois;
- **elt4** a pour indication d'occurrence *****. Cela signifie que l'élément peut apparaître autant de fois que l'auteur le désire, voire pas du tout.

"?" : 0 ou 1 occurrence,

"*" : 0 ou plus,

"+" : 1 ou plus.

RICH ISMAIL

Déclarations d'éléments

Exemple 1 :

- ❑ Les indicateurs d'occurrence peuvent être utilisés en conjonction avec les séquences ou les choix d'éléments. Ainsi...

<!ELEMENT elt0 (elt1+, elt2*, elt3?) >

- ... permet d'indiquer une séquence composée d'au moins un élément **elt1**, puis d'un nombre indéterminé d'éléments **elt2** (éventuellement nul), enfin au plus un élément **elt3**.

- ❑ Exemple d'utilisation d'un choix d'éléments avec indicateurs d'occurrence par élément :

<!ELEMENT elt0 (elt1* | elt2* | elt3*) >

✓ Exemple d'utilisation valide :

```
<elt0>  
  <elt2>(…)</elt2>  
  <elt2>(…)</elt2>  
</elt0>
```

✗ Exemple d'utilisation non valide :

```
<elt0>  
  <elt3>(…)</elt3>  
  <elt2>(…)</elt2>  
</elt0>
```

Déclarations d'éléments

Exemple 2 :

<!ELEMENT agenda (contact)* >	Définition d'un élément agenda composé de zéro ou plus éléments contact.
<!ELEMENT contact (nom, prenom?, telephone+, image)>	Définition d'un élément contact, composé de trois éléments : nom, prenom (0 ou 1 fois = facultatif), telephone (au moins 1 fois)
<!ELEMENT nom (#PCDATA)>	Définition d'un élément nom contenant une chaîne de caractère
<!ELEMENT prenom (#PCDATA)> <!ELEMENT telephone (#PCDATA)>	Idem pour élément prenom Idem pour élément telephone
<!ELEMENT image EMPTY>	Définition d'un élément image de contenu vide

Déclarations d'éléments

Exemple 3 :

```
<!ELEMENT DEVIS (IDENTIFIANT, STATUT, MONTANT, ARTICLE+)>
<!ELEMENT IDENTIFIANT (#PCDATA)>
<!ELEMENT STATUT (#PCDATA)>
<!ELEMENT MONTANT (#PCDATA )>
<!ATTLIST MONTANT devise CDATA #REQUIRED>
<!ELEMENT ARTICLE (REFERENCE, QUANTITE, PRIX_UNITAIRE, IMAGE+)>
<!ELEMENT REFERENCE (#PCDATA)>
<!ELEMENT QUANTITE (#PCDATA)>
<!ELEMENT PRIX_UNITAIRE (#PCDATA)>
<!ATTLIST PRIX_UNITAIRE devise CDATA #REQUIRED>
<!ELEMENT IMAGE EMPTY>
<!ATTLIST IMAGE href CDATA #REQUIRED>
```

Déclaration d'un élément dont le contenu est une suite d'éléments

Déclaration d'un élément dont le contenu est une suite de caractères

Déclaration d'un élément vide

Déclaration de liste d'attributs

Déclarations d'éléments

- Exemple d'utilisation d'un choix d'éléments avec indicateur d'occurrence global :

<!ELEMENT elt0 (elt1 | elt2 | elt3)* >

- ✓ Exemple d'utilisation valide :

<elt0>

<elt2>(…)</elt2>

<elt3>(…)</elt3>

<elt1>(…)</elt1>

</elt0>

Dans ce dernier cas, il n'y a pas de contrainte visible sur l'ordre d'apparition et le nombre des éléments. C'est la déclaration la plus souple possible.

Exercice 1 :

- ☐ Rédiger une **DTD** pour une bibliographie. Cette bibliographie :
 - Contient des livres et des articles ;
 - Les informations nécessaires pour un livre sont :
 - ✓ son titre général ;
 - ✓ les noms des auteurs ;
 - ✓ ses tomes et pour chaque tome, leur nombre de pages ;
 - ✓ des informations générales : l'éditeur, le lieu d'édition, l'année d'impression, son numéro ISBN ;
 - Les informations nécessaires pour un article sont :
 - ✓ son titre ;
 - ✓ les noms des auteurs ;
 - ✓ ses références de publication : nom du journal, numéro des pages, année de publication et numéro du journal
 - On réservera aussi un champ optionnel pour un avis personnel.
- ☐ Tester cette **DTD** avec un document **XML**.

Solution :

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<!ELEMENT biblio (livre|article)*>
<!ELEMENT livre (titre, auteur+, tome*, edition, avis?)>
<!ELEMENT titre (#PCDATA)>
<!ELEMENT auteur (#PCDATA)>
<!ELEMENT tome (nb_pages)>
<!ELEMENT nb_pages (#PCDATA)>
<!ELEMENT edition (editeur, lieu_edition, annee_impression, isbn)>
<!ELEMENT editeur (#PCDATA)>
<!ELEMENT lieu_edition (#PCDATA)>
<!ELEMENT annee_impression (#PCDATA)>
<!ELEMENT isbn (#PCDATA)>
<!ELEMENT avis (#PCDATA)>
<!ELEMENT article (titre, auteur+, journal)>
<!ELEMENT journal (nom_journal, page, num_journal, annee)>
<!ELEMENT nom_journal (#PCDATA)>
<!ELEMENT page (#PCDATA)>
<!ELEMENT num_journal (#PCDATA)>
<!ELEMENT annee (#PCDATA)>
```

[link](#)

Déclarations d'attributs

Le **DTD** permet aussi de définir des contraintes sur les attributs des éléments d'un document **XML**.

- Déclarer et attacher à un élément donné chaque attribut séparément (**Non Recommandé**)
- Les réunir sous la forme d'une liste. (**Recommandé**)
- Chaque attribut défini dans la liste possède un **nom** et un **type**.
- On peut lui donner une **valeur par défaut**, ou bien indiquer qu'il est **obligatoire**.
- Le mot-clef de cette déclaration est **ATTLIST**.

<!ATTLIST nom_element nom_attribut type mode>

Déclarations d'attributs

❑ Type chaîne de caractères :

- Il s'agit du type d'attribut le plus courant.
- Une chaîne de caractères peut être composée de caractères ainsi que **d'entités** analysables.
- Le mot-clef utilisé pour la déclaration de chaîne de caractère est **CDATA**.

– Exemple de déclaration de **CDATA** :

```
<!ELEMENT elt (...) >
```

```
<!ATTLIST elt attr CDATA>
```

– Exemples d'utilisations :

```
<elt attr="Chaîne de caractères">(...)</elt>
```

Déclarations d'attributs

❑ Valeurs par défaut :

- Chaque attribut peut être **obligatoire**, optionnel ou **fixe** et avoir une **valeur par défaut**.
- Les exemples suivants montrent la déclaration d'un attribut appelé **attr** attaché à un élément nommé **elt**.

– Déclaration d'un attribut avec une valeur par défaut :

```
<!ELEMENT elt (...) >
```

```
<!ATTLIST elt attr CDATA "valeur" >
```

Un tel attribut n'est pas obligatoire. S'il est omis dans le fichier XML lors de l'utilisation de l'élément **elt**, il est considéré comme valant **valeur**.

Dans cet exemple, si on écrit : **<elt>(...)</elt>**,

cela est équivalent à écrire : **<elt attr="**valeur**">(...)>/elt>**.

Déclarations d'attributs

- Déclaration d'un attribut requis :

<!ELEMENT elt (...) >

<!ATTLIST elt attr CDATA #REQUIRED >

Un tel attribut est obligatoire. Son absence déclenche une erreur du vérificateur syntaxique sur le fichier XML.

- Déclaration d'un attribut optionnel :

<!ELEMENT elt (...) >

<!ATTLIST elt attr CDATA #IMPLIED >

- Déclaration d'un attribut avec une valeur fixe :

<!ELEMENT elt (...) >

<!ATTLIST elt attr CDATA #FIXED "valeur" >

Cette fonctionnalité est utile lors d'une mise à jour d'une DTD, pour anticiper la compatibilité avec des versions ultérieures.

DTD : Déclarations d'attributs

Déclarations d'attributs

Exemple :

Type	DTD	XML
Attribut obligatoire	<pre><!ELEMENT nom_elt EMPTY> <!ATTLIST nom_elt attr CDATA #REQUIRED></pre>	<code><nom_elt attr= "valeur"/></code>
Attribut non obligatoire	<pre><!ELEMENT nom_elt Empty> <!ATTLIST nom_elt attr CDATA #IMPLIED></pre>	<pre><nom_elt attr= "v2"/> Ou bien : <nom_elt/></pre>
Attribut à valeur fixe	<pre><!ELEMENT nom_elt EMPTY> <!ATTLIST nom_elt attr CDATA #FIXED "4"></pre>	<code><nom_elt attr="x"/></code>

DTD : Déclarations d'attributs

Déclarations d'attributs

❑ Data Type

- ✓ **PCDATA** : **P**arsed **C**haracter **D**ATA (ou chaîne de caractères **parsée**):

Déclaration : `<!ELEMENT elt (#PCDATA) >`

Exemple : `<elt>texte & texte</elt>`

"texte & texte" sera analysé par l'analyseur syntaxique (**parser**). Il recherchera les entités et les balises.

Resultat : `texte & texte`

- ✓ **CDATA** : **C**haracter **D**ATA ou chaîne de caractères :

`<elt attribut="pa -ra"/>`

"pa -ra" est du type **CDATA** et ne sera pas analysée elle est donc traitée comme une chaîne de caractère, et non une balise **XML**.

`PCDATA(pa-ra)` → `pa-ra`

`CDATA(pa-ra)` → `pa-ra`

Déclarations d'attributs

□ Types ID :

- Indique que l'attribut en question peut servir **d'identifiant** dans le fichier **XML**.
 - Deux éléments ne pourront pas posséder le même attribut avec la même valeur.
- Exemple de déclaration de type ID optionnel :

```
<!ELEMENT elt1 (...) >  
  <!ATTLIST elt1 attr ID #IMPLIED >  
<!ELEMENT elt2 (...) >  
  <!ATTLIST elt2 attr ID #IMPLIED >  
<!ELEMENT elt3 (...) >  
  <!ATTLIST elt3 attr ID #IMPLIED >
```

La déclaration précédente interdit par exemple...

```
<elt1 attr="machin"> </elt1>  
<elt2 attr="truc"> </elt2>  
<elt3 attr="machin"> </elt3>
```

... ainsi que

```
<elt1 attr="machin"> </elt1>  
<elt2 attr="machin"> </elt2>  
<elt3 attr="truc"> </elt3>
```

C'est par exemple le cas de l'attribut **id** en **HTML**.

Déclarations d'attributs

❑ Type énuméré :

- Limiter la liste de valeurs possibles pour un attribut.
- Donner une autre valeur dans le fichier XML provoque une erreur.

– Exemple de déclaration d'une liste de choix d'attributs :

```
<!ELEMENT img EMPTY >
```

```
<!ATTLIST img format (GIF | JPEG | PNG) "PNG" >
```

- ✓ Cet exemple déclare un attribut **format** d'un élément **img**. La valeur de cet attribut peut être **PNG**, **GIF** ou **JPEG**.
- ✓ Si aucune valeur n'est affectée à cet attribut, c'est la valeur par défaut qui le sera, ici **PNG**.
- ✓ Remarque : On notera l'absence de guillemets dans la liste des valeurs possibles.

Déclarations d'attributs

- ❑ Utilisation de liste pour les attributs :

```
<!ELEMENT elt (...) >
```

```
<!ATTLIST elt
```

```
    attr1 CDATA #IMPLIED
```

```
    attr2 CDATA #REQUIRED
```

```
>
```

Les retours à la ligne sont optionnels, et la mise en page n'est pas contrainte.

DTD : Déclarations d'attributs

Exercice :

- ☐ Modifier la **DTD** précédente...
 - En ajoutant un attribut optionnel **soustitre** à l'élément **titre** ;
 - En faisant de l'élément **tome** un élément **vide** et en lui ajoutant un attribut requis **nb_pages** et un attribut optionnel **soustitre** ;
 - en faisant de l'élément **nom_journal** un attribut de l'élément **journal** et en lui donnant comme valeur par défaut **Feuille de Chou** ;
 - En faisant de l'élément **annee** un attribut de type énuméré, prenant comme valeurs possibles **2000**, **2001**, **2002**, **"avant_2000"** et **"inconnue"** et proposant comme valeur par défaut **inconnue**.

- ☐ Utiliser cette **DTD** pour créer un fichier **XML** valide.

DTD : Déclarations d'attributs

Solution :

```
<?xml version="1.0" encoding="ISO-8859-1" ?>

<!ELEMENT biblio (livre|article)*>
  <!ELEMENT livre (titre, auteur+, tome*, edition, avis?)>
    <!ATTLIST titre soustitre CDATA #IMPLIED>
    <!ELEMENT titre (#PCDATA)>
    <!ELEMENT auteur (#PCDATA)>
    <!ELEMENT tome EMPTY>
      <!ATTLIST tome
        nb_pages CDATA #REQUIRED
        soustitre CDATA #IMPLIED>
    <!ELEMENT edition (editeur, lieu_edition, lieu_impression,
isbn)>
      <!ELEMENT editeur (#PCDATA)>
      <!ELEMENT lieu_edition (#PCDATA)>
      <!ELEMENT lieu_impression (#PCDATA)>
      <!ELEMENT isbn (#PCDATA)>
      <!ELEMENT avis (#PCDATA)>
```

DTD : Déclarations d'attributs

Solution suite :

```
<!ELEMENT article (titre, auteur+, journal)>
  <!ELEMENT journal (page, num_journal)>
    <!ATTLIST journal
      nom_journal CDATA "Feuille de Chou"
      annee (2000 | 2001 | 2002 | avant_2000 | inconnue) "inconnue">
  <!ELEMENT page (#PCDATA)>
  <!ELEMENT num_journal (#PCDATA)>
  <!ELEMENT annee (#PCDATA)>
```

Exercice 2 :

Donner une **DTD** qui valide des documents sous forme de **carnets d'adresses** pour des personnes.

☐ Un carnet d'adresse a la forme suivante :

- ✓ Chaque personne doit avoir un nom et un prénom,
- ✓ chaque personne possède un identifiant unique (attribut de type **ID**),
- ✓ Pour chaque personne on peut spécifier son sexe dans une liste de choix (M | F) (attribut **facultatif**),
- ✓ Chaque personne peut avoir un email ou plusieurs (**optionnel**).

☐ Utiliser cette **DTD** pour créer un fichier **XML** valide.

Solution : Fichier DTD

```
<?xml version="1.0" encoding="UTF-8"?>  
  
<!ELEMENT carnetAdresse (personne)+>  
  
<!ELEMENT personne (nom, prenom, email*)>  
  
<!ELEMENT nom (#PCDATA)>  
  
<!ELEMENT prenom (#PCDATA)>  
  
<!ELEMENT email (#PCDATA)>  
  
<!ATTLIST personne id ID #REQUIRED sexe (M | F) #IMPLIED>
```

Solution : Fichier XML

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE carnetAdresse SYSTEM "carnetsAdresse.dtd">
<carnetAdresse>
  <personne id="p1" sexe="M">
    <nom>Ait Ali</nom>
    <prenom>Ali</prenom>
    <email>ali@gmail.com</email>
  </personne>
  <personne id="p2" sexe="F">
    <nom>Ait Ahmed</nom>
    <prenom>Fatima</prenom>
    <email>Fatima@gmail.com</email>
    <email>Fatima2@gmail.com</email>
  </personne>
</carnetAdresse>
```

Déclarations d'entités

L'**entité** associe un **nom** à une **valeur**. Ce nom est employé dans le document **XML** comme une forme d'alias ou de raccourci vers la valeur suivant la syntaxe **&nom;**

La valeur d'une entité peut être **interne** ou **externe**.

□ Entité interne:

– Déclaration :

```
<!ENTITY nom "VALEUR">
```

– Exemple :

- Dans le **DTD**, on définit l'entité :

```
<!ENTITY T2 "TDI 202 Khémisset">
```

- Dans **XML**, on utilise l'entité déjà définie :

```
<PARA>Bonjour les stagiaires de &T2; </PARA>
```

Équivalent à :

```
<PARA>Bonjour les stagiaires de TDI 202 Khémisset </PARA>
```

Déclarations d'entités

- ❑ **Entité externe** : Les mots-clés **SYSTEM** et **PUBLIC** servent à réaliser un lien vers une valeur présente dans un fichier externe.

– **Exemple** :

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE book [
  <!--Entités externes -->
  <!ENTITY chapter1 SYSTEM "chapter1.xml">
  <!ENTITY chapter2 SYSTEM "chapter2.xml"> ]>
<book>
  <!--Inclusion du fichier chapter1.xml --> &chapter1;
  <!--Inclusion du fichier chapter2.xml --> &chapter2;
</book>
```

Déclarations d'entités

- ❑ **Entité paramétrique** : peut servir à la réalisation de la DTD pour limiter les répétitions de blocs de définition (par exemple, un attribut présent dans plusieurs éléments)..

- **Déclaration :**

<!ENTITY %nom "VALEUR">

L'instruction **%nom**; sert à utiliser une entité paramétrique dans la **DTD**.

- **Exemple :**

- Définition de l'entité :

<!ENTITY %TD "CDATA #REQUIRED">

- Utilisation de l'entité :

<!ATTLIST chapitre titre %TD;>

Inconvénients des DTD

- ❖ Difficile à lire
- ❖ Non extensible (ce n'est pas un document XML).
- ❖ Ne permet pas de typer les données
- ❖ Ne peut prendre en compte qu'un seul espace de nom : ce n'est pas possible d'importer des définitions de balises définies par ailleurs dans un fichier XML défini par une DTD.

Apports des schémas XSD

- ❖ XML Schéma propose des nouveautés en plus des fonctionnalités fournies par les DTD :
- ❖ Le typage des données est introduit, ce qui permet la gestion de booléens, d'entiers, d'intervalles de temps... Il est même possible de créer de nouveaux types à partir de types existants.
- ❖ La notion d'héritage. Les éléments peuvent hériter du contenu et des attributs d'un autre élément.
- ❖ Le support des espaces de nom.
- ❖ Les indicateurs d'occurrences des éléments peuvent être tout nombre non négatif (rappel : dans une DTD, on est limité à 0, 1 ou un nombre infini d'occurrences pour un élément).
- ❖ Les schémas sont très facilement concevables par modules.

Structure du document XML Schema

Exemple :

Le document XML :

```
<entree>
  <nom>Harry Cover</nom>
  <telephone>0102030405</telephone>
</entree>
```

Le document XML-Schema correspondant :

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema" >
  <xsd:element name="entree">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="nom" type="xsd:string"
          minoccurs="1" maxoccurs="1"/>
        <xsd:element name="telephone" type="xsd:decimal"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
```

Entete

Définition de la balise complexe *entree*

Définition de la balise String *nom*

Définition de la balise de type Décimal *telephone*

Structure du document XML Schema

Un schéma xml est un fichier xml avec extension **".xsd"**, comme tout document xml, un **Schéma XML** commence par un **prologue**, et un **élément racine**.

```
<?xml version="1.0" encoding="ISO-8859-1"?>
```

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
```

```
<!-- déclarations d'éléments, d'attributs et de types ici -->
```

```
</xsd:schema>
```

- ✓ L'élément racine est l'élément **xsd:schema**
- ✓ tout élément d'un schéma doit commencer par le préfixe **xsd**.

Déclarations d'éléments

- ❑ Un élément, dans un schéma, se déclare avec la balise **<xs:element>**,

Syntaxe :

```
<xsd:element name="nom_element" type="type_element">
```

Exemple :

```
<xsd:element name="date-naissance" type="xsd:date">
```

```
<xsd:element name="nombre-enfants" type="xsd:integer">
```

Déclarations d'éléments

- ❑ Les éléments peuvent être de type **simple** ou **complexe**,

Exemple :

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="contacts" type="typeContacts"/>
  <xsd:element name="remarque" type="xsd:string"/>
  <!-- déclarations de types ici -->
</xsd:schema>
```

- Le schéma déclare deux éléments : un élément **contacts** et un élément **remarque**.
- Chaque élément est « **typé** » :
 - ✓ L'élément **contacts** est ainsi du type **typeContacts**, qui est un **type complexe** défini par l'utilisateur.
 - ✓ L'élément **remarque** quant à lui est du type **xs:string** qui est un **type simple** prédéfini de **XML Schema**.

Eléments de type simple

- ❑ Un **élément simple** est un élément **XML** qui ne peut contenir que du texte. Il ne peut contenir aucun autre élément ou attribut.
- ❑ Le texte peut être de plusieurs types différents : **booléen**, **chaîne**, **date**, **etc...**,
- ❑ Le texte peut s'agir aussi d'un type personnalisé qu'on peut définir nous-même : **Restrictions**, **Enumération**, **Extension**, **union** ou **list**

Eléments de type simple

➤ Document **XML**

```
<lastname>Ali</lastname>  
<age>34</age>  
<dateborn>1968-03-27</dateborn>
```

➤ Schéma **xsd** associé

```
<xs:element name="lastname" type="xs:string"/>  
<xs:element name="age" type="xs:integer"/>  
<xs:element name="dateborn" type="xs:date"/>
```

les types simples prédéfinis:

string	Chaîne de caractère
byte	-1, 126
unsignedByte	0, 126
integer	-126789, -1, 0, 1, 126789
positiveInteger	1, 126789
negativeInteger	-126789, -1
nonNegativeInteger	0, 1, 126789
nonPositiveInteger	-126789, -1, 0
int	-1, 126789675
unsignedInt	0, 1267896754

les types simples prédéfinis:

long	-1, 12678967543233
unsignedLong	0, 12678967543233
short	-1, 12678
unsignedShort	0, 12678
decimal	-1.23, 0, 123.4, 1000.00
float	-INF, -1E4, -0, 0, 12.78E-2, 12, INF, NaN
double	-INF, -1E4, -0, 0, 12.78E-2, 12, INF, NaN
boolean	true, false 1, 0
time	13:20:00.000, 13:20:00.000-05:00

les types simples prédéfinis:

dateTime	1999-05-31T13:20:00.000-05:00
date	1999-05-31
gYear	1999
gMonth	05
gYearMonth	1999-02
gDay	31
gMonthDay	05-31
Name	shipTo
anyURI	http://www.example.com/

les types simples prédéfinis:

string	Chaine de caractères
normalizedString	pas de : tabulation / de saut de ligne / de retour chariot
Name	Nom XML
language	Code de langue comme fr, en-GB...
anyURI	comme http://www.istanticsafi.ma/~cours/

Eléments de type simple

- Valeur par **défaut** et valeur **fixée**

- ✓ Un élément simple peut avoir une valeur par défaut ou une valeur fixée

- Une valeur par **défaut** est assignée si aucune autre valeur n'est affectée

```
<xs:element name="color" type="xs:string" default="red"/>
```

- Une valeur **fixée** est aussi assignée mais peut être changée

```
<xs:element name="color" type="xs:string" fixed="red"/>
```

Élément de type simple

❑ Restriction sur les valeurs

- Des restrictions sont utilisées sur les éléments pour ne tolérer que des valeurs acceptables
- Les restrictions en XML sont appelées facettes
- Restriction à des valeurs : la valeur de l'élément age doit être comprise entre 0 et 100.

```
<xs:element name="age">  
  <xs:simpleType>  
    <xs:restriction base="xs:integer">  
      <xs:minInclusive value="0"/>  
      <xs:maxInclusive value="100"/>  
    </xs:restriction>  
  </xs:simpleType>  
</xs:element>
```

Éléments de type simple

❑ Restriction sur une modèle (pattern)

- Restriction de "letter" à des lettres minuscules :

```
<xs:element name="letter">  
  <xs:simpleType>  
    <xs:restriction base="xs:string">  
      <xs:pattern value="[A-Z]"/>  
    </xs:restriction>  
  </xs:simpleType>  
</xs:element>
```

- Restriction de "initials" à trois lettres majuscules :

```
<xs:element name="initials">  
  <xs:simpleType>  
    <xs:restriction base="xs:string">  
      <xs:pattern value="[A-Z][A-Z][A-Z]"/>  
    </xs:restriction>  
  </xs:simpleType>  
</xs:element>
```

Éléments de type simple

❑ Restriction sur une modèle (pattern)

➤ Restriction sur une liste de choix:

```
<xs:element name="sexe">  
  <xs:simpleType>  
    <xs:restriction base="xs:string">  
      <xs:pattern value="Homme|Femme"/>  
    </xs:restriction>  
  </xs:simpleType>  
</xs:element>
```

XSD : XML Schema

❑ Résumé des contraintes sur les types de données

Constraint	Description
enumeration	Defines a list of acceptable values
fractionDigits	Specifies the maximum number of decimal places allowed. Must be equal to or greater than zero
length	Specifies the exact number of characters or list items allowed. Must be equal to or greater than zero
maxExclusive	Specifies the upper bounds for numeric values (the value must be less than this value)
maxInclusive	Specifies the upper bounds for numeric values (the value must be less than or equal to this value)
maxLength	Specifies the maximum number of characters or list items allowed. Must be equal to or greater than zero
minExclusive	Specifies the lower bounds for numeric values (the value must be greater than this value)
minInclusive	Specifies the lower bounds for numeric values (the value must be greater than or equal to this value)
minLength	Specifies the minimum number of characters or list items allowed. Must be equal to or greater than zero
pattern	Defines the exact sequence of characters that are acceptable
totalDigits	Specifies the exact number of digits allowed. Must be greater than zero
whiteSpace	Specifies how white space (line feeds, tabs, spaces, and carriage returns) is handled

Éléments de type simple

❑ Restriction on longueur

- Pour fixer la longueur de la valeur d'un élément, nous devons utiliser la contrainte **length**,

Exemple : "password" avec restriction (la valeur doit être exactement 8 caractères)

```
<xs:element name="password">  
  <xs:simpleType>  
    <xs:restriction base="xs:string">  
      <xs:length value="8"/>  
    </xs:restriction>  
  </xs:simpleType>  
</xs:element>
```

Éléments de type simple

❑ Restriction on longueur

- Pour limiter la longueur de la valeur d'un élément, nous devons utiliser les contraintes **maxLength** et **minLength**,

Exemple : "password" avec restriction (5 caractères au minimum et 8 au maximum)

```
<xs:element name="password">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:maxLength value="8"/>
      <xs:minLength value="5"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```


Éléments de type simple

❑ Enumération

- Pour limiter l'étendue d'un élément de **XML** à un ensemble de valeurs acceptables, on a l'habitude d'utiliser **l'énumération**
- Cet exemple montre l'élément "car" avec restriction à "Audi", "Golf" et "BMW"

```
<xs:element name="car">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:enumeration value="Audi"/>
      <xs:enumeration value="Golf"/>
      <xs:enumeration value="BMW"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

Éléments de type simple

❑ Enumération

- On peut également l'écrire comme suit :

```
<xs:element name="car" type="carType"/>
```

```
<xs:simpleType name="carType">  
  <xs:restriction base="xs:string">  
    <xs:enumeration value="Audi"/>  
    <xs:enumeration value="Golf"/>  
    <xs:enumeration value="BMW"/>  
  </xs:restriction>  
</xs:simpleType>
```

- Dans ce cas-ci le type "carType" peut être employé par d'autres éléments parce que ce n'est pas une partie de l'élément "car" (type simple globale)

Déclarations d'attributs

- ❑ Un attribut est toujours de **type simple**,
- ❑ L'attribut peut être **global** et donc réutilisable au sein de plusieurs définitions de type complexe
- ❑ Seuls les éléments complexes peuvent avoir des attributs

Syntaxe :

```
<xs:attribute name="nom_attribut" type="type_attribut"/>
```

Exemple :

```
<xs:attribute name="langue" type="xsd:string"/>
```

```
<xs:attribute name="categorie" type="xsd:string"/>
```

Déclaration d'attributs

Exemple :

➤ élément XML

```
<livre lang="EN">Smith</livre>
```

➤ Voici la définition correspondante de type simple

```
<xs:element name="livre">
```

```
  <xs:complexType>
```

```
    <xs:attribute name="lang" type="xs:string"/>
```

```
  </xs:complexType>
```

```
</xs:element>
```

Déclaration d'attributs

- On peut également affecter **une valeur par défaut**

✓ `<xs:attribute name="lang" type="xs:string" default="EN"/>`

- On peut également affecter **une valeur fixe**

✓ `<xs:attribute name="lang" type="xs:string" fixed="EN"/>`

- Tous les attributs sont **optionnels** par défaut :

✓ `<xs:attribute name="lang" type="xs:string" use="optional"/>`

- On peut les rendre **obligatoires**

✓ `<xs:attribute name="lang" type="xs:string" use="required"/>`

Eléments de type complexe

- ❑ Un élément est dit de type **complexe** s'il contient autres éléments (enfants) ou/et a des attributs.

- ❑ Il existe quatre types d'éléments **complexes**:

- ✓ éléments **vides**,

```
<module code="M202"/>
```

- ✓ éléments qui ne contiennent que d'autres éléments,

```
<formateur>  
  <nom>Ali</nom>  
  <prenom>Amir Dine</prenom>  
</ formateur >
```

Eléments de type complexe

- ✓ éléments contenant du texte et des attributs,

```
<examen type="pratique">  
    Programmation web  
</examen>
```

- ✓ éléments contenant à la fois d'autres éléments et du texte.

```
<description>cours fait  
    à <dateSeance>2011-05-20</ dateSeance>  
</description>
```

Eléments de type complexe

- ❑ Pour définir un type complexe, on peut utiliser la balise **complexType** :

```
<xsd:complexType ...>  
    ...  
</xsd:complexType>
```

- ❑ Le type peut être **global** ou **local**.

Eléments de type complexe

- ❑ Le type **local** est un type **anonyme**,

Exemple :

```
<xsd:element name="formateur">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="nom" type="xsd:string"/>
      <xsd:element name="prenom" type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
```

Eléments de type complexe

- ❑ Le type **global** est associé à un **nom**,
 - Il permet de réutiliser le type dans un autre élément,
 - Il permet aussi de déclarer un élément complexe baser sur un élément complexe existant et ajouter d'autres éléments (notion **d'extension**).

Exemple :

```
<xsd:element name="formateur" type="infosFormateur"/>
<xsd:complexType name="infosFormateur">
  <xsd:sequence>
    <xsd:element name="nom" type="xsd:string"/>
    <xsd:element name="prenom" type="xsd:string"/>
  </xsd:sequence>
</xsd:complexType>
```

Eléments de type complexe

Exemple : Déclaration de "personinfo" (type globale) réutilisable dans différents elements

```
<xs:element name="employee" type="personinfo"/>
<xs:element name="student" type="personinfo"/>
<xs:element name="member" type="personinfo"/>

<xs:complexType name="personinfo">
  <xs:sequence>
    <xs:element name="firstname" type="xs:string"/>
    <xs:element name="lastname" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

XSD : XML Schema

Exemple 2 : On peut également se baser sur un élément d'un type complexe existant et lui ajouter des éléments :

```
<xs:element name="employee" type="fullpersoninfo"/>
<xs:complexType name="personinfo">
  <xs:sequence>
    <xs:element name="firstname" type="xs:string"/>
    <xs:element name="lastname" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
<xs:complexType name="fullpersoninfo">
  <xs:complexContent // définit des restrictions sur complexType
    <xs:extension base="personinfo">
      <xs:sequence>
        <xs:element name="ville" type="xs:string"/>
        <xs:element name="pays" type="xs:string"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

➤ **"employee"** doit contenir, dans l'ordre : **"firstname"**, **"lastname"**, **"ville"** et **"pays"**

Eléments de type complexe

❑ Type complexe vide

- ✓ Un élément complexe vide ne peut pas avoir de contenu, mais il a des attributs.

Exemple :

- Element XML : `<product prodId="1032"/>`
- Déclaration xsd :

```
<xs:element name="product">  
  <xs:complexType>  
    <xs:attribute name="prodId" type="xs:positiveInteger"/>  
  </xs:complexType>  
</xs:element>
```

ou bien : (déclaration global)

```
<xs:element name="product" type="prodtype"/>  
<xs:complexType name="prodtype">  
  <xs:attribute name="prodId" type="xs:positiveInteger"/>  
</xs:complexType>
```

Eléments de type complexe

- ❑ On peut aussi contrôler COMMENT les éléments doivent être utilisés dans les documents avec des indicateurs :
 - ✓ Indicateurs d'ordre :
 - **sequence**,
 - **choice**,
 - **All**.
 - ✓ Indicateurs d'occurrences :
 - **maxOccurs**,
 - **minOccurs**.
 - ✓ Indicateurs de groupes :
 - **xs:group**,
 - **xs:attributeGroup**.

Eléments de type complexe

✓ Indicateurs d'ordre :

- **Opérateurs de séquence** : **xs:sequence** définit un nouveau type formé d'une suite des éléments en séquence ordonnée.
 - C'est l'équivalent de l'opérateur **" , "** des **DTD**.

```
<xsd:element name="livre">  
  <xsd:complexType>  
    <xsd:sequence>  
      <xsd:element name="titre" type="xsd:string"/>  
      <xsd:element name="auteur" type="xsd:string"/>  
      <xsd:element name="isbn" type="xsd:string"/>  
    </xsd:sequence>  
  </xsd:complexType>  
</xsd:element>
```

XSD

```
<!ELEMENT livre (titre, auteur, isbn)>  
<!ELEMENT titre (#PCDATA)>  
<!ELEMENT auteur (#PCDATA)>  
<!ELEMENT isbn (#PCDATA)>
```

DTD

Eléments de type complexe

✓ Indicateurs d'ordre :

- **Opérateurs de choix** : **xsd:choice** définit un nouveau type formé d'une suite des éléments énumérés de choix.
- C'est l'équivalent de l'opérateur "**|**" des **DTD**..

```
<xsd:element name="publication">
  <xsd:complexType>
    <xsd:choice>
      <xsd:element name="livre" type="xsd:string"/>
      <xsd:element name="article" type="xsd:string"/>
      <xsd:element name="rapport" type="xsd:string"/>
    </xsd:choice>
  </xsd:complexType>
</xsd:element>
```

XSD

```
<!ELEMENT publication (livre | article | rapport)>
<!ELEMENT livre      (#PCDATA)>
<!ELEMENT article    (#PCDATA)>
<!ELEMENT rapport    (#PCDATA)>
```

DTD

Eléments de type complexe

- ✓ Indicateurs d'ordre :
 - **Opérateurs d'ensemble : `xsd:all`**, est une nouveauté par rapport aux **DTD**. Il indique que les éléments enfants doivent apparaître une fois (ou pas du tout), mais dans n'importe quel ordre.

```
<xs:complexType>
  <xs:all>
    <xs:element name="nom" type="xs:string" />
    <xs:element name="prénom" type="xs:string" />
    <xs:element name="dateDeNaissance" type="xs:date" />
    <xs:element name="adresse" type="xs:string" />
    <xs:element name="adresseElectronique" type="xs:string" />
    <xs:element name="téléphone" type="numéroDeTéléphone" />
  </xs:all>
</xs:complexType>
```

Eléments de type complexe

✓ Déclaration d'attributs :

- Si un tel élément contient des attributs, alors les déclarations de ces derniers doivent se faire juste avant la balise fermante **</xs:complexType>**

```
<xs:complexType>
  <xs:sequence>
    <xs:element name="dateDeNaissance" type="xs:date" />
    <xs:element name="adresse" type="xs:string" />
    <xs:element name="adresseElectronique" type="xs:string" />
    <xs:element name="téléphone" type="numéroDeTéléphone" />
  </xs:sequence>
  <xs:attribute name="nom" />
  <xs:attribute name="prénom" />
</xs:complexType>
```

Eléments de type complexe

❑ Type complexe à contenu mixte

- ✓ Il peut contenir des attributs, des éléments et du texte

Exemple : l'élément **"letter"**

```
<letter>
```

```
  Dear Mr.<name>John Smith</name>.
```

```
  Your order <ordered>1032</ordered>
```

```
  will be shipped on <shipdate>2001-07-3</shipdate>.
```

```
</letter>
```

Eléments de type complexe

❑ Type complexe à contenu mixte

- ✓ Pour permettre à du texte d'apparaître entre les enfants-éléments de "letter", l'attribut **mixed** doit être à "true "
- ✓ L'étiquette **<xs:sequence>** signifie que les éléments définis (name, ordered et shipdate) doivent apparaître dans cet ordre à l'intérieur d'un élément "letter "

```
<xs:element name="letter">
  <xs:complexType mixed="true" >
    <xs:sequence>
      <xs:element name="name" type="xs:string"/>
      <xs:element name="orderid" type="xs:positiveInteger"/>
      <xs:element name="shipdate" type="xs:date"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Eléments de type complexe

✓ Indicateurs d'occurrence :

- L'attributs **minOccurs** permet de préciser le nombre minimal d'occurrences d'un élément ou d'un groupe.
- L'attributs **maxOccurs** permet de préciser le nombre maximal d'occurrences d'un élément ou d'un groupe.
- Ils sont l'équivalent des opérateurs **?**, ***** et **+** des DTD.
- L'attribut **minOccurs** prend un entier comme valeur.
- L'attribut **maxOccurs** prend un entier ou la chaîne **unbounded** comme valeur pour indiquer qu'il n'y a pas de nombre maximal.
- La valeur par défaut de ces deux attributs est la valeur **1**.

Eléments de type complexe

Exercice : soit en DTD :

```
<!ELEMENT elt (elt1, elt2?, elt3*)>
```

```
<!ELEMENT elt1 (#PCDATA)>
```

```
<!ELEMENT elt2 (#PCDATA)>
```

```
<!ELEMENT elt3 (#PCDATA)>
```

```
<!ATTLIST elt id ID #REQUIRED sexe CDATA #IMPLIED>
```

Donner son équivalent en **xsd**.

Eléments de type complexe

Solution :

```
<xs:element name="elt">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="elt1" type="xs:string"/>
      <xs:element name="elt2" type="xs:string" minOccurs="0"
maxOccurs="1"/>
      <xs:element name="elt3" type="xs:string" minOccurs="0"
maxOccurs="unbounded"/>
    </xs:sequence>
    <xs:attribute name="id" type="xs:int" use="required"/>
    <xs:attribute name="sexe" type="xs:string" use="optional"/>
  </xs:complexType>
</xs:element>
```

Eléments de type complexe

✓ Indicateurs de groupes :

- Il est possible de nommer des groupes d'éléments et des groupes d'attributs afin de pouvoir les réutiliser.
- Ce mécanisme aide à structurer un schéma complexe et vise à obtenir une meilleure modularité dans l'écriture des schémas.
- Le groupe d'éléments est défini par **xsd:group**
- Le groupe d'attributs est défini par **xsd:attributeGroup**.

□ **xsd:group** :

- permet de définir un groupe d'éléments dont le nom est donné par l'attribut **name**.
- Le contenu de l'élément **xsd:group** est un fragment de type nécessairement inclus dans un élément **xsd:sequence**, **xsd:choice** ou **xsd:all**.
- L'utilisation d'un groupe est équivalente à l'insertion de son contenu.
- L'intérêt d'un groupe est de pouvoir l'utiliser à plusieurs reprises et de factoriser ainsi les parties communes à plusieurs types.

XSD : XML Schema

- **Exemple** : Donner un document **xml** valide selon le schéma ci-dessous :

```
<?xml version="1.0" encoding="utf-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:group name="persongroup">
    <xs:sequence>
      <xs:element name="nom" type="xs:string"/>
      <xs:element name="prenom" type="xs:string"/>
      <xs:element name="date" type="xs:date"/>
    </xs:sequence>
  </xs:group>
  <xs:element name="formateur" type="infosFormateur"/>
  <xs:complexType name="infosFormateur">
    <xs:sequence>
      <xs:group ref="persongroup"/>
      <xs:element name="matricule" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

Eléments de type complexe

❑ **xsd:attributeGroup** :

- permet de définir un groupe d'attributs dont le nom est donné par l'attribut name.
- Le contenu de l'élément **xsd:attributeGroup** est constitué de déclarations d'attributs introduites par l'élément **xsd:attribute**.

```
<xsd:attributeGroup name="personattrgroup">
  <xsd:attribute name="nom" type="xs:string"/>
  <xsd:attribute name="prenom" type="xs:string"/>
  <xsd:attribute name="date" type="xs:date"/>
</xsd:attributeGroup>

<xsd:element name="formateur">
  <xsd:complexType>
    <xsd:attributeGroup ref="personattrgroup"/>
  </xsd:complexType>
</xsd:element>
```

Déclarations d'attributs

Exemple 2 :

```
<xs:element name="elt1">
  <xs:complexType mixed="true">
    <xs:attributeGroup ref="groupe1" />
  </xs:complexType>
</xs:element>
<xs:element name="elt2">
  <xs:complexType mixed="true">
    <xs:attributeGroup ref="groupe1" />
  </xs:complexType>
</xs:element>
<xs:attributeGroup name="groupe1">
  <xs:attribute name="attr1" use="required" />
  <xs:attribute name="attr2" default="blabla" use="optional" />
  <xs:attribute name="attr3" />
  <xs:attribute name="attr4" fixed="blabla" />
</xs:attributeGroup>
```

Extension de type simple

- L'extension permet uniquement d'ajouter des attributs pour donner un type complexe à contenu simple.
 - ✓ Lors d'une extension d'un type simple, l'élément **xsd:extension** est toujours enfant d'un élément **xsd:simpleContent**.
 - ✓ Les déclarations des attributs qui sont ajoutés sont placées dans le contenu de l'élément **xsd:extension**

XSD : XML Schema

Extension de type simple

Exemple : Définition d'un type **Prix** qui étend le type prédéfini **xsd:decimal** en lui ajoutant un attribut **devise** de type **xsd:string**

```
<xsd:complexType name="Prix">
  <xsd:simpleContent>
    <xsd:extension base="xsd:decimal">
      <!-- Attribut ajouté -->
      <xsd:attribute name="devise" type="xsd:string"/>
    </xsd:extension>
  </xsd:simpleContent>
</xsd:complexType>

<xsd:element name="price" type="Prix"/>
```

XSD

```
<price devise="euro"> 15 </price>
```

XML

Extension de type complexe

□ Types complexes à contenu simple

Il est possible d'étendre un type complexe à contenu simple pour lui ajouter de nouveaux attributs.

- ✓ On obtient alors un nouveau type complexe à contenu simple qui possède les attributs du type de base et ceux déclarés par l'extension.
- ✓ L'extension d'un tel type est similaire à l'extension d'un type simple.
- ✓ L'élément **xsd:extension** est encore enfant d'un élément **xsd:simpleContent**.
- ✓ Les déclarations des attributs qui sont ajoutés sont placées dans le contenu de l'élément **xsd:extension**.

XSD : XML Schema

Extension de type complexe

Exemple :

```
<xsd:complexType name="Prix">
  <xsd:simpleContent>
    <xsd:extension base="xsd:decimal">
      <xsd:attribute name="devise" type="xsd:string"/>
    </xsd:extension>
  </xsd:simpleContent>
</xsd:complexType>
<xsd:complexType name="PrixPays">
  <xsd:simpleContent>
    <xsd:extension base="Prix ">
      <!-- Attribut ajouté -->
      <xsd:attribute name="pays" type="xsd:string"/>
    </xsd:extension>
  </xsd:simpleContent>
</xsd:complexType>

<xsd:element name="price" type="PrixPays"/>
```

XSD

```
<price devise="euro" pays="france" > 15 </price>
```

XML

Extension de type complexe

□ Types complexes à contenu complexe

L'extension d'un type complexe à contenu complexe consiste à ajouter du contenu et/ou des attributs. Le contenu est ajouté après le contenu du type de base. L'ajout d'attribut est semblable au cas des types complexes à contenu simple.

- ✓ L'élément **xsd:extension** est encore enfant d'un élément **xsd:complexContent**.

Extension de type complexe

Exemple :

```
<xs:complexType name="nomType">
  <xs:sequence>
    <xs:element name="nom" type="xs:string"/>
    <xs:element name="prenom" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

```
<xs:complexType name=" nomType1 ">
  <xs:complexContent>
    <xs:extension base=" nomType">
      <xs:attribute name="age" type="xs:float"/>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

<xs:element name="formateur" type=" nomType1"/>
```

```
<formateur age="15.6">
  <nom>AIT ALI</nom>
  <prenom>ALI</prenom>
</formateur>
```

XML

XSD